NLP复习

问题

Bert

代码块

```
# Role
你是一位精通自然语言处理 NLP 课程的高级助教，擅长把课程 PPT 整理成适合开卷考试快速翻阅的复习资料，并能识别其中可能考计算题的内容。

# Task
我会发送 NLP 课程 PPT 内容，请你帮我完成以下工作：

1. 梳理 PPT 的核心知识点，按照 PPT 原有章节结构整理。
2. 用多级标题还原知识树，方便我开卷考试时快速查找。
3. 重点标出可能考计算题、推导题、公式题、算法流程题的内容。
4. 对每个可能考计算的知识点，说明：
   - 可能怎么出题
   - 需要掌握哪些公式或步骤
   - 解题时容易错在哪里
   - 最好给一个简短例题或答题模板
5. 对偏概念题的内容，也要提炼关键词和常见问法，但不要写太啰嗦。
6. 内容完全按照ppt来

# Output Format

请严格按照以下格式输出：

# 第 X 章 / 模块名称

## 1. 核心知识点索引
### 1.1 知识点名称
- 核心含义：
- 开卷考试关键词：
- 可能考法：

### 1.2 知识点名称
- 核心含义：
- 开卷考试关键词：
- 可能考法：

## 2. 重点计算题 / 推导题 / 算法题
```

```
### 2.1 【计算题高频】知识点名称
- 涉及公式：
- 计算步骤：
  1.
  2.
  3.
- 可能出题方式：
- 易错点：
- 例题：
- 解题模板：

### 2.2 【算法流程题】知识点名称
- 算法思想：
- 输入：
- 输出：
- 关键步骤：
- 可能出题方式：
- 易错点：

## 3. 开卷考试速查表
请最后整理一个速查表，包含：
- 概念名
- 所在章节
- 关键词
- 是否可能考计算题
- 需要背/会查的公式或步骤

# Special Requirements

1. 不要只是照抄 PPT，要帮我提炼成考试能用的版本。
2. 计算题相关内容要重点展开，概念题简洁一点。
3. 如果 PPT 里出现公式、矩阵、概率、距离、相似度、语言模型、HMM、CRF、词向量、注意力机制、Transformer、评估指标等内容，请优先判断是否可能考计算题。
4. 如果某个知识点很可能考计算题，请用明显标签标出：
   【计算题高频】
   【公式必会】
   【算法流程】
   【易错点】
5. 输出要清晰、有层级，方便我复制到复习文档里。

在整理完知识点后，请额外帮我生成一组练习题：
1. 基础概念题 5 道
2. 计算题 5 道
3. 算法流程题 3 道
4. 综合应用题 2 道
```

```
最后考试：
填空10空，每空2分，共20分
简答4个，每个5分，共20分
分析计算题3个，共45分
综合分析题1个，共15分

```

第1章 绪论

1.1 基本概念

语言的主要载体

自然语言处理

语言理解

语言生成

1.2 自然语言处理研究的内容

按照应用场景划分

机器翻译（平行语料库、RNN/LSTM、Seq2Seq Encoder-Decoder、Teacher Forcing、Attention、Transformer Encoder-Decoder）

原语言的分析和理解（中文分词、词性标注、句法分析、命名实体识别、词义消歧、Word2Vec、BERT、Transformer Encoder）

目标语言的生成（n-gram语言模型、前馈神经网络语言模型、RNN/LSTM语言模型、Seq2Seq Decoder、Attention、Transformer Decoder）

机器翻译的难点（词义消歧、动态上下文表示、平行语料库、Attention、Transformer）

信息检索（词袋模型、向量空间模型、TF-IDF、BM25、Word2Vec/BERT向量表示、余弦相似度、向量检索、RAG）

跨语言检索（机器翻译、平行语料库、多语言概念词典、向量检索）

多模态检索（CNN、Transformer、向量表示、余弦相似度）

信息检索技术发展（词袋模型、TF-IDF、BM25、Word2Vec、BERT、向量检索、RAG）

字面匹配（词袋模型、TF-IDF、BM25）

从词形到浅层语义（Word2Vec、GloVe、ELMo、BERT、余弦相似度、向量检索）

从浅层语义到检索增强生成（BERT嵌入模型、向量数据库、ANN/HNSW、Top-K检索、BM25混合检索、Cross-Encoder重排序、RAG、GraphRAG）

自动文摘（RNN/LSTM、Seq2Seq Encoder-Decoder、Attention、Transformer Encoder-Decoder、Decoder-only大语言模型）

文档分类（词袋模型、TF-IDF、Word2Vec、TextCNN、RNN、LSTM、BiLSTM、BERT序列分类）

智能交互系统（语音识别、意图分类、命名实体识别、RNN/LSTM Encoder-Decoder、BERT、Transformer、知识图谱、RAG、Decoder-only大语言模型）

辅助教学（口语语料库、语音识别、BERT序列分类、BERT Token Classification、Transformer、RAG、Decoder-only大语言模型）

口语发音与流利度评估（口语语料库、语音识别、HMM、RNN/LSTM、Transformer）

作文自动评分与润色（BERT序列分类、Seq2Seq Encoder-Decoder、Attention、Transformer、Decoder-only大语言模型、任务微调、LoRA）

文字识别（CNN、n-gram语言模型、RNN语言模型）

文字编辑和自动校对（n-gram语言模型、RNN语言模型、BERT掩码语言模型、Transformer、Decoder-only大语言模型）

语音识别（口语语料库、HMM、RNN/LSTM、n-gram语言模型、Transformer）

文语转换（口语语料库、RNN/LSTM、Encoder-Decoder、Attention、Transformer）

情感及观点分析（词袋模型、TF-IDF、Word2Vec、TextCNN、LSTM、BiLSTM、BERT序列分类、命名实体识别）

知识图谱（命名实体识别、实体消歧、WordNet、HowNet、三元组表示、图遍历、RAG、GraphRAG）

文本自动生成（n-gram语言模型、前馈神经网络语言模型、RNN/LSTM语言模型、Seq2Seq、Attention、Transformer Decoder-only、Top-k采样、Top-p采样、任务微调、LoRA）

1.3 自然语言处理的基本问题

语音学问题

形态学问题

词素

词根、前缀、后缀和词尾

句法学问题

语义学问题

语用学问题

语用学和语义学的对比

1.4 自然语言处理面临的困难

自然语言中的歧义现象

结构歧义

语义歧义

自然语言中的未知语言现象

新的词汇

新的含义

新的用法和句型

1.5 NLP基本方法与技术

理性主义方法

基于规则的方法

规则库开发

词典标注

推导算法设计

Chomsky文法理论

经验主义方法

统计机器学习方法

大规模真实数据的收集与标注

统计模型构建

连结主义方法

神经网络方法

连续实数空间表示

神经网络模型构建

1.6 自然语言处理市场前景广阔

NLP算法工程师

语音识别方向

知识图谱方向

对话机器人方向

北京天润融通招聘需求

第2章 数学基础

2.1 概率论基础

基本概念

概率

最大似然估计

条件概率

全概率公式

贝叶斯决策理论

贝叶斯法则

二项式分布

期望

方差

概率链式法则

2.2 信息论基础

熵

英文信息源的熵

联合熵

条件熵

联合熵与条件熵的关系

相对熵

Kullback-Leibler散度

交叉熵

交叉熵损失函数

困惑度

语言模型与困惑度

互信息

互信息、条件熵与联合熵

利用互信息解决汉语分词问题

双字耦合度

互信息与双字耦合度的比较

噪声信道模型

噪声信道模型与**拼写纠错**

噪声信道模型的**应用**

2.3 应用举例

词汇歧义消解

一词多义现象

词义消歧

基于上下文的词义消歧

上下文中的词、词性与位置

基于贝叶斯分类器的词义消歧

最大似然估计求概率

“打”字词义消歧示例

2.4 附录

第4章 语料库与语言知识库

4.1 基本概念

语言数据库与知识库

大规模语言数据库

模型参数训练

评测标准

知识库

词汇语义库

词法、句法规则库

常识库

语料库

医学领域命名实体识别样本

QA问答语料

4.2 语料库的类型

按语言种类划分

单语语料库

双语或多语语料库

按是否标注划分

词性标注语料库

句法结构标注语料库

语义标注语料库

生语料

熟语料

平行语料库

篇章对齐

句子对齐

结构对齐

4.3 典型语料库介绍

宾夕法尼亚大学树库

Chinese Tree Bank

词性标注

句法结构标注

北京大学CLKB

现代汉语语法信息词典

汉语短语结构规则库

现代汉语多级加工语料库

多语言概念词典

平行语料库

多领域术语库

多级加工语料样例

口语语料库

The LJ Speech Dataset

CASIA-CASSIL语音对话语料库

CoNLL系列语料库

WikiCorpus

CAIL法律语料库

4.4 词汇知识库

知识库

结构化知识

人工提炼和组织的知识

WordNet

同义词集合

按词义组织词汇信息

语义词典

语义关系

同义关系

反义关系

上下位关系

部分关系

WordNet的基本功能

WordNet的应用

词汇消歧

语义推理

语义相似度计算

使用NLTK的WordNet模块

知网

上下位关系

同义关系

反义关系

对义关系

部件—整体关系

属性—宿主关系

材料—成品关系

施事、经验者、关系主体—事件关系

受事、内容、领属物—事件关系

工具—事件关系

场所—事件关系

时间—事件关系

值—属性关系

实体—值关系

事件—角色关系

相关关系

知网词语描述样例

知识图谱

实体关系与概念属性

知识图谱三元组

缓解大语言模型幻觉

多跳逻辑推理

小样本学习

DBpedia

CN-DBpedia

Freebase

中文医学知识图谱

第5章 语言模型（1/3）

语言模型

语言模型的应用

5.1 传统语言模型

5.1.1 n元文法

语句概率的计算

概率链式法则

统计基元

历史序列

参数数量指数级增长问题

有限历史假设

N-gram模型

一元文法（Unigram）

二元文法（Bigram）

三元文法（Trigram）

马尔可夫链

句首标志BOS

句尾标志EOS

音字转换问题

基于N-gram的候选汉字串选择

5.1.2 参数估计

训练数据与语料

最大似然估计

N-gram参数估计

二元文法参数估计

句子概率计算

数据匮乏与数据稀疏

零概率问题

5.1.3 数据平滑方法

加1法

拉普拉斯平滑

加k平滑

对数概率

减值法

Good-Turing估计法

折扣计数

未见事件概率

后退法

Katz后退

高阶N-gram与低阶N-gram

绝对减值法

插值法

线性插值

插值法的递归定义

均匀分布模型

第5章 语言模型（2/3）

5.2 神经语言模型

5.2.1 问题的提出

N-gram模型回顾

数据稀疏问题

忽略语义相似性问题

离散词的向量表示

One-hot向量

One-hot向量的维度与相似度问题

连续空间的分布式表示

低维稠密词向量

5.2.2 前馈神经网络语言模型

人工神经网络

神经元数学描述

输入向量

权值向量

偏置

激活函数

神经元输出

Sigmoid型函数

Logistic函数

Tanh函数

ReLU函数

前馈神经网络

多层感知器

输入层、隐藏层与输出层

网络权值矩阵与偏置向量

前馈信息传播

参数学习

交叉熵损失函数

正则化项

Frobenius范数

随机梯度下降法

学习率

反向传播算法

基于反向传播的随机梯度下降训练过程

基于FNN的语言模型

根据历史词预测下一个词

词向量表示

查找表

词表大小

词向量维数

历史词向量拼接

隐藏层非线性变换

Softmax回归预测

多分类概率分布

前馈神经网络语言模型计算示例

小窗口历史信息建模

5.2.3 循环神经网络语言模型

循环神经网络

当前输入

历史隐状态

隐状态更新

下一个词的概率

循环神经网络计算示例

误差反向传播

交叉熵损失

梯度消失

梯度爆炸

梯度消失与梯度爆炸的原因

使用ReLU激活函数

批量归一化

残差结构

梯度裁剪

权重初始化

长短时记忆网络

LSTM单元

细胞状态

门结构

遗忘门

输入门

细胞状态更新

输出门

LSTM缓解梯度消失与梯度爆炸

前馈神经网络语言模型工具

循环神经网络语言模型工具

LSTM语言模型工具

GPT

BERT

Word2Vec

离散符号表示与分布式连续向量表示

自注意力语言模型

第5章 语言模型（3/3）

5.3 文本表示

动因

向量空间模型

表示学习模型

基于深度学习的表示模型

词的表示学习

短语表示学习

句子表示学习

基于语言模型的词向量学习

Word Embedding

Word2Vec

CBOW与Skip-gram模型

BERT

ELMo

自监督学习

静态词向量

Word2Vec

GloVe

动态上下文表示

ELMo

BERT

RoBERTa

CBOW

上下文词预测中心词

CBOW基本原理

输入层

隐藏层

输出层

One-hot向量

查找表

上下文词向量求和或平均

Softmax归一化

反向传播更新参数

Skip-gram模型

中心词预测上下文词

预训练词向量在神经网络模型中的使用

预训练词向量的加载

Embedding层

冻结预训练词向量

微调预训练词向量

随机初始化词向量

短语表示学习

简单平均或求和

特定短语嵌入

Phrase2Vec

N-Gram特征

句子表示学习

平均池化方法

BERT的CLS Token

第6章 马尔可夫模型

马尔可夫模型与条件随机场

序列数据

序列标注任务

6.1 马尔可夫模型

马尔可夫过程

一阶马尔可夫链

马尔可夫假设

不动性假设

状态转移概率

状态转移概率矩阵

马尔可夫状态图

初始状态概率

状态序列概率

6.2 隐马尔可夫模型

隐马尔可夫模型

隐藏状态序列

观测输出序列

状态转移独立假设

观测独立假设

袋子取球示例

HMM的组成

状态数

观测符号数

状态转移概率矩阵

发射概率矩阵

初始状态概率分布

HMM参数表示

给定HMM生成观察序列

序列生成问题

HMM的三个基本问题

观察序列概率计算

最优状态序列选择

模型参数学习

6.3 前向算法

观察序列概率问题

所有状态路径的概率累加

动态规划

前向变量

前向概率

前向变量递推

观察序列总概率

前向算法描述

初始化

循环计算

结束与输出

6.4 前向算法—实例分析

观察集合与状态集合

初始状态分布

观测序列概率计算

初始前向变量

前向概率递推

6.4 后向算法

后向算法

后向变量

后向概率

后向变量递推

后向算法描述

6.5 Viterbi搜索算法

最优状态序列

最大概率路径

动态规划搜索

局部最优路径

Viterbi变量

最优前驱状态

递归计算

终止状态选择

最优路径回溯

Viterbi搜索过程

6.6 参数学习

模型参数学习问题

有状态标注的参数学习

最大似然估计

初始状态概率估计

状态转移概率估计

发射概率估计

无状态标注的参数学习

期望最大算法

模型参数初始化

隐变量期望值

模型参数更新

参数收敛

状态转移概率估计

状态占用概率估计

模型参数重估计

HMM应用—中文分词

BEMS状态集

中文句子观测序列

基于Viterbi算法的中文分词

HMM训练样本集

语音识别

生物信息学

模式识别

金融分析

医疗健康

HMM用于唇语解读

第7章 分词与命名实体识别

7.1 汉语自动分词概要

分词的意义

汉语自动分词

歧义切分问题

未登录词识别问题

交集型切分歧义

交集串

链长

组合型歧义

未登录词

人名、地名和组织机构名

新词、术语和缩略语

未登录词对模型的影响

汉语自动分词的基本原则

7.2 分词性能评价

封闭测试与开放测试

专项测试与总体测试

歧义字段切分能力

集外词处理能力

命名实体识别能力

精准率

召回率

集外词召回率

集内词召回率

F-测度值

F1值

分词性能评价示例

7.3 自动分词基本算法

有词典切分与无词典切分

基于规则的方法与基于统计的方法

最大匹配法

正向最大匹配算法

逆向最大匹配算法

双向最大匹配算法

FMM算法描述

双向最大匹配消歧规则

最少分词法

最短路径法

切分词图

基于语言模型的分词方法

基于二元文法的切分词图

由字构词的分词方法

基于字序列标注的分词方法

B、M、E、S词位标记

基于HMM的分词

基于BiLSTM的分词

BiLSTM与Softmax

BiLSTM与CRF

7.4 未登录词识别

命名实体识别

其他新词识别

命名实体

实体边界

实体类型

命名实体识别应用场景

信息抽取

对话机器人

知识图谱构建

中文姓名识别

中文姓名识别的难点

基于规则和词典的中文姓名识别

姓名概率估计

姓名评价函数

修饰规则

左界规则

右界规则

中文地名识别

中文地名识别的困难

地名资源知识库

识别规则库

命名实体识别的统一方法

序列标注

BIO标注体系

医疗领域命名实体识别

7.5 Token生成

Token

Tokenizer

Token ID

文本拆分

数字编码

向量嵌入

基于词的Token生成

基于子词的Token生成

BPE

WordPiece

Unigram

BPE初始化词表

BPE字符对统计与迭代合并

BPE分词应用

WordPiece初始化词表

WordPiece拆分与迭代合并

WordPiece分词阶段

7.6 词性标注

词性标注概述

词性兼类歧义

UPenn Treebank词性标注训练集

北大计算语言学研究所词性标注训练集

基于字符串匹配的字典查找算法

基于统计的词性标注方法

基于HMM的词性标注

基于深度学习的词性标注方法

7.7 分词与词性标注系统

中科院计算所ICTCLAS

复旦大学FNLP

Stanford词性标注系统

HanLP词性标注系统

第8章 深度神经网络

深度学习与传统机器学习

人工选择特征与模型分类

端到端学习

自动提取特征

特征工程

深度神经网络

多隐藏层前馈神经网络

输入层、隐藏层与输出层

层次化特征提取

简单特征与抽象特征

卷积神经网络结构

卷积神经网络

数据输入层

卷积层

激活层

池化层

全连接层

输入层

灰度图像输入

RGB图像输入

输入通道数

卷积层

局部特征提取

特征图

卷积核

浅层特征与深层特征

局部感知

感受野

卷积运算

卷积核大小

步幅

填充

SAME填充

VALID填充

卷积后特征图尺寸

卷积核数量与输出通道数

多卷积核特征提取

2D卷积

3D卷积

参数共享

局部连接

Sigmoid函数

Tanh函数

ReLU

Leaky ReLU

Softmax

GELU

最大池化

平均池化

池化效果

全连接层

输出层

全连接与Softmax分类

Dropout方法

模型过拟合

NLP中的卷积神经网络

文本分类

词向量矩阵

一维卷积

N-gram语义特征

多尺寸卷积核

卷积神经网络的优点

卷积神经网络的缺点

卷积神经网络实现

TextCNN模型

Embedding层

Conv1d卷积层

全局平均池化

交叉熵损失函数

Adam优化器

模型训练

预训练词向量

冻结Embedding参数

Hugging Face Trainer

循环神经网络RNN

时序预测问题

层间连接

同层循环连接

历史状态

RNN模型应用：文本分类

末态池化方式

LSTM文本分类

末态池化分类实现

平均池化方式

BiLSTM文本分类

平均池化分类实现

RNN模型应用：序列预测

N-N结构

序列标注任务

输入输出等长序列

词性标注

命名实体识别

分词

语音帧分类

Encoder-Decoder结构

Seq2Seq

编码器

解码器

语义向量

末时刻隐状态编码

隐状态平均编码

Teacher Forcing

机器翻译

Encoder-Decoder结构的缺点

注意力机制

Query、Key与Value

注意力权重

注意力计算过程

Query与Key相关性计算

缩放因子

Softmax归一化

Value加权求和

注意力输出

注意力在序列预测中的应用

动态语义向量Context

机器翻译中的注意力

注意力机制实现

基于注意力的Seq2Seq翻译模型

主流深度学习框架

PyTorch

MindSpore

MindSpore AI框架

基于MindSpore实现LeNet-5网络

卷积层参数设置

模型训练

模型保存

Checkpoint

模型推理

基于MindSpore实现RNN模型

第9章 大语言模型1：Transformer

1 Transformer概述

Transformer

Attention Is All You Need

LSTM序列模型

并行计算

长序列依赖

2 Transformer整体架构

Encoder

Decoder

编码器堆叠

解码器堆叠

编码器Block

解码器Block

自注意力层

带掩码的多头注意力层

编码器—解码器注意力层

前馈神经网络

Add & Norm

3 编码器

编码器的作用

上下文向量表示

输入层

单词Embedding

位置Embedding

绝对位置编码

相对位置编码

正弦位置编码

正弦位置编码计算

正弦位置编码的优势

单词Embedding与位置Embedding相加

ALiBi

旋转位置编码

动态位置编码

自注意力机制

Query

Key

Value

自注意力机制实现

注意力分数计算

缩放点积注意力

多头注意力机制

多特征子空间

多头注意力拼接

线性映射

Add & LayerNorm

残差连接

梯度消失

Layer Normalization

Batch Normalization

层归一化计算

单个词的完整编码过程

前馈神经网络

线性变换

ReLU激活函数

编码器输出

4 解码器

解码器的作用

解码器结构

带掩码的多头注意力机制

因果掩码

Mask矩阵

防止未来信息泄露

交叉注意力层

编码器输出作为Key和Value

解码器输出作为Query

输出层

Linear层

Softmax层

Logits

贪心搜索

束搜索

集束宽度

随机采样

温度参数

Top-k采样

Top-p采样

5 训练与预测

编码器训练

解码器训练

串行预测

并行训练

Teacher Forcing

Masked Self-Attention

6 Transformer关键变体

Decoder-only结构

单向自注意力

因果注意力

自回归生成

下一个词预测

GPT

LLaMA

Qwen

Encoder-only结构

双向注意力

文本理解

文本分类

实体抽取

问答

BERT

RoBERTa

Vision Transformer

图像分块

Patch嵌入

图像位置编码

7 BERT模型

BERT编码器结构

双向上下文表示

BERT输入

Tokenization

CLS标记

SEP标记

Token ID

Position ID

Token Type ID

Token Embedding

Position Embedding

Segment Embedding

句子对输入

Encoder Layer

自注意力层

前馈神经网络

残差连接

LayerNorm

BERT BASE

BERT LARGE

模型层数

隐藏层维度

注意力头数量

BERT输出

CLS向量

Pooler Output

Sequence Output

句子级分类

Token级分类

单句分类

句子对分类

文本相似度

问答

序列标注

BertForSequenceClassification

文本分类

意图识别

文本质量评分

BertForTokenClassification

命名实体识别

词性标注

关键词抽取

BertForQuestionAnswering

抽取式问答

答案起止位置预测

BERT预训练

掩码语言模型

下一句预测

随机遮盖Token

MASK标记

80%-10%-10%遮盖策略

句子关系判断

BertForMaskedLM

BertForNextSentencePrediction

Decoder-only、Encoder-only与Encoder-Decoder结构对比

8 Transformer应用实现

使用BERT进行文本分类

冻结BERT参数

训练分类头

基于预训练GPT模型生成文本

AutoModelForCausalLM

生成长度

随机采样

温度参数

本地模型加载

9 初识大语言模型

大语言模型

Transformer架构

海量文本训练

参数规模

训练数据规模

涌现能力

通用性与多任务处理

上下文学习

少样本学习

知识的广度与深度

复杂推理

代码生成与理解

长文本理解与创作

跨语言与跨领域迁移

思维链

大语言模型的构建

预训练

自监督学习

掩码语言模型

自回归生成

监督微调

领域与任务适应

人类反馈强化学习

奖励模型

RLHF

第9章 大语言模型2：模型微调

1 模型微调技术概述

1.1 模型微调

有监督微调

Supervised Fine-Tuning

预训练模型

特定任务数据

领域数据

通用能力

领域专属能力

1.2 为什么需要微调

通用模型的局限性

特定任务表现

输出风格

专业术语

定制化

性能提升

成本效益

1.3 SFT微调技术分类

按训练目标分类

指令微调

传统任务微调

领域适应微调

思维链微调

思维链训练样本

按参数更新策略分类

全参数微调

参数高效微调

冻结基座模型

2 参数高效微调

PEFT核心思想

LoRA

Adapter Tuning

Prefix Tuning

Prompt Tuning

Soft Prompt

显存占用

训练速度

多任务适配器

灾难性遗忘

2.1 LoRA原理

低秩假设

模型权重更新量

矩阵的秩

独立信息

低秩矩阵分解

降维矩阵

升维矩阵

LoRA参数量

冻结原始权重

低秩矩阵注入

注意力层Q投影矩阵

注意力层V投影矩阵

前向传播

Rank

缩放因子Alpha

反向传播

交叉熵损失

低秩矩阵梯度更新

权重合并

2.2 LoRA微调实现

LoRA参数配置

LoraConfig

目标模块

LoRA Dropout

任务类型

加载基座模型

加载分词器

应用LoRA配置

可训练参数比例

加载与预处理数据集

训练参数配置

批次大小

梯度累积

学习率

训练轮数

混合精度训练

梯度检查点

LoRA适配器保存

合并模型保存

基座模型与LoRA适配器加载

合并模型加载

模型推理

2.3 QLoRA

量化低秩适应

4-bit量化

NF4

双重量化

分页优化器

QLoRA工作流程

量化加载基座模型

插入LoRA适配器

混合精度训练

动态反量化

保存LoRA适配器

BitsAndBytesConfig

prepare_model_for_kbit_training

LayerNorm精度

FP4

Load in 4-bit

2.4 显存优化策略

梯度检查点

以时间换空间

激活值重计算

混合精度训练

FP16

BF16

FP32参数与梯度

梯度累积

等效批次大小

Adafactor

8-bit优化器

缩短序列长度

3 指令微调

Instruction Tuning

指令—响应数据

指令理解

内容生成

格式遵循

传统微调任务样本

3.1 指令微调的关键：高质量数据集

数据质量

数据多样性

人工标注

模型生成

教师模型

公开数据集

Alpaca

ShareGPT

Self-Instruct

准确性

多样性

语言质量

逻辑清晰度

3.2 常见数据集格式

Alpaca格式

Instruction字段

Input字段

Output字段

单轮问答

文本生成

翻译任务

ShareGPT格式

Conversations字段

From字段

Value字段

多轮对话

聊天机器人

3.3 指令微调训练流程

准备指令微调数据

转换Prompt格式

配置LoRA参数

配置SFT训练参数

初始化SFTTrainer

训练并保存模型

3.4 指令微调训练实现

数据准备

Qwen Prompt格式

聊天模板

用户角色

助手角色

加载基座模型

配置LoRA

配置SFTTrainer参数

训练数据字段

序列最大长度

SFTTrainer训练

Loss Mask

仅对回答部分计算损失

微调模型保存

微调模型推理

PeftConfig

PeftModel

Apply Chat Template

生成提示

跳过输入Prompt

截取回答内容

3.5 LlamaFactory

一站式大模型微调与部署框架

Web UI

命令行工具

YAML配置文件

开源大模型支持

多模态模型支持

QLoRA量化训练

FlashAttention-2

Unsloth

DeepSpeed

预训练

监督微调

奖励模型训练

RLHF

DPO

模型导出

模型量化

第9章 大语言模型3：RAG检索增强生成

1 RAG

为什么大模型会“一本正经地胡说”

知识截止日期

幻觉（Hallucination）

1.1 RAG概述

RAG的核心思想

RAG的实现

相比于通过微调LLM注入新知识，RAG的优势

1.2 RAG的工作流程

离线索引（Indexing）

在线查询（Query）

离线阶段：构建索引

文档加载

文本分块（Chunking）

文档向量化

在线阶段：检索并生成

问题向量化

相似度检索

上下文注入生成

1.3 向量数据库：RAG的记忆系统

什么是向量数据库

向量数据库的工作原理

HNSW索引

ANN搜索

HNSW分层结构

层内贪心搜索

向量数据库的内存代价

Chroma

FAISS

Qdrant

Weaviate

1.4 分块策略

分块大小对RAG效果的影响

固定大小分块

语义分块

递归字符分块

RecursiveCharacterTextSplitter

小块与大块的对比

1.5 检索质量优化

基础RAG的局限

语义与关键词的错配

混合检索

向量检索（稠密检索）

关键词检索（BM25）

倒数排名融合

BM25算法

IDF（逆文档频率）

重排序（Reranking）

交叉编码器（Cross-Encoder）

重排序核心逻辑

HyDE（虚拟文档）

多查询扩展

2 从零构建一个RAG问答系统

RAG问答系统结构

LangChain

安装LangChain核心库

加载文档

文本分块

向量化并存入向量数据库

构建RAG查询函数

使用混合检索策略并重排序

RetrievalQA

3 GraphRAG概述

普通RAG的盲区

多跳推理

GraphRAG的思路

3.1 GraphRAG工作流程

查询处理

图遍历

子图检索

信息整合

响应生成

自然语言查询转换为Cypher图查询语言

基于LangChain的GraphRAG分步实现

3.2 GraphRAG实现

GraphCypherQAChain

翻译用户问题

查询知识图谱

总结答案

第9章 大语言模型4：模型量化

1 量化技术概述

什么是量化

1.1 为什么需要量化

降低显存占用

提升推理速度

降低端侧部署成本

1.2 量化带来的问题

性能与精度下降

误差累积

量化误差产生的原因

截断

舍入

范围估计不准

2 模型精度类型

浮点数精度

FP32单精度浮点数

符号位、指数位和尾数位

FP16半精度浮点数

BF16脑浮点16

FP16与BF16对比

整数（量化）精度

INT8

INT4

NF4

3 量化对象

权重（Weights）

Transformer中的Linear层权重

Q/K/V投影矩阵

FFN层权重

静态量化

激活值（Activation）

异常值（Outliers）

动态量化

仅量化权重

4 量化方法分类

4.1 对称与非对称量化

对称量化

非对称量化

对称量化原理

对称量化计算公式

缩放因子Scale

浮点转定点

定点转浮点

非对称量化原理

非对称量化计算公式

零点值Z

校准

为什么需要校准

校准的难点

校准数据

Min/Max校准

Moving Average Min/Max校准

Histogram直方图校准

Percentile分位数校准

熵校准

KL散度校准

4.2 训练后量化和量化感知训练

训练后量化（PTQ）

量化感知训练（QAT）

仅权重量化

全整数量化

GPTQ

GPTQ量化生成工具

GPTQ模型推理引擎

预量化GPTQ模型

AWQ

AWQ量化生成工具

AWQ模型推理引擎

预量化AWQ模型

伪量化算子

反向传播补偿量化误差

5 模型量化实践

基于AutoAWQ和Transformers实现

利用AutoAWQ生成量化模型

加载量化模型并推理

基于AutoGPTQ和Transformers实现

GPTQ校准数据格式

GPTQ量化配置

执行量化并保存模型

基于bitsandbytes实现

运行时动态量化

NF4量化格式

QLoRA训练与推理

基于PyTorch Quantization实现

动态量化Linear层

PyTorch PTQ/QAT框架

第1章 绪论

1.1 基本概念

- **【概念/定义】**：自然语言处理（NLP）是人工智能核心方向，将语言作为计算对象，研究算法和模型，使计算机能理解、生成、转换自然语言。
- **【特征/特点】**：包含语言理解与语言生成，处理文本、语音等多模态。
- **【应用场景】**：文本分类、情感分析、语义解析、机器翻译、文本摘要、对话系统。

1.2 自然语言处理研究的内容

- 机器翻译：分析原语言 + 生成目标语言，应用于文献翻译、网页翻译。
- 信息检索：字面匹配 → 词形/浅层语义 → 检索增强生成，应用于搜索引擎。
- 自动文摘：提取文档主要内容，生成摘要。
- 文档分类：按主题或内容自动归类，应用于垃圾邮件识别、新闻分类。
- 智能交互系统：理解用户意图并与用户自然交互，应用于聊天机器人、智能客服。
- 辅助教学：口语评测、作文自动评分与润色。
- OCR文字识别：将印刷/手写文字转化为计算机文本。
- 文本编辑与自动校对：拼写、语法检查，文档排版。
- 语音识别（ASR）：将语音信号转为书面文本，难点在同音词和口音。
- 文语转换（TTS）：将文本转为语音信号。
- 情感与观点分析：识别观点及情感色彩，应用于商品评论、舆情监控。

- 知识图谱：构建实体关系语义网络，辅助检索。
- 文本自动生成：算法生成语法正确、语义连贯的文本，如自动作诗、新闻稿、代码生成。

1.3 自然语言处理的基本问题

- 语音学：研究词与语音（音素、音节、音调）关联，应用于 ASR、TTS、口语评测。
- 形态学：词由词素构成的内部结构和规则，应用于词形还原、词干提取、拼写纠错。
- 句法学：句子成分关系及规则，用于句法分析、构建句法树。
- 语义学：研究词在句子中的意义及整体语义。
- 语用学：研究说话者意图，通过语境分析理解隐含意义。

1.4 NLP面临的困难

- 歧义现象：
  - 结构歧义：句子结构多义，如“咬死了猎人的狗”。
  - 语义歧义：词语多重含义，如“喜欢乡下的孩子”。
- 未知语言现象：新词汇、新含义、新用法与句型，如网络语言、外来语、专业术语。

1.5 NLP基本方法与技术

- 理性主义：基于规则分析，利用规则库、词典标注和推导算法。
- 经验主义：统计机器学习方法，通过大规模标注语料建立统计模型。
- 连结主义：神经网络方法，向量表示语言，学习数据模式自动调整网络权重。

1.6 自然语言处理市场前景

- 就业方向：语音识别、知识图谱、对话机器人
- 技能要求：HMM、EM、LDA、CNN、RNN、LSTM，熟悉 Java/C++/Python，掌握数据结构与算法。

第2章 数学基础

2.1 概率论基础

> **【概念/定义】** 本节涉及 NLP 中常用概率基础：概率、最大似然估计、条件概率、全概率公式、贝叶斯决策理论、贝叶斯法则、二项式分布、期望、方差。

【方法/技术】

- **最大似然估计 MLE**：用样本中事件出现的相对频率估计其概率。
- **条件概率**：在某事件已发生条件下，另一事件发生的概率。
- **全概率公式**：把复杂事件拆成若干互斥情况求和。
- **贝叶斯定理**：由结果反推原因的概率。

【作用/目的】

- 为语言模型、文本分类、词义消歧、HMM、朴素贝叶斯等模型提供数学基础。

【计算/公式】

- **相对频率 / 最大似然估计**：

  $$q_N(s_k) = \frac{n_N(s_k)}{N}$$

  其中 $n_N(s_k)$ 是样本 s_k 出现次数，N 是实验总次数。当样本数足够大时：

  $$\lim_{N\to\infty} q_N(s_k) = P(s_k)$$

- **链式法则**：

  $$P(a,b) = P(a|b)P(b)$$

  $$P(X_1, X_2, \cdots, X_n) = P(X_1|X_2, \cdots, X_n)P(X_2|X_3, \cdots, X_n)\cdots P(X_n)$$

- **全概率公式**：

  $$P(A) = \sum_{i=1}^{n} P(A|B_i)P(B_i)$$

  适用条件：$B_1, B_2, \cdots, B_n$ 两两互斥，且构成完备事件组。

- **贝叶斯定理**：

  $$P(B_i|A) = \frac{P(B_i)P(A|B_i)}{\sum_{j=1}^{n} P(B_j)P(A|B_j)}$$

  当只有一个假设 B 时：

  $$P(B|A) = \frac{P(A|B)P(B)}{P(A)}$$

【计算题重点】：

1. 给出现次数，求最大似然概率。
2. 给条件概率和先验概率，代入贝叶斯公式求后验概率。
3. 给多个互斥事件，使用全概率公式求总概率。

2.2 信息论基础

> 【概念/定义】 信息论用于度量随机变量的不确定性、分布差异、模型预测效果等，是语言模型评价、文本表示、分词、消歧等任务的重要基础。

【方法/技术】

- **熵**：度量随机变量平均不确定性。
- **联合熵**：度量两个随机变量整体的不确定性。
- **条件熵**：已知一个变量后，另一个变量剩余的不确定性。
- **相对熵 / KL散度**：衡量两个概率分布的差异。
- **交叉熵**：衡量预测分布与真实分布之间的差距。
- **困惑度**：语言模型常用评价指标，越小越好。
- **互信息**：衡量两个变量之间的相关程度。
- **噪声信道模型**：通过“观测结果”反推“原始输入”。

【作用/目的】

- 衡量语言模型好坏。
- 衡量词之间的结合强度。
- 支持拼写纠错、语音识别、机器翻译、中文分词、词性标注等任务。

【计算/公式】

- **熵**：

$$H(X) = -\sum_{x \in X} p(x) \log_2 p(x)$$

含义：平均信息量 / 不确定性。熵越大，不确定性越大。

等概率熵：$H(X) = \log_2 N$

例如：27 个字符等概率：$H(X) = \log_2 27 \approx 4.75$

- **联合熵**：

$$H(X,Y) = -\sum_{x\in X}\sum_{y\in Y} p(x,y)\log_2 p(x,y)$$

- **条件熵**：

$$H(Y|X) = -\sum_{x\in X}\sum_{y\in Y} p(x,y)\log_2 p(y|x)$$

- **联合熵与条件熵关系**：

$$H(X,Y) = H(X) + H(Y|X)$$

注意：$H(Y|X) \neq H(X|Y)$

- **相对熵 / KL散度**：

$$D_{KL}(P||Q) = \sum_x p(x)\log\frac{p(x)}{q(x)}$$

特点：分布越接近，KL 越小；完全相同则为 0。

- **交叉熵**：

$$H(P,Q) = -\sum_x p(x)\log q(x)$$

或：

$$H(P,Q) = H(P) + D_{KL}(P||Q)$$

机器学习中，常用交叉熵作为损失函数。

- **困惑度 (Perplexity, PPL)**：

$$PP(W) = 2^{H(W)}$$

也可写为：

$$PP(W) = P(w_1, w_2, \cdots, w_N)^{-\frac{1}{N}}$$

若按语言模型展开：

$$PP(W) = \left(\prod_{i=1}^{N} P(w_i | w_1, \cdots, w_{i-1}) \right)^{-\frac{1}{N}}$$

含义：模型预测下一个词时的平均不确定性；困惑度越小，语言模型越好。

- **互信息**：

  $$I(X;Y) = H(X) - H(X|Y)$$

  等价形式：

  $$I(X;Y) = \sum_{x} \sum_{y} p(x,y) \log_2 \frac{p(x,y)}{p(x)p(y)}$$

  含义：知道 Y 后，X 的不确定性减少了多少。

- **汉语分词中的互信息**：

  $$I(x;y) = \log_2 \frac{p(x,y)}{p(x)p(y)}$$

  互信息越大，两个字结合越紧密，越可能成词。

- **双字耦合度**：

  $$\text{Couple}(c_i, c_{i+1}) = \frac{N(c_i c_{i+1})}{N(c_i c_{i+1}) + N(\cdots c_i | c_{i+1} \cdots)}$$

  其中：

  - $N(c_i c_{i+1})$：两个字连续出现且构成词的次数。
  - $N(\cdots c_i | c_{i+1} \cdots)$：两个字连续出现但跨词的次数。

  耦合度越大，越倾向于不断开。

- **噪声信道模型中的拼写纠错**：

  $$\hat{w} = \arg\max_{w \in V} P(w|x)$$

  由贝叶斯公式，对固定输入 x，$P(x)$ 不变，因此：

  $$\hat{w} = \arg\max_{w \in V} P(x|w)P(w)$$

  其中：

  - $P(w)$：候选词 w 的先验概率。
  - $P(x|w)$：正确词 w 被误写为 x 的概率。

⚠️**【计算题重点】**：

1. 给概率分布，计算熵。
2. 给联合分布，计算联合熵、条件熵。
3. 给两个分布，计算 KL 散度或交叉熵。
4. 给语言模型概率，计算困惑度 PPL。
5. 给词频 / 共现频率，计算互信息或双字耦合度。
6. 给拼写错误词和候选词概率，使用噪声信道模型选最大概率候选。

2.3 应用举例

> **【概念/定义】** 词义消歧（Word Sense Disambiguation, WSD）是指区分同一词在不同上下文中的具体语义。

【特征/特点】

- 一词多义普遍存在。
- 同一个词在不同上下文中可能表达不同含义。
- 词义判断依赖上下文。

【方法/技术】

- **基本思路**：利用多义词周围上下文信息判断词义。
- **常用上下文信息**：词、词性、位置。
- **示例**：以目标词为中心，取前后窗口，如 $-2, -1, +1, +2$。
- 可用**贝叶斯分类器**实现词义消歧。

【作用/目的】

- 解决自然语言中的词汇歧义问题。
- 提高机器翻译、信息检索、问答系统、文本理解等任务准确率。

【计算/公式】

假设多义词 w 的上下文为 C，候选语义为 s_i，选择概率最大的语义：

$$\hat{s} = \arg\max_{s_i} P(s_i|C)$$

根据贝叶斯公式：

$$P(s_i|C) = \frac{P(s_i)P(C|s_i)}{P(C)}$$

因为 $P(C)$ 对所有候选义项相同，可忽略：

$$\hat{s} = \arg\max_{s_i} P(s_i)P(C|s_i)$$

在独立性假设下：

$$P(C|s_i) = \prod_{v_k \in C} P(v_k|s_i)$$

最终分类公式：

$$\hat{s} = \arg\max_{s_i} \left[P(s_i) \prod_{v_k \in C} P(v_k|s_i) \right]$$

最大似然估计：

$$P(v_k|s_i) = \frac{N(v_k, s_i)}{N(s_i)}$$

$$P(s_i) = \frac{N(s_i)}{N(w)}$$

其中：

- $N(s_i)$：训练数据中语义 s_i 出现次数。
- $N(v_k, s_i)$：上下文词 v_k 与语义 s_i 共现次数。
- $N(w)$：目标多义词 w 出现总次数。

⚠️**【计算题重点】**：

1. 给多义词、上下文窗口、各义项频数，计算每个义项概率。
2. 按公式 $P(s_i)\prod P(v_k|s_i)$ 比较大小，选择最大者作为消歧结果。

- **例**：
  - 若 $N(\text{鼓}, s_1) = 5$，$N(s_1) = 100$，则：

    $$P(\text{鼓}|s_1) = \frac{5}{100} = 0.05$$
  - 若 $N(s_1) = 100$，$N(\text{打}) = 800$，则：

    $$P(s_1) = \frac{100}{800} = 0.125$$

第4章 语料库与语言知识库

4.1 基本概念

- **【概念/定义】**：语料库和语言知识库是 NLP 系统的重要资源，通常为处理模块提供模型训练数据、评测标准和结构化语言知识。
- **【特征/特点】**：
  - 大规模语言数据库：用于模型参数训练、系统评测。
  - 知识库：包括词汇语义库、词法规则库、句法规则库、常识库。
- **【概念/定义】**：语料库 corpus 是经过科学取样和加工的大规模电子文本库，存放真实使用中出现过的语言材料。
- **【应用场景】**：
  - 命名实体识别 NER。
  - 问答 QA。
  - 文本分类。
  - 机器翻译。
  - 语言模型训练。
- **【方法/技术】**：
  - 语料可带标注，例如医学命名实体识别中的 BIO 标注：
    - `B-OPERATION`：手术实体开始。
    - `I-OPERATION`：手术实体内部。
    - `B-DISEASE`：疾病实体开始。
    - `I-DISEASE`：疾病实体内部。
    - `O`：非实体。
- **【计算/公式】**：本节无重点公式，主要考概念题、分类题和 BIO 标注理解。

4.2 语料库的类型

- **【特征/特点】**：
  - 按语言种类划分：
    - 单语语料库。
    - 双语或多语语料库。
  - 按是否标注划分：
    - 生语料：直接收集、未经加工的语言资源集。
    - 熟语料：已完成分词、词性标注、句法标注、语义标注等处理。
  - 按标注类型划分：
    - 词性标注语料库。
    - 句法结构标注语料库，也称树库。
    - 语义标注语料库。
- **【概念/定义】**：平行语料库是指两种或多种语言之间进行平行采样和加工的语料库，常见于机器翻译中的双语对齐语料库。
- **【方法/技术】**：
  - 平行语料常见对齐方式：
    - 篇章对齐。
    - 句子对齐。
    - 结构对齐。
- **【应用场景】**：
  - 机器翻译。
  - 跨语言信息检索。
  - 双语词典构建。
- **【计算/公式】**：本节无重点公式，可能考“生语料/熟语料区别”“平行语料库定义”“对齐类型”。

4.3 典型语料库介绍

- **【概念/定义】**：宾夕法尼亚大学树库 UPenn Tree Bank 是典型句法标注语料库，用于词性标注和句法结构分析模型训练。
- **【特征/特点】**：
  - 由美国宾夕法尼亚大学 M. Marcus 教授主持。
  - 1993 年完成约 300 万词次英语句子的语法结构标注。

    - 2000 年完成第一版汉语树库，约 10 万词次、4185 个句子。
    - Chinese Tree Bank 中汉语词性被划分为 33 类，句法标记 23 类。
- **【方法/技术】**：
    - 词性标注：如 `他/PN`、`提出/VV`、`措施/NN`。
    - 句法结构标注：标识短语结构以及短语之间的层次句法关系。
- **【概念/定义】**：北京大学 CLKB 是中文语言知识资源库，包括语法信息词典、规则库、加工语料库、概念词典、平行语料库和术语库。
- **【特征/特点】**：
    - 现代汉语语法信息词典：8 万词，360 万语法属性描述。
    - 汉语短语结构规则库：600 多条语法规则。
    - 现代汉语多级加工语料库：1.5 亿字。
    - 多语言概念词典：10 万个同义词集概念。
    - 平行语料库：100 万英汉句对。
    - 多领域术语库：35 万汉英对照术语。
- **【概念/定义】**：口语语料库主要服务语音识别、语音翻译、语音合成等研究。
- **【应用场景】**：
    - 语音识别 ASR。
    - 语音翻译。
    - 语音合成 TTS。
    - 对话系统。
- **【特征/特点】**：
    - CASIA-CASSIL 语音对话语料库：选自 15000 余段汉语电话对话录音。
    - 每段平均不少于 90 秒、10 个回合。
    - 场景包括旅馆、餐馆、机场。
- **【概念/定义】**：
    - CoNLL 系列语料库：包含 NER、句法分析、语义角色标注等任务的标准评测语料。
    - WikiCorpus：基于维基百科构建的大规模多语言通用语料库，是 BERT、GPT 等预训练模型的重要训练来源。
    - CAIL 法律语料库：中文法律领域语料库，包含裁判文书、法律问答等。
- **【计算/公式】**：本节无重点公式，主要考典型语料库名称、用途、特点。

4.4 词汇知识库

- **【概念/定义】**：知识库是存储结构化知识的系统，内容是经过人工提炼和组织的知识。
- **【特征/特点】**：
  - 以符号、图等形式组织知识。
  - 用于推理、问答和决策支持。
- **【概念/定义】**：WordNet 是普林斯顿大学开发的词汇语义知识库，主要用于解决词典中同义信息的组织问题。
- **【特征/特点】**：
  - 根据词义而不是词形组织词汇信息。
  - 本质上是一部语义词典。
  - 按同义词集合 synset 组织词义。
  - 语义关系可看作同义词集合之间的双向指针。
  - 五大类词汇：名词、动词、形容词、副词、虚词。
- **【方法/技术】**：
  - WordNet 中的 4 种主要语义关系：
    - 同义关系 synonymy。
    - 反义关系 antonymy。
    - 上下位关系 hypernym / hyponym。
    - 部分整体关系 meronymy。
- **【应用场景】**：
  - 词义消歧。
  - 语义推理。
  - 语义相似度计算。
  - 机器翻译。
- **【概念/定义】**：知网 HowNet 是中文词汇语义知识库，认为知识是由概念、概念关系、属性、属性关系构成的系统。
- **【特征/特点】**：
  - 描述概念之间、属性之间、事件角色之间的复杂关系。
  - 可区分同一个词在不同语境下的不同义项。
  - 例如“打”可对应 buy、knit 等不同语义。
- **【方法/技术】**：

    - 知网描述的主要关系：
        - 上下位关系。
        - 同义关系。
        - 反义关系。
        - 对义关系。
        - 部件-整体关系。
        - 属性-宿主关系。
        - 材料-成品关系。
        - 施事/经验者/主体-事件关系。
        - 受事/内容/领属物-事件关系。
        - 工具-事件关系。
        - 场所-事件关系。
        - 时间-事件关系。
        - 值-属性关系。
        - 实体-值关系。
        - 事件-角色关系。
        - 相关关系。
- **【概念/定义】**：知识图谱 KG 用图结构描述实体之间的关系，以及实体或概念的属性。
- **【方法/技术】**：
    - 知识图谱常用三元组表示：

    三元组主要分为以下两种基本表达形式：

    1. **关系三元组（SPO）**：用于描述实体与实体之间的某种关系。

    $$(\text{实体}_1, \text{关系}, \text{实体}_2)$$

    2. **属性三元组**：用于描述实体自身的某种属性及对应的具体取值。

    $$(\text{实体}, \text{属性}, \text{属性值})$$

    --- **【应用示例】**

    关系三元组示例：　$(\text{姚明}, \text{出生地}, \text{上海})$

    (解释：表达“姚明”与“上海”这两个实体之间存在“出生地”的关联。)

    属性三元组示例：　$(\text{姚明}, \text{身高}, 226\text{cm})$

    (解释：表达实体“姚明”拥有“身高”属性，其属性值为 “226cm”。)
- **【作用/目的】**：

    - 缓解大语言模型幻觉问题。
    - 支持多跳逻辑推理。
    - 支持小样本学习。
    - 为 NLP 系统提供结构化先验知识。
- **【应用场景】**：
    - 问答系统。
    - 语义搜索。
    - 推荐系统。
    - 医学知识问答。
    - 法律知识检索。
    - 大模型 RAG。
- **【方法/技术】**：
    - 知识图谱辅助大模型的一般流程：
        - 用户提问。
        - 从知识库中检索准确事实。
        - 将事实输入大模型。
        - 大模型基于事实生成回答。
    - 多跳推理：通过图遍历寻找实体关系路径。
- **【典型知识图谱】**：
    - DBpedia：从维基百科抽取事实信息。
    - CN-DBpedia：复旦大学开发的中文知识图谱。
    - Freebase：早期大规模知识图谱，后被 Google 收购。
    - 中文医学知识图谱：面向医学领域的中文知识图谱。
- **【计算/公式】**：本节无复杂公式，重点是三元组表示、知识图谱作用、WordNet/HowNet 语义关系。

5语言模型

第5章 语言模型（1/3）

> 【概念/定义】 语言模型是一种用于预测和生成自然语言文本的统计模型，本质上判断一个语句是否合理。

【特征/特点】

- 合理语句 $\rightarrow$ 给出较高概率。
- 不合理语句 $\rightarrow$ 给出较低概率。

【应用场景】

- 语音识别
- 手写体文字识别
- 机器翻译
- 文本生成

【计算/公式】 语言模型核心是计算句子概率：

$$P(s) = P(w_1, w_2, \ldots, w_m)$$

5.1 传统语言模型

> 【概念/定义】 传统语言模型主要基于统计方法，通过语料中词序列的出现频率估计句子概率。

【特征/特点】

- 基于概率统计。
- 依赖大规模语料。
- 常用 n-gram 建模。

【核心内容】

- n 元文法。

- 参数估计。
- 数据平滑方法。

5.1.1 n元文法

> **【概念/定义】** n 元文法（N-gram）是一种只考虑有限长度历史的语言模型，假设当前词只依赖前 $n-1$ 个词。

【作用/目的】

- 降低完整句子概率计算复杂度。
- 缓解历史序列过长导致的参数爆炸问题。

【方法/技术】

- 完整链式法则中，当前词依赖前面所有词。
- n-gram 中，当前词只依赖前 $n-1$ 个词。
  - 当 $n=1$：uni-gram，一元文法，当前词独立于历史。
  - 当 $n=2$：bi-gram，二元文法，也称 1 阶马尔可夫链。
  - 当 $n=3$：tri-gram，三元文法，也称 2 阶马尔可夫链。

【局限/不足】

- 完整历史建模参数量巨大。
- 随着句子长度增加，历史组合路径指数级增长。
- 容易出现数据稀疏问题。

【应用场景】

- 拼音输入法。
- 音字转换。
- 语音识别候选句选择。

- 机器翻译候选句排序。

【计算/公式】 句子 $s = w_1 w_2 \dots w_m$ 的完整概率：

$$P(s) = P(w_1, w_2, \dots, w_m)$$

$$P(s) = P(w_1)P(w_2|w_1)P(w_3|w_1 w_2) \cdots P(w_m|w_1 w_2 \dots w_{m-1})$$

简写为：

$$P(s) = \prod_{i=1}^{m} P(w_i|w_1, w_2, \dots, w_{i-1})$$

当 $i = 1$ 时：

$$P(w_1|w_0) = P(w_1)$$

n-gram 近似：

$$P(w_i|h_i) \approx P(w_i|w_{i-n+1}, \dots, w_{i-1})$$

即：

$$P(w_i|w_1, \dots, w_{i-1}) \approx P(w_i|w_{i-n+1}, \dots, w_{i-1})$$

使用起止标记：

$$\langle BOS \rangle \; w_1 w_2 \dots w_m \; \langle EOS \rangle$$

对于 n-gram，句子概率可写为：

$$P(s) = \prod_{i=1}^{m+1} P(w_i|w_{i-n+1}^{i-1})$$

其中：$w_0 = \langle BOS \rangle, \quad w_{m+1} = \langle EOS \rangle$

- **例：** 句子“我是一个学生”

$$P(\text{我是一个学生}) = P(\text{我})P(\text{是}|\text{我})P(\text{一}|\text{我, 是})P(\text{个}|\text{我, 是, 一})P(\text{学生}|\text{我, 是, 一, 个})$$

- **音字转换：**

$$\hat{CStr} = \arg\max_{CStr_i} P(CStr_i|Py)$$

根据贝叶斯公式：

$$\hat{CStr} = \arg\max_{CStr_i} \frac{P(Py|CStr_i)P(CStr_i)}{P(Py)}$$

因为 $P(Py)$ 对所有候选相同，可忽略：

$$\hat{CStr} \approx \arg\max_{CStr_i} P(CStr_i)$$

使用 2-gram 计算候选句概率：

$$P(CStr) = P(w_1|\langle BOS\rangle)P(w_2|w_1)\cdots P(\langle EOS\rangle|w_m)$$

⚠️**【计算题重点】**：

1. 给一个句子，按链式法则展开概率。
2. 判断 uni-gram、bi-gram、tri-gram 分别依赖多少历史词。
3. 给拼音候选句，用 n-gram 计算候选句概率并选择最大者。
4. **注意必须加入**：$\langle BOS\rangle$，$\langle EOS\rangle$

5.1.2 参数估计

> **【概念/定义】** 参数估计是指利用训练语料中的计数信息，估计 n-gram 条件概率。

【方法/技术】

- 收集、标注大规模训练语料。
- 使用最大似然估计 MLE 计算条件概率。
- n-gram 概率 = 对应 n-gram 出现次数 / 历史串出现次数。

【局限/不足】

- 如果某个 n-gram 在训练语料中未出现，MLE 会得到 0 概率。
- 一个条件概率为 0，会导致整个句子概率为 0。
- 这就是**数据稀疏（sparse data）**问题。

【计算/公式】 n-gram 最大似然估计：

$$P(w_i|w_{i-n+1}^{i-1}) = \frac{c(w_{i-n+1}^{i})}{c(w_{i-n+1}^{i-1})}$$

其中：

- $c(w_{i-n+1}^{i})$：完整 n-gram 出现次数。
- $c(w_{i-n+1}^{i-1})$：历史串出现次数。

对于 2-gram：

$$P(w_i|w_{i-1}) = \frac{c(w_{i-1}w_i)}{c(w_{i-1})}$$

- **训练语料：**
  a. $\langle BOS\rangle\ John\ read\ Moby\ Dick\ \langle EOS\rangle$
  b. $\langle BOS\rangle\ Mary\ read\ a\ different\ book\ \langle EOS\rangle$
  c. $\langle BOS\rangle\ She\ read\ a\ book\ by\ Cher\ \langle EOS\rangle$

- **用 2-gram 求：** $P(John\ read\ a\ book)$
  - 展开为：$P(John|\langle BOS\rangle)P(read|John)P(a|read)P(book|a)P(\langle EOS\rangle|book)$
  - 由语料统计：
    - $P(John|\langle BOS\rangle) = \frac{1}{3}$
    - $P(read|John) = \frac{1}{1}$
    - $P(a|read) = \frac{2}{3}$
    - $P(book|a) = \frac{1}{2}$
    - $P(\langle EOS\rangle|book) = \frac{1}{2}$
  - 所以：

$$P(John\ read\ a\ book) = \frac{1}{3} \times 1 \times \frac{2}{3} \times \frac{1}{2} \times \frac{1}{2} \approx 0.06$$

- **新句子：** $P(Cher\ read\ a\ book)$
  - 展开：$P(Cher|\langle BOS\rangle)P(read|Cher)P(a|read)P(book|a)P(\langle EOS\rangle|book)$
  - 由于：
    - $P(Cher|\langle BOS\rangle) = \frac{0}{3} = 0$
    - $P(read|Cher) = \frac{0}{1} = 0$
  - 所以：

    $$P(Cher\ read\ a\ book) = 0$$

⚠️**【计算题重点】**：

1. 先给句子加：$\langle BOS\rangle$，$\langle EOS\rangle$
2. 再统计 bigram 计数。
3. 用公式：$P(w_i|w_{i-1}) = \frac{c(w_{i-1}w_i)}{c(w_{i-1})}$
4. 最后把每一项条件概率相乘。
5. 若某一项为 0，则整句概率为 0，需要数据平滑。

5.1.3 数据平滑方法

> **【概念/定义】** 数据平滑是为了解决 n-gram 中未见事件导致零概率的问题，通过调整计数或混合低阶模型，为未出现事件分配一定概率。

【作用/目的】

- 避免零概率。
- 缓解数据稀疏。
- 提高语言模型对新句子的泛化能力。

【方法/技术】

- 加 1 法。
- 减值法。
- 插值法。

1. 加 1 法 (Additive smoothing / Laplace smoothing)

【计算/公式】

$$P(w_i|w_{i-1}) = \frac{c(w_{i-1}w_i)+1}{c(w_{i-1})+|V|}$$

其中：

- $|V|$：训练语料词汇表大小。
- 分子加 1：避免未见 bigram 概率为 0。
- 分母加 $|V|$：保证概率归一化。

一般也可加 k （其中 $k<1$）：

$$P(w_i|w_{i-1}) = \frac{c(w_{i-1}w_i)+k}{c(w_{i-1})+k|V|}$$

- **接前面例子**，词汇量 $|V|=11$，平滑后：
  - $P(Cher|\langle BOS\rangle) = \frac{0+1}{3+11} = \frac{1}{14}$
  - $P(read|Cher) = \frac{0+1}{1+11} = \frac{1}{12}$
  - $P(a|read) = \frac{2+1}{3+11} = \frac{3}{14}$
  - $P(book|a) = \frac{1+1}{2+11} = \frac{2}{13}$
  - $P(\langle EOS\rangle|book) = \frac{1+1}{2+11} = \frac{2}{13}$
  - 因此：

    $$P(Cher\ read\ a\ book) = \frac{1}{14}\times\frac{1}{12}\times\frac{3}{14}\times\frac{2}{13}\times\frac{2}{13} \approx 0.00003$$

  - 平滑后的：

    $$P(John\ read\ a\ book) = \frac{2}{14}\times\frac{2}{12}\times\frac{3}{14}\times\frac{2}{13}\times\frac{2}{13} \approx 0.0001$$

💡 为避免概率连乘过小，可改求**对数概率**：

$$\log P(s) = \sum_i \log P(w_i | w_{i-n+1}^{i-1})$$

【局限/不足】

- 加 1 法会过度抬高未见事件概率。
- 会降低已见事件概率。
- 在大词表场景下效果通常较粗糙。

---

2. 减值法 (Discounting)

> **【基本思想】** 降低已出现 n-gram 的计数，把减下来的概率量分配给未见事件。

【主流技术】

- Good-Turing 估计。
- 后退法 Back-off。
- 绝对减值法 Absolute discounting。

📌 Good-Turing 估计

【计算/公式】

$$r^* = (r+1)\frac{n_{r+1}}{n_r}$$

其中：

- r ：某 n-gram 原始出现次数。
- n_r ：出现 r 次的 n-gram 个数。
- r^* ：折扣后的计数。

平滑后概率：

$$P_r = \frac{r^*}{N}$$

其中 N 是样本集中 n-gram 总出现次数。

未出现过的 n-gram 的概率总和：

$$P_0 = \frac{n_1}{N}$$

- **含义**：未见事件的总概率等于只出现过一次的事件所占比例。

【计算题重点】：给 r, n_r, n_{r+1}，计算 $r^* = (r+1)\frac{n_{r+1}}{n_r}$，再计算 $P_r = \frac{r^*}{N}$。

- *例如*：出现 1 次的 bigram 有 $n_1 = 2053$，出现 2 次的 bigram 有 $n_2 = 458$，则：

  $$r^* = 2 \times \frac{458}{2053} \approx 0.446$$

📌 后退法 (Back-off)

> **【基本思想】** 如果高阶 n-gram 计数大于 0，则使用高阶概率；如果高阶计数为 0，则退回低阶 n-gram，并乘归一化系数。

【计算/公式】 Katz Back-off：

$$P_{katz}(w_i|w_{i-1}) = \begin{cases} d_r \dfrac{c(w_{i-1}w_i)}{c(w_{i-1})}, & c(w_{i-1}w_i) > 0 \\ \alpha(w_{i-1}) P_{ML}(w_i), & c(w_{i-1}w_i) = 0 \end{cases}$$

其中：

- 折扣率：$d_r = \dfrac{r^*}{r}$
- $P_{ML}(w_i)$：一元文法的最大似然估计概率。

⚠️**【计算题重点】**：先判断高阶 n-gram 是否出现。出现则用折扣后的高阶概率；未出现则后退到低阶概率。注意乘以归一化系数 α。

📌 绝对减值法 (Absolute discounting)

> **【基本思想】** 从每个非零计数 r 中减去固定值 b，将减下来的概率量分配给未见事件。

【计算/公式】

$$P_r = \begin{cases} \dfrac{r-b}{N}, & r > 0 \\ \dfrac{b(R-n_0)}{Nn_0}, & r = 0 \end{cases}$$

其中：

- b ：减去的常量，且 $b \leq 1$ 。
- n_0 ：样本中未出现事件的个数。
- N ：样本集中所有 n-gram 事件的总出现次数。
- R ：所有非零计数事件的总出现次数。计算公式为： $R = \sum_{r>0} r \cdot N_r$

⚠**【计算题重点】**：已出现事件用 $\frac{r-b}{N}$ ；未出现事件均分减出来的概率量。注意区分 r, N, R, n_0, b 。

---

3. 插值法 (Interpolation)

> **【基本思想】** 用低阶 n-gram 模型对高阶 n-gram 模型进行线性插值。

【作用/目的】

- 高阶模型信息更具体，但更稀疏。
- 低阶模型更稳定，但上下文信息较少。
- 插值法综合两者优势。

【计算/公式】 三元插值：

$$P'(w_3|w_1, w_2) = \lambda_1 P(w_3|w_1, w_2) + \lambda_2 P(w_3|w_2) + \lambda_3 P(w_3)$$

其中： $\lambda_1 + \lambda_2 + \lambda_3 = 1$

递归插值形式：

$$P_{interp}(w_i|w_{i-(n-1)}, \ldots, w_{i-1}) = \lambda P_{ML}(w_i|w_{i-(n-1)}, \ldots, w_{i-1}) + (1-\lambda) P_{interp}(w_i|w_{i-(n-2)}, \ldots, w_{i-1})$$

递归结束时，可令 unigram 为最大似然估计或均匀分布：

$$P_0(w_i) = \frac{1}{|V|}$$

⚠**【计算题重点】**：

1. 给 $\lambda_1, \lambda_2, \lambda_3$ 和各阶概率，直接代入加权求和。
2. 必须满足：$\lambda_1 + \lambda_2 + \lambda_3 = 1$
3. **核心考点**：插值法不是只在高阶为 0 时才使用低阶，而是**始终混合**各阶概率。

5.2 神经语言模型

5.2.1 问题的提出

【概念/定义】：

离散符号表示的 n-gram 模型存在问题：

- **数据稀疏**：一些 n-gram 在训练语料中可能未出现。
- **忽略语义相似性**：语义相似的词无法共享信息（如“枯燥”与“乏味”）。

【方法/技术】：

离散词 one-hot 向量表示：

$$\text{枯燥} = [0, 1, 0, \ldots, 0]^T, \quad \text{乏味} = [0, 0, 0, 1, 0, \ldots, 0]^T$$

- **问题**：高维稀疏，词间相似度 0。

探索低维稠密连续向量表示（词向量 embedding）：

- 可通过训练学习词向量，使语义相似的词向量接近。

【应用场景】：

- 解决 n-gram 数据稀疏。
- 提升语义建模能力。

不同神经网络区别：激活函数；网络结构

常见激活函数：logistic，Tanh，ReLU

5.2.2 前馈神经网络语言模型 (FNN-LM)

【概念/定义】：

使用人工神经网络对语言序列进行建模，预测下一个词概率。

【神经元数学描述】：

$$y = f\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

- x_i：输入向量分量。
- w_i：权值。
- b：偏置。
- f：非线性激活函数（如 Sigmoid, Tanh, ReLU）。

【网络结构】：

- 多层感知器 MLP / FNN。
- 信息沿一个方向传播，无环。

每层输出：

$$\alpha^{(l)} = f(W^{(l)} \cdot \alpha^{(l-1)} + b^{(l)})$$

可看作复合函数：

$$\phi(x; W, b) : x = \alpha^{(0)} \to \alpha^{(L)}$$

【训练目标】：

结构化损失函数（MSE 或交叉熵）：

$$\mathcal{R}(W, b) = \frac{1}{N} \sum_{n=1}^{N} \|y^{(n)} - \hat{y}^{(n)}\|_2^2 + \frac{\lambda}{2} \|W\|_F^2$$

- λ：正则化系数。

随机梯度下降更新：

$$W^{(l)} \leftarrow W^{(l)} - \eta \frac{\partial \mathcal{R}}{\partial W^{(l)}}, \quad b^{(l)} \leftarrow b^{(l)} - \eta \frac{\partial \mathcal{R}}{\partial b^{(l)}}$$

- 使用反向传播 BP 计算梯度。

【词向量表示】：

查找表 Look-up Table：

$$x = \mathrm{LT}(w_{t-n+1}, \ldots, w_{t-1})$$

将前 $n-1$ 个词向量拼接，送入 FNN 输出每个词的概率：

$$w_t = p(w_t | \mathrm{context})$$

【输出层】：

Softmax 多分类：

$$p(y = c|x) = \frac{\exp(z_c)}{\sum_{k=1}^{C} \exp(z_k)}$$

5.2.3 循环神经网络语言模型 (RNN-LM)

【概念/定义】：

- RNN 可保留历史信息，通过循环结构捕捉序列上下文。

【数学描述】：

隐状态更新：

$$h_t = \tanh(Uw_t + Wh_{t-1})$$

输出概率：

$p(w_t|h_{t-1}, w_t)$ 或经过 $\mathrm{Softmax}$ 得到词概率分布

【问题分析】：

- **梯度消失 / 梯度爆炸**：
  - 链式乘法导致梯度衰减或膨胀。
  - 激活函数导数 < 1，易梯度消失。

【解决方法】：

- 使用 ReLU 激活函数。
- 批量归一化。
- 残差结构。
- 梯度裁剪。
- 合理权重初始化。

【长短时记忆 LSTM】：

- 解决 RNN 梯度消失问题。
- 通过**细胞状态** C_t 控制记忆流：
  - **遗忘门** f_t：控制保留上一步信息。
  - **输入门** i_t：控制新信息写入。
  - **输出门** o_t：控制输出。

公式：

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \quad h_t = o_t \odot \tanh(C_t)$$

【应用场景】：

- 语言模型。
- 文本生成。
- 语音识别。

- 机器翻译。
- 处理长序列依赖。

【开源工具】：

- FNN-LM：http://nlg.isi.edu/software/nplm/
- RNN-LM：http://rnnlm.org/
- LSTM-LM：https://www-i6.informatik.rwth-aachen.de/web/Software/rwthlm.php
- GPT：https://github.com/openai/gpt-2
- BERT：预训练双向 Transformer
- Word2Vec：http://code.google.com/p/word2vec

【总结】：

- 离散符号表示 → 分布式连续向量表示。
- 前馈神经网络 FNN。
- 循环神经网络 RNN。
  - 基本 RNN。
  - LSTM。
- 自注意力语言模型（后续章节）。

5.3表示学习模型

5.3.3 表示学习模型

表示学习模型把高维、离散的词转换为低维、稠密、能反映语义关系的向量。

主要包括：

- 词表示
- 短语表示
- 句子表示

基于语言模型的词向量学习

核心思想：**词的含义由上下文决定**。

模型通过“根据上下文预测词”学习词向量，这属于自监督学习。

静态词向量

- 同一个词在不同句子中向量不变。

- 代表：Word2Vec、GloVe。
- 优点：查表即可，速度快。
- 缺点：无法处理多义词。

动态上下文表示

- 同一个词在不同句子中向量不同。
- 代表：ELMo、BERT、RoBERTa。
- 能根据具体上下文处理一词多义。

CBOW

- 用上下文词预测中心词。
- 输入多个上下文词，输出一个中心词。
- 主要用于学习词向量，忽略词序，不能生成文本。

基本流程：

1. one-hot 向量查词向量表 W；
2. 上下文词向量求和或平均；
3. 乘输出矩阵 W′；
4. Softmax 得到各词概率；
5. 反向传播更新 W和 W′。

【易错点】CBOW 是“上下文预测中心词”。

Skip-gram

- 用中心词预测周围上下文词。
- 输入一个中心词，输出多个上下文词。

【易错点】Skip-gram 与 CBOW 方向相反。

预训练词向量的使用

三种方式：

1. Embedding 随机初始化并训练；
2. 加载预训练词向量并冻结；
3. 加载预训练词向量后继续微调。

短语表示学习

- 简单求和或平均：计算简单，但忽略语序。

- Phrase2Vec：把高频短语当作一个整体 token。
- N-Gram：保留部分局部顺序信息。

句子表示学习

- 平均池化：对所有词向量取平均。
- BERT 的 `[CLS]`：用 `[CLS]` 位置输出表示整句语义。

第6章 马尔科夫模型

【概念/定义】：

马尔可夫模型和条件随机场都是处理序列数据的模型，可用于序列标注任务。

【应用场景】：

- **中文分词。**
- **词性标注。**
- **命名实体识别。**
- **语音识别。**
- **生物序列分析。**

6.1 马尔可夫模型

【概念/定义】：

马尔可夫过程是指系统在多个状态之间转移，每个时刻的状态转移概率只依赖之前有限个状态。

【特征/特点】：

系统有 N 个状态：

$S_1, S_2, \ldots, S_N$

- q_t：表示系统在 t 时刻的状态。

一般情况下，当前状态依赖之前多个状态：

$$P(q_t = S_j | q_{t-1} = S_i, q_{t-2} = S_k, \ldots)$$

【方法/技术】：

- **一阶马尔可夫假设**：当前状态只依赖前一时刻状态。
- $P(q_t = S_j | q_{t-1} = S_i, q_{t-2} = S_k, \ldots) = P(q_t = S_j | q_{t-1} = S_i)$
- **不动性假设**：状态转移概率与时间 t 无关。
- $P(q_t = S_j | q_{t-1} = S_i) = a_{ij}$

> *只要是从 S_i 转移到 S_j，概率都一样。*

【计算/公式】：

状态转移概率约束：

$a_{ij} \geq 0$

$$\sum_{j=1}^{N} a_{ij} = 1$$

状态序列 $S_1, S_2, \ldots, S_T$ 的概率：

$$P(S_1, S_2, \ldots, S_T) = P(S_1)P(S_2|S_1)P(S_3|S_2)\cdots P(S_T|S_{T-1})$$

写成乘积形式：

$$P(S_1, S_2, \ldots, S_T) = \pi_{S_1} \prod_{t=1}^{T-1} a_{S_t S_{t+1}}$$

其中：

$\pi_i = P(q_1 = S_i)$

【计算题重点】：

给状态转移图，计算某一状态序列概率。

- **步骤**：
  a. 找初始状态概率。
  b. 沿路径依次乘状态转移概率。
- **例**：
- $P(t, i, p) = P(S_1 = t)P(S_2 = i|S_1 = t)P(S_3 = p|S_2 = i) = 1.0 \times 0.3 \times 0.6 = 0.18$

6.2 隐马尔可夫模型

【概念/定义】：

隐马尔可夫模型 HMM 是一个双重随机过程：状态序列不可直接观察，但观测序列由隐藏状态以一定概率生成。

【特征/特点】：

- 隐状态不可见。
- 观测值可见。
- 状态之间满足马尔可夫假设。
- 观测值由当前隐状态生成。
- 马尔可夫模型和 HMM 都是有向图模型。

【例子】：

- **袋子摸球**：
  - 袋子编号：隐状态。
  - 球的颜色：观测值。
  - 从哪个袋子取球不可见，只能看到颜色序列。
- **天气问题**：
  - 天气：隐状态。
  - 土壤干湿：观测值。

【方法/技术】：

HMM 的五个组成部分：

- 状态数 N。
- 观测符号数 M。
- 状态转移概率矩阵 A。
- 发射概率矩阵 B。
- 初始状态概率分布 π。

HMM 通常记作：

$$\mu = (A, B, \pi)$$

或：

$$\mu = (S, O, A, B, \pi)$$

【计算/公式】：

- **状态转移概率**：
- $a_{ij} = P(q_{t+1} = S_j | q_t = S_i)$
- 约束：
- $a_{ij} \geq 0, \quad \sum_{j=1}^{N} a_{ij} = 1$
- **发射概率**：
- $b_j(k) = P(O_t = v_k | q_t = S_j)$
- 约束：
- $b_j(k) \geq 0, \quad \sum_{k=1}^{M} b_j(k) = 1$

- **初始状态概率：**
- $\pi_i = P(q_1 = S_i)$
- 约束：
- $\pi_i \geq 0, \quad \sum_{i=1}^{N} \pi_i = 1$

【作用/目的】：

- 根据观测序列推测隐藏状态序列。
- 解决序列标注问题。

【三个基本问题】：

- **问题 1：概率计算问题**
- 给定模型 $\mu = (A, B, \pi)$ 和观察序列 $O = O_1 O_2 \dots O_T$，计算：$P(O|\mu)$
- **问题 2：解码问题**
- 给定模型 μ 和观察序列 O，求最优状态序列：$Q = q_1 q_2 \dots q_T$
- **问题 3：参数学习问题**
- 给定观察序列 O，估计模型参数 A, B, π，使 $P(O|\mu)$ 最大。

【计算题重点】：

HMM 的三大问题要对应算法：

- 计算 $P(O|\mu)$：前向算法 / 后向算法。
- 求最优状态序列：Viterbi 算法。
- 求参数：最大似然估计 / EM 算法。

6.3 前向算法

【概念/定义】：

前向算法用于解决 HMM 的问题 1，即给定模型和观测序列，快速计算观测序列出现概率。

【作用/目的】：

- 避免枚举所有隐状态路径。
- 使用动态规划减少重复计算。

【方法/技术】：

直接枚举所有状态序列时，若隐状态数为 N，序列长度为 T，可能路径数为：N^T，计算量很大。

前向算法缓存局部概率，逐步递推。

【计算/公式】：

对给定状态序列 $Q = q_1 q_2 \cdots q_T$：

$$P(O, Q|\mu) = P(Q|\mu)P(O|Q, \mu)$$

状态序列概率：

$$P(Q|\mu) = \pi_{q_1} a_{q_1 q_2} a_{q_2 q_3} \cdots a_{q_{T-1} q_T}$$

观测序列在该状态序列下的概率：

$$P(O|Q, \mu) = b_{q_1}(O_1) b_{q_2}(O_2) \cdots b_{q_T}(O_T)$$

所以：

$$P(O|\mu) = \sum_{Q} P(O, Q|\mu)$$

【方法/技术】：

定义前向变量：

$$\alpha_t(i) = P(O_1, O_2, \ldots, O_t, q_t = S_i|\mu)$$

> **含义**：在 t 时刻到达状态 S_i，并已经观测到 $O_1, \ldots, O_t$ 的概率。

【计算/公式】：

- **初始化**：
- $\alpha_1(i) = \pi_i b_i(O_1), \quad 1 \le i \le N$
- **递推**：
- $\alpha_{t+1}(j) = \left[\sum_{i=1}^{N} \alpha_t(i) a_{ij} \right] b_j(O_{t+1})$
  - $\alpha_t(i)$：上一时刻到达状态 S_i 的概率。
  - a_{ij}：从状态 S_i 转移到 S_j 的概率。
  - $b_j(O_{t+1})$：状态 S_j 发射观测值 O_{t+1} 的概率。
- **终止**：
- $P(O|\mu) = \sum_{i=1}^{N} \alpha_T(i)$

【计算题步骤】：

1. 写出模型参数 A, B, π 和观测序列 O。
2. 初始化每个状态：
3. $\alpha_1(i) = \pi_i b_i(O_1)$
4. 从 $t = 1$ 开始逐层递推：

5. $\alpha_{t+1}(j) = \left[\sum_{i=1}^{N} \alpha_t(i) a_{ij}\right] b_j(O_{t+1})$
6. 最后求和：
7. $P(O|\mu) = \sum_{i=1}^{N} \alpha_T(i)$

【计算题重点】：

- 这是历年高频计算题。
- 注意“**先求和，再乘发射概率**”：
- $\left[\sum_{i} \alpha_t(i) a_{ij}\right] b_j(O_{t+1})$
- 不要漏乘发射概率。
- 不要把 a_{ij} 和 $b_j(O_t)$ 混淆。

【例子】：

若观测序列：

$O = \{红, 白, 红\}$

初始状态分布：

$\pi = (0.2, 0.4, 0.4)$

且：

$b_1(红) = 0.5, \quad b_2(红) = 0.4, \quad b_3(红) = 0.7$

则：

$\alpha_1(1) = 0.2 \times 0.5 = 0.1$

$\alpha_1(2) = 0.4 \times 0.4 = 0.16$

$\alpha_1(3) = 0.4 \times 0.7 = 0.28$

6.4 后向算法

【概念/定义】： 后向算法也是用于计算 $P(O|\mu)$ 的动态规划方法，从序列末尾向前递推。

【方法/技术】： 定义后向变量：

$\beta_t(i) = P(O_{t+1}, O_{t+2}, \ldots, O_T | q_t = S_i, \mu)$

> **含义**：从 t 时刻状态 S_i 出发，经过所有可能的后续状态路径，生成后续观测序列的概率。

【计算/公式】：

- **初始化**：

- $\beta_T(i) = 1, \quad 1 \leq i \leq N$
- **递推**：
- $\beta_t(i) = \sum_{j=1}^{N} a_{ij} b_j(O_{t+1}) \beta_{t+1}(j)$
- **最终观测序列概率**：
- $P(O|\mu) = \sum_{i=1}^{N} \pi_i b_i(O_1) \beta_1(i)$

【计算题步骤】：

1. **末尾初始化**：
2. $\beta_T(i) = 1$
3. **从 $T-1$ 倒推到 1**：
4. $\beta_t(i) = \sum_{j=1}^{N} a_{ij} b_j(O_{t+1}) \beta_{t+1}(j)$
5. **最后计算**：
6. $P(O|\mu) = \sum_{i} \pi_i b_i(O_1) \beta_1(i)$

【计算题重点】：

- 前向算法是“从前往后”。
- 后向算法是“从后往前”。
- 后向递推时要考虑：
  a. 从当前状态 i 转移到下一状态 j。
  b. 下一状态 j 发射 O_{t+1}。
  c. 从 j 继续生成后续观测序列。

6.5 Viterbi 搜索算法

【概念/定义】： Viterbi 算法用于解决 HMM 的问题 2，即在给定模型和观测序列时，寻找概率最大的隐藏状态序列。

【作用/目的】：

- 找到最能解释观测序列的状态路径。
- 避免暴力枚举所有状态序列。

【方法/技术】：

- 使用动态规划。

- 每个时刻记录到达每个状态的局部最优路径。
- 同时记录最优前驱状态，用于最后回溯路径。

【计算/公式】： 定义 Viterbi 变量：

$$\delta_t(i) = \max_{q_1,\ldots,q_{t-1}} P(q_1,\ldots,q_{t-1},q_t = S_i, O_1,\ldots,O_t|\mu)$$

> **含义**：到达 t 时刻状态 S_i，并观测到 $O_1,\ldots,O_t$ 的最大路径概率。

- **初始化：**
- $\delta_1(i) = \pi_i b_i(O_1)$
- **递推：**
- $\delta_t(j) = \max_{1\le i\le N} [\delta_{t-1}(i)a_{ij}]\, b_j(O_t)$
- **记录最优前驱：**
- $\psi_t(j) = \arg\max_{1\le i\le N} [\delta_{t-1}(i)a_{ij}]$
- **终止：**
- $P^* = \max_{1\le i\le N} \delta_T(i)$
- $q_T^* = \arg\max_{1\le i\le N} \delta_T(i)$
- **回溯：**
- $q_t^* = \psi_{t+1}(q_{t+1}^*), \quad t = T-1, T-2, \ldots, 1$

【计算题步骤】：

1. **初始化：**
2. $\delta_1(i) = \pi_i b_i(O_1)$
3. **逐时刻递推：**
4. $\delta_t(j) = \max_i [\delta_{t-1}(i)a_{ij}] b_j(O_t)$
5. **每个状态记录最大值来自哪个前驱：**
6. $\psi_t(j) = \arg\max_i [\delta_{t-1}(i)a_{ij}]$
7. **最后一列找最大** $\delta_T(i)$。
8. **根据** ψ **指针反向回溯最优状态序列。**

【计算题重点】：

- 前向算法用的是“**求和**”：$\sum_i$
- Viterbi 用的是“**取最大**”：$\max_i$

- 前向算法求观测序列总概率。
- Viterbi 求最优状态路径。
- Viterbi 必须记录路径来源，否则只能得到最大概率，得不到完整状态序列。

6.6 参数学习

【概念/定义】： 参数学习用于解决 HMM 的问题 3，即根据观察序列估计模型参数 $\pi_i, a_{ij}, b_j(k)$，使 $P(O|\mu)$ 最大。

【方法/技术】：

- 若训练数据带有状态标注：直接使用**最大似然估计**。
- 若训练数据没有状态标注：使用 **EM 算法**。

【计算/公式】： 状态已知时：

- **初始概率估计**：
- $\pi_i = \dfrac{\text{所有样本序列中第一个时刻状态为 } S_i \text{ 的次数}}{\text{样本序列总数}}$
- **状态转移概率估计**：
- $a_{ij} = \dfrac{\text{从状态 } S_i \text{ 转移到状态 } S_j \text{ 的次数}}{\text{从状态 } S_i \text{ 转移到任意状态的总次数}}$
- **发射概率估计**：
- $b_j(k) = \dfrac{\text{状态为 } S_j \text{ 且观测值为 } v_k \text{ 的次数}}{\text{状态为 } S_j \text{ 的总次数}}$

无状态标注时使用 EM：

1. 随机初始化模型参数，得到 μ_0。
2. 根据当前模型估计隐变量的期望值。
3. 用期望次数代替真实次数，重新估计参数，得到 μ_1。
4. 重复直到参数收敛。

定义从状态 S_i 转移到状态 S_j 的概率估计：

$$\xi_t(i,j) = P(q_t = S_i, q_{t+1} = S_j | O, \mu)$$

用前向、后向变量表示：

$$\xi_t(i,j) = \frac{\alpha_t(i) a_{ij} b_j(O_{t+1}) \beta_{t+1}(j)}{P(O|\mu)}$$

定义在时间 t 处于状态 S_i 的概率 $\gamma_t(i) = P(q_t = S_i | O, \mu)$，由 ξ 求 γ：

$$\gamma_t(i) = \sum_{j=1}^{N} \xi_t(i,j)$$

参数重估公式：

$$\pi_i = \gamma_1(i)$$

$$a_{ij} = \frac{\sum_{t=1}^{T-1} \xi_t(i,j)}{\sum_{t=1}^{T-1} \gamma_t(i)}$$

$$b_j(k) = \frac{\sum_{t=1,\, O_t=v_k}^{T} \gamma_t(j)}{\sum_{t=1}^{T} \gamma_t(j)}$$

【计算题重点】：

- 有标注：直接数次数，做最大似然估计。
- 无标注：用 EM，核心是“期望次数代替真实次数”。
- **重点区分**：
  - $\xi_t(i,j)$：从 i 到 j 的转移期望。
  - $\gamma_t(i)$：时刻 t 在状态 i 的概率。

HMM 应用 - 中文分词

【概念/定义】： HMM 可用于中文分词，将分词问题转化为序列标注问题。

【方法/技术】：

- **状态集合**：$Q = \{B, E, M, S\}$
  - B：词的开始 begin。
  - M：词的中间 middle。
  - E：词的结束 end。
  - S：单字成词 single。
- **观测序列**：中文句子中的字符序列。
- **任务**：求最优状态序列。

【计算/公式】： 对观测字符序列 $C = \{c_1, c_2, \ldots, c_n\}$，求：

$$\max P(q_1, q_2, \ldots, q_n | c_1, c_2, \ldots, c_n)$$

其中 $q_i \in \{B, E, M, S\}$，用 Viterbi 算法求最优状态序列。

【例子】： 句子：今天天气不错 若最优状态序列为：$B\ E\ B\ E\ B\ E$ 分词结果：今天/天气/不错

【应用场景】：

- 中文分词。
- 词性标注。
- 命名实体识别。

- 语音识别。
- 生物信息学。
- 模式识别。
- 金融趋势预测。
- 医疗健康序列分析。

7分词与命名实体识别

明白，已为你将“第7章 分词与命名实体识别”调整为一级标题（`#`），并保持整体排版与公式的飞书完美兼容性。

你可以直接复制下方内容粘贴到飞书文档中：

第7章 分词与命名实体识别

7.1 汉语自动分词概要

【概念/定义】：

汉语文本以单字为基础，词与词之间没有显性边界，因此分词就是给句子添加词语边界，使词串反映句子本意。自动分词是中文信息处理的基础。

【作用/目的】：

- 为后续中文文本处理提供基础。
- 支持信息检索、机器翻译、文本分类、词性标注、命名实体识别等任务。

【局限/不足】：

分词主要难点：

- 歧义切分问题。
- 未登录词识别问题。

【特征/特点】：

- **交集型切分歧义**：对于汉字串 AJB，若 AJ 和 JB 同时为词，则产生交集型歧义，J 称为交集串。
  - *例*:
    - 中国/人为/了/实现/自己的/梦想

        - 中国人/为了/实现/自己的/梦想
    - **链长**：交集型歧义中交集串集合的元素个数。
- **组合型歧义**：多个词既可单独成词，也可组合成更长词，导致分词结果不唯一。
    - *例*:
        - 门/把/手/弄/坏/了
        - 门/把手/弄/坏/了

【概念/定义】：

未登录词是指不在词典中的词。

【特征/特点】：

- 常见未登录词：
    - 人名、地名、组织机构名。
    - 新词、术语、缩略语。
- 传统模型依赖固定词表，对未登录词常替换为：$\langle \mathrm{UNK} \rangle$ 或直接丢弃。

【方法/技术】：

汉语自动分词**基本原则**：

- 二字、三字、四字词以及结合紧密、使用稳定的词应切分。
- 四字成语一般作为分词单位。
- 五字及以上谚语、格言等，若分开后不违背组合意义，应予切分。
- 结合紧密、使用稳定的词组一般不切分。
- 惯用语和有转义的词或词组应作为整体。
- 略语一般作为分词单位。
- 分词单位加后缀“儿”形成儿化音时，可作为整体。
- 阿拉伯数字等保留原有形式。
- 现代汉语中其他语言音译外来词一般不切分。

【计算/公式】：

本节主要考概念题，重点区分“交集型歧义”“组合型歧义”“未登录词”。

7.2 分词性能评价

【概念/定义】：

分词性能评价用于衡量分词系统输出结果与标准答案之间的一致程度。

【方法/技术】：

- **封闭测试**：只允许使用固定训练语料学习。
- **开放测试**：可以使用任意资源。
- **专项测试**：针对某一能力测试，如歧义切分、集外词处理、命名实体识别。
- **总体测试**：整体评价系统分词效果。

【计算/公式】：

- **精准率 Precision**：
- $P = \frac{n}{N} \times 100\%$
  - N：系统输出的分词结果个数。
  - n：系统输出中正确的个数。
- **召回率 Recall**：
- $R = \frac{n}{M} \times 100\%$
  - M：标准答案中的词语个数。
  - n：系统输出中正确的个数。
- **F-Measure**：
- $F = \frac{(\beta^2 + 1)PR}{\beta^2 P + R} \times 100\%$
- 当 $\beta = 1$ 时（即 F_1 值）：
- $F_1 = \frac{2PR}{P + R} \times 100\%$

【计算题重点】：

常考给 N, M, n，求 P, R, F_1。

- *例：系统输出 $N = 5260$，标准答案 $M = 4510$，正确 $n = 4120$：*
- $P = \frac{4120}{5260} \times 100\% = 78.33\%$
- $R = \frac{4120}{4510} \times 100\% = 91.35\%$
- $F_1 = \frac{2 \times 78.33 \times 91.35}{78.33 + 91.35} = 84.34\%$

【特征/特点】：

- P 高：系统切出来的词中正确比例高。
- R 高：标准答案中的词被系统找回得多。
- F_1：综合考虑精准率和召回率。
- R_{OOV}：集外词召回率。

- R_{IV}：集内词召回率。

在不修改任何实质内容的前提下，我已经为你剔除了原文本中重复渲染的冗余公式，并将排版规范化为飞书 Markdown 与 LaTeX 的最佳实践标准。

你可以直接复制下方内容粘贴到飞书文档中：

7.3 自动分词基本算法

【概念/定义】：

自动分词算法可分为有词典切分和无词典切分，也可分为基于规则的方法和基于统计的方法。

【方法/技术】：

- 最大匹配法。
- 最少分词法 / 最短路径法。
- 基于语言模型的分词方法。
- 由字构词 / 基于字序列标注的分词方法。

【概念/定义】：

最大匹配法 Maximum Matching 是一种有词典的机械切分方法。

【方法/技术】：

- **正向最大匹配 FMM**：从句首开始，每次取词典中能匹配的最长词。
- **逆向最大匹配 BMM**：从句尾开始，每次取词典中能匹配的最长词。
- **双向最大匹配 Bi-directional MM**：比较 FMM 和 BMM 的切分结果，选择更合理者。

【方法/技术】：

- **FMM 算法步骤**：
  a. 设词典最长词长度为 l。
  b. 从待切分字符串当前位置取长度为 l 的匹配字段。
  c. 若词典中存在该字段，则切分成功。
  d. 若不存在，则去掉字段最后一个汉字，继续匹配。
  e. 重复直到切分完成。
- **BMM** 与 FMM 类似，但从句尾开始，匹配失败时去掉前面的一个汉字。

【方法/技术】：

双向最大匹配消歧规则：

1. 若正向和逆向结果词数不同，取**词数较少者**。
2. 若词数相同且结果相同，任取一个。

3. 若词数相同但结果不同，取**单字较少者**。

【局限/不足】：

- 依赖词典。
- 难处理未登录词。
- 对复杂歧义切分能力有限。

【计算题重点】：

给词典、最大词长、待切分句子，要求写 FMM/BMM 分词结果。

- **做题时**：FMM 从左往右；BMM 从右往左。每次优先匹配最长词，匹配失败就逐字缩短。

【概念/定义】：

最少分词法，也称最短路径法，是通过构建切分图，寻找词数最少的切分路径。

【方法/技术】：

对待切分字符串 $S = c_1 c_2 \dots c_n$：

1. 建立有向无环图。
2. 每个字之间默认建边。
3. 如果 $w = c_i c_{i+1} \dots c_j$ 是词典中的词，则在节点 v_{i-1} 和 v_j 之间建边。
4. 从所有路径中选择词数最少的路径作为分词结果。

【局限/不足】：

- 对许多歧义词仍难区分。
- 字符串较长时，长度相同的最短路径数量可能增加。
- 通常适合粗分，用于生成候选切分结果。

【计算题重点】：

给词典和句子，画切分图或列候选路径。找词数最少路径。若最短路径不唯一，仍需后续消歧。

【概念/定义】：

基于语言模型的分词方法是从候选切分中选择概率最大的切分结果。

【计算/公式】：

对待切分句子 S，设一种候选切分为 $W = w_1 w_2 \dots w_k$，最优切分：

$$W^* = \arg\max_W P(W|S)$$

根据贝叶斯思想：

$$W^* = \arg\max_W P(W)P(S|W)$$

一般切分 W 确定后 S 也确定，因此重点比较：

$$W^* = \arg\max_W P(W)$$

若使用二元语言模型（Bi-gram）：

$$P(W) = \prod_{i=1}^{k} P(w_i|w_{i-1})$$

【优点】：

- 训练语料足够大、覆盖领域足够多时，可获得较高准确率。

【局限/不足】：

- 依赖训练语料规模和质量。
- 计算量较大。
- 处理未登录词需要额外识别模块。

【计算题重点】：

给若干候选切分和 bigram 概率，计算：

$$P(W) = P(w_1|\langle BOS\rangle)P(w_2|w_1)\dots P(\langle EOS\rangle|w_k)$$

选择概率最大的切分结果。

【概念/定义】：

由字构词方法将分词看作字的分类问题，也就是序列标注问题。

【方法/技术】：

每个字对应一个词位标签：$\{B, M, E, S\}$

- B：词首。
- M：词中。
- E：词尾。
- S：单字成词。

> 如果词位作为隐藏状态，可以用 HMM 分词。

【作用/目的】：

- 统一处理词表词和未登录词，不必单独设计未登录词识别模块。

【方法/技术】：

- **HMM 分词**：
  - 隐藏状态：B, M, E, S。
  - 观测值：句子中的汉字。
  - 用 Viterbi 求最优状态序列。

- **BiLSTM 分词**：
  - 每个位置用 Softmax 输出概率最大标签。
  - **问题**：忽略标签之间的关联。
- **BiLSTM + CRF**：
  - 在 BiLSTM 输出层加入 CRF。
  - 通过 CRF 转移概率考虑标签序列约束。
  - CRF 关注整个标签序列概率，而不是单个标签概率。

【计算/公式】：

对字序列 $C = c_1 c_2 \dots c_n$，求最优标签序列：

$$Q^* = \arg\max_Q P(Q|C)$$

HMM 中可用 Viterbi：

无效公式

【计算题重点】：

给一串字和 $B/M/E/S$ 标签，写分词结果。

- *例：$B\ E\ B\ E\ S$ 表示前两个字成一个词，中间两个字成一个词，最后一个字单独成词。*
- 可能考 HMM/Viterbi 分词路径计算。

在不修改任何实质内容的前提下，我已经为你剔除了原文本中重复渲染的冗余公式，并将排版规范化为飞书 Markdown 与 LaTeX 的最佳实践标准。

7.4 未登录词识别

【概念/定义】：

未登录词识别是自动发现并切分出不在预设词典中的词，并进一步判断其类别。

【特征/特点】：

- 包括：
  - 命名实体识别。
  - 其他新词识别。
- 命名实体包括：人名、地名、组织机构名、数字、日期、货币数量。
- 其他新词包括：专业术语、缩写词、新普通词汇。

【概念/定义】：

命名实体识别 NER 包括两个任务：

1. 识别实体边界。

2. 判断实体类型。

【应用场景】：

- **信息抽取**：抽取商品价格、产地等。
- **对话机器人**：识别日期、地点、人名等关键信息。
- **知识图谱构建**：抽取实体和关系。

【特征/特点】：

中文姓名识别难点：

- 姓氏和名字都可单独用于指人。
- 姓氏、名字、全名都可能是普通词。
- 缺乏明显启发标记。

【方法/技术】：

基于规则和词典的中文姓名识别：

1. 以姓氏作为触发信息。
2. 从姓氏库和名字库中查找潜在姓名。
3. 估计潜在姓名概率。
4. 通过评价函数和识别规则筛选。

【计算/公式】：

设姓名：

$C_{\text{name}} = X m_1 m_2$

(其中：X 为姓氏，m_1 为名字首字，m_2 为名字尾字)

- **姓氏使用频率**：
- $F(X) = \dfrac{X\text{用作姓氏的次数}}{X\text{出现的总次数}}$
- **名字首字使用频率**：
- $F(m_1) = \dfrac{m_1\text{作为名字首字出现的次数}}{m_1\text{出现的总次数}}$
- **名字尾字使用频率**：
- $F(m_2) = \dfrac{m_2\text{作为名字尾字出现的次数}}{m_2\text{出现的总次数}}$
- **姓名评价函数**：
- $f = -\ln P(C_{\text{name}})$
- 当 f 大于姓氏 X 的最小阈值 $T_{\min}(X)$ 时，该汉字串确定为中文姓名。

【方法/技术】：

- **修饰规则**：若姓名前是数字，或姓氏与后面句号距离小于 2 个字符，则否定。
- **左界规则**：姓名前是称谓、标点、句首，或姓氏使用频率为 100%，则左边界确定。
- **右界规则**：姓名后是称谓、指界动词、标点、句尾，或尾字使用频率为 100%，则右边界确定。

【特征/特点】：

中文地名识别难点：

- 地名数量大，缺乏明确规范定义。
- 地名简称复杂，且地名和普通词冲突。
- 地名可能是其他专名的一部分，且长度不一。

【方法/技术】：

地名识别资源：地名库、地名首字库、地名尾字库、地名中间字库、筛选规则、确认规则、否定规则。

【方法/技术】：

命名实体识别统一方法：**转化为序列标注问题**。

假定实体类型包括：$\mathrm{PER},\ \mathrm{LOC},\ \mathrm{ORG}$

- **标注规则（BIO）**：
  - 实体开始：`B-XXX`
  - 实体内部：`I-XXX`
  - 非实体：`O`
  - *例*：`B-PER`（人名开始）、`I-PER`（人名内部）、`B-LOC`（地名开始）、`I-LOC`（地名内部）、`B-ORG`（机构名开始）、`I-ORG`（机构名内部）。

【计算题重点】：

- 给句子，要求标 BIO 标签。
- 给标签，要求还原实体边界和实体类型。
- 医疗领域实体也可用相同方法，如疾病、症状、药物、检查、手术等。

7.5 Token 生成

【概念/定义】：

在大语言模型中，Token 是文本处理的最小基本单位，是原始文本经过 Tokenizer 处理后映射到词表中的整数 ID。

【特征/特点】：

- 英文中，Token 可能是单词、子词或标点。
- 中文中，Token 通常是字、词或子词。
- 传统 NLP 多依赖分词工具，以词为基本单位。
- 大语言模型通常以子词（Subword）为基本单位，可有效缓解未登录词（OOV）问题。

【方法/技术】：

- **Token 生成三步**：文本拆分 → 数字编码 → 向量嵌入。
  - *文本拆分例*：“人工智能”可拆为“人工” “智能”；“unbelievable”可拆为“un” “believe” “able”。
  - *数字编码*：每个 Token 映射到唯一整数 ID。
  - *向量嵌入*：用向量表示 Token。
- **分词粒度对比**：
  - **Word-based（基于词）**：整词语义清晰；但词表容易爆炸，容易出现 OOV。
  - **Subword-based（基于子词）**：高频词作为完整 Token，低频词拆成常见子词。LLM 常用。
  - *常见子词算法*：**BPE**（GPT 常用）、**WordPiece**（BERT 常用）、**Unigram**（LLaMA 常用）。
- **BPE（字节对编码）算法**：
  - *核心思想*：统计相邻字符对频率，逐步合并高频字符对，形成 subword。
  - *步骤*：
    i. 初始化词表：把文本拆成最小单位。
    ii. 统计字符对频率。
    iii. 合并出现频率最高的相邻字符对。
    iv. 重复直到达到预设词表大小。
  - *分词阶段*：按最长优先匹配词表中的 subword。

【例子】：

语料：我 爱 自 然 语 言 处 理 自 然 语 言 很 重 要

- 第一次合并：`自 然` → `自然`
- 第二次合并：`自然 语` → `自然语`
- 第三次合并：`自然语 言` → `自然语言`

新文本“自然语言处理”的分词结果：`自然语言/处/理`

【方法/技术】：

- **WordPiece 算法**：
  - *核心思想*：通过语言模型概率选择最优拆分，优先保留概率高、语义较完整的 subword。

- *步骤*：
  i. 初始化词表：中文按单字，英文按字符。
  ii. 计算相邻字符对合并后的概率。
  iii. 选择概率最大的字符对合并。
  iv. 重复直到达到词表大小或无法合并。
- *分词阶段*：从左到右，优先匹配最长子词。若无匹配子词则取单个字符，避免 OOV。

【计算/公式】：

本节一般不考复杂公式，可能考 BPE 合并过程或 Token 与传统分词的区别。

7.6 词性标注（前辈7.5

【概念/定义】：

词性标注是在已经切分好的文本中，给每个词标注所属词类，如名词、动词、代词、形容词等。

【作用/目的】：

- 支持句子理解。
- 支持情感分析、句法分析、信息抽取等后续任务。

【局限/不足】：

- **词性兼类歧义**：一个词可能有多种词性。
  - *例*：“好”（形容词 / 动词）；“会”（名词 / 动词）。

【特征/特点】：

- **UPenn Treebank**：33 类词性标注。
- **北大计算语言学研究所标注集**：26 个基本词类代码，74 个扩充代码，共 106 个代码。

【方法/技术】：

- **基于字符串匹配的字典查找算法**：先分词，再从字典中查找词性。*缺点：不能解决词性兼类问题。*
- **基于统计的词性标注方法**：可用 HMM。观测序列为分词后的词语序列，隐藏序列为词性标注序列。
- **基于深度学习的词性标注方法**：使用神经网络建模上下文。

【计算/公式】：

HMM 词性标注可写为：

$$T^* = \arg\max_T P(T|W)$$

(其中：$W = w_1, w_2, \ldots, w_n$ 为词序列；$T = t_1, t_2, \ldots, t_n$ 为词性序列)

用 HMM 解码时可用 Viterbi：

$$\delta_t(j) = \max_i [\delta_{t-1}(i) a_{ij}] b_j(w_t)$$

【计算题重点】：

- 给词序列，判断观测序列和隐藏序列分别是什么（**观测序列：词；隐藏序列：词性**）。
- 若给出 HMM 参数，可能要求用 Viterbi 算法求最优词性序列。

7.7 分词与词性标注系统

【应用场景】：

公开分词与词性标注系统可用于中文分词、词性标注、命名实体识别等实验。

【典型系统】：

- 中科院计算所 ICTCLAS / NLPIR
- 复旦大学 FudanNLP
- Stanford POS Tagger
- HanLP

【计算/公式】：

本节无计算公式，主要了解典型工具名称和用途。

第8章 深度神经网络

深度学习 vs 传统机器学习

【概念/定义】：

深度学习通过多层神经网络进行端到端学习，能够自动提取特征；传统机器学习通常需要人工选择特征，再进行模型分类。

【特征/特点】：

- **传统机器学习**：人工特征工程 + 分类模型。
- **深度学习**：自动特征提取 + 分类/预测。
- **核心差异**：深度学习减少对人工特征工程的依赖。

【作用/目的】：

- 让模型直接从原始数据中学习有效特征。
- 适合图像、语音、文本等复杂数据。

深度神经网络

【概念/定义】：

深度神经网络是包含多个隐藏层的前馈神经网络。

【结构】：

输入层 → 多个隐藏层 → 输出层。

【特征/特点】：

- 层次化特征提取：底层提取简单特征，高层提取抽象特征。

【应用场景】：

图像分类、文本分类、语音识别、序列预测、机器翻译。

模型	NLP 中的主要作用	常见应用
FNN	对固定长度的文本向量进行分类或预测	文本分类、情感分析、语言模型输出层、词性分类
CNN	提取相邻词语构成的局部特征	文本分类、情感分析、关键词检测、句子匹配
RNN	按顺序处理文本，利用上下文信息	语言模型、机器翻译、序列标注、文本生成

卷积神经网络结构CNN

【概念/定义】：

卷积神经网络（CNN）是一种以卷积层为核心的深度神经网络，擅长提取局部特征。

【结构】：

数据输入层 → 卷积层 → 激活层 → 池化层 → 全连接层 → 输出层。

【特征/特点】：

- PyTorch 中卷积层和激活函数通常是独立的。
- Keras / TensorFlow 中常把激活函数集成到卷积层中。

【概念/定义】：

输入层负责接收原始数据。

【特征/特点】：

- **灰度图**：二维矩阵（宽 × 高）。
- **RGB 彩色图**：三维矩阵（宽 × 高 × 输入通道数）。

- *例*：一张 224×224 彩色图片输入 CNN 后形状为：$224 \times 224 \times 3$。

【概念/定义】：

卷积层通过卷积核在输入上滑动，自动提取局部特征，生成特征图（Feature Map）。

【作用/目的】：

- **浅层**：提取边缘、纹理等低级特征。
- **深层**：提取眼睛、耳朵、物体部件等高级特征。

【方法/技术】：

- 卷积核（Filter / Kernel）是一组可学习的权重矩阵。
- **卷积运算本质**：局部区域对应元素相乘后求和。

【计算/公式】：

单个卷积窗口计算：

$$y = \sum_i \sum_j x_{ij} k_{ij}$$

(其中：x_{ij} 为输入局部区域数值，k_{ij} 为卷积核权重)

输出特征图尺寸公式：

$$H_{out} = \lfloor \frac{H_{in} + 2P - K}{S} \rfloor + 1$$

$$W_{out} = \lfloor \frac{W_{in} + 2P - K}{S} \rfloor + 1$$

(其中：H_{in}, W_{in} 为输入高度和宽度；K 为卷积核大小；S 为步幅 stride；P 为填充 padding)

- *例：输入 100×100，卷积核 3×3，步幅 $S = 2$，填充 $P = 1$，则：*
- $H_{out} = \frac{100 + 2 \times 1 - 3}{2} + 1 = 50$
- $W_{out} = \frac{100 + 2 \times 1 - 3}{2} + 1 = 50$
- 输出特征图尺寸为：50×50。

【计算题重点】：

- 给出输入尺寸、卷积核大小、步幅、padding，计算输出尺寸。公式中是 $+2P$，千万不要漏掉前面的系数 2。
- 若结果不是整数，实际框架中通常涉及向下取整。

【方法/技术】：

卷积层关键参数：

- **卷积核大小**：决定每次提取特征的局部区域范围。常见如 $3 \times 3, 5 \times 5$。卷积核越大，感受野越大，但参数和计算量也越大。

- **步幅 stride (S)**：每次滑动的距离。 S 越小，提取特征越多； S 太大可能漏掉信息。
- **填充 padding (P)**：在输入矩阵周围补 0。作用是保留边界信息，控制输出尺寸。
  - `SAME`：不足卷积核大小时补 0。
  - `VALID`：不足卷积核大小时丢弃。

【方法/技术】：

- N 个卷积核输出 N 张特征图。
- **输出通道数 = 卷积核数量**。
- 更深层卷积核可以提取更抽象的语义特征。

【概念/定义】：

- **2D 卷积**：处理平面图像，卷积核在宽 × 高平面上滑动。
- **3D 卷积**：处理立体数据（如视频），卷积核在宽 × 高 × 深度/时间上滑动。

【应用场景】：

- 2D 卷积：图像分类、目标识别。
- 3D 卷积：视频理解、动作识别、医学三维影像。

【特征/特点】：

卷积层两大优势：

1. **参数共享**：同一个卷积核在不同位置复用，大幅减少参数量。
2. **局部连接**：每个神经元只连接局部窗口，相比全连接减少连接数。

【概念/定义】：

激活层对卷积层输出做非线性映射。

【作用/目的】：

- 引入非线性。如果没有激活函数，多层网络仍等价于单层线性变换，无法拟合复杂函数。

【方法/技术】：

常用激活函数：Sigmoid、Tanh、ReLU、Leaky ReLU、Softmax、GELU。

【计算/公式】：

ReLU 激活函数：

$$ReLU(x) = \max(0, x)$$

即：

$$ReLU(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

【计算题重点】：

给一个矩阵让其通过 ReLU，规则很简单：**正数保留，负数变 0，0 仍为 0**。

【概念/定义】：

池化层通常跟在卷积层后，用于压缩特征图。

【作用/目的】：

特征降维、压缩数据量和参数量、减小过拟合。

【方法/技术】：

- **最大池化 Max Pooling**：取窗口最大值。
- **平均池化 Average Pooling**：取窗口平均值。

【计算/公式】：

- 最大池化：$y = \max(x_1, x_2, \ldots, x_n)$
- 平均池化：$y = \frac{1}{n} \sum_{i=1}^{n} x_i$

【计算题重点】：

给一个矩阵和池化窗口（如 2×2），要求做 Max 或 Average Pooling。注意**步幅 stride**，其会影响输出大小。

【概念/定义】：

全连接层（FC）中每个节点都与上一层所有节点相连。

【作用/目的】：

综合前面卷积层和池化层提取到的特征，用于分类或回归前的特征整合。

【特征/特点】：

参数量最多，容易占用大量内存。

- *例：若上一层有 25088 个节点，全连接层有 4096 个节点，则权重数量为* 25088×4096 。

【计算题重点】：

给上一层节点数和当前全连接层节点数，计算参数量：

$$\text{参数量} = \text{输入节点数} \times \text{输出节点数} + \text{输出节点数}$$

(其中最后一项为偏置参数 bias)

【概念/定义】：

输出层是 CNN 的最后一层，分类任务中常使用 **全连接 + Softmax**。

【作用/目的】：

为每个类别输出一个概率，选择概率最大类别作为预测结果。

【计算/公式】：

Softmax 公式：

$$P(y=i)=\frac{e^{z_i}}{\sum_{j=1}^{C} e^{z_j}}$$

【概念/定义】：

Dropout 是一种正则化方法，用于缓解过拟合。

【方法/技术】：

- 训练时以概率 p 随机断开部分神经元连接（每个 batch 随机丢弃部分输入）。
- **作用**：防止模型过度依赖某些特征，提高泛化能力。
- **局限**：一般只在**训练阶段**使用，推理（测试）阶段通常不再随机丢弃。

NLP 中的卷积神经网络

【概念/定义】：

CNN 也可用于 NLP（如文本分类），通常被称为 **TextCNN**。

【方法/技术】：

输入句子转换为词向量矩阵 $\rightarrow$ 经过 1D 卷积层提取局部 n-gram 语义特征 $\rightarrow$ 经过 Max Pooling 得到句向量表示 $\rightarrow$ 送入全连接层 $\rightarrow$ Softmax 输出类别概率。

【输入形式】：

句子中所有单词组成词向量矩阵：

$X \in \mathbb{R}^{n \times k}$

(其中：n 为句子中单词数，k 为词向量维度。每一行对应一个单词的词向量，按原始顺序排列。)

【方法/技术】：

NLP 中卷积核大小通常为：$h \times k$

(其中 h 为窗口包含的词数，k 为词向量维度)

- **核心特征**：卷积核列数固定为 k （与词向量维度一致）；卷积核高度 h 可变，用于提取不同长度的 n-gram 特征。
- **多尺寸卷积核**：可以同时捕捉不同长度的局部语义（例如 $h=2$ 提取 2-gram 特征）。

【优点/局限】：

- **优点**：计算高效、可并行训练、局部特征提取能力强、不容易过拟合。
- **局限**：**难以建模长距离依赖**，相比 RNN / Transformer 对全局词序和长程上下文建模较弱。

【计算/公式】：

若输入文本矩阵为 $n \times k$，卷积核为 $h \times k$，stride 为 1，则 `VALID` 卷积后的输出序列长度为：

$$\text{长度} = n - h + 1$$

【计算题重点】：

给出句长 n 、词向量维度 k 、卷积核高度 h ，判断卷积核大小和输出长度。**记住：NLP 中卷积核的宽度固定等于词向量维度 k 。**

卷积神经网络实现

【方法/技术】：

TextCNN 基本结构：`Embedding 层` → `Conv1d 卷积层` → `ReLU 激活` → `第二个 Conv1d` → `全局平均池化` → `全连接输出`。

【方法/技术】：

常见训练步骤：

1. 定义模型与损失函数（如交叉熵 `CrossEntropyLoss`）。
2. 定义优化器（如 `Adam`）。
3. 前向传播计算 loss → **梯度清零**（`optimizer.zero_grad()`） → 反向传播（`loss.backward()`） → 参数更新（`optimizer.step()`）。

【代码理解重点】：

- `nn.Embedding(vocab_size, embed_dim)`：把 token ID 转成词向量。
- `transpose(1, 2)`：把形状转为 Conv1d 需要的形式：$(\text{batch_size}, \text{embed_dim}, \text{seq_len})$。
- `nn.Conv1d(embed_dim, 32, 3, padding=1)`：输入通道数等于 `embed_dim`，输出通道数 32，卷积核大小 3，padding 为 1。
- `adaptive_avg_pool1d(x, 1)`：全局平均池化，将序列长度维压缩为 1。
- 可使用 `nn.Embedding.from_pretrained` 导入预训练词向量，设置 `freeze=True` 表示不更新其参数。

循环神经网络 RNN

【概念/定义】：

RNN 是适合处理序列数据的神经网络。其核心特征是**同一层神经元之间也存在连接**，上一个时刻的隐状态会影响下一个时刻。

【特征/特点】：

具有时序依赖建模能力。当前时刻的输出同时依赖于当前时刻的输入与上一时刻的隐状态。

【应用场景】：

下个词预测、文本分类、序列标注、机器翻译、天气/温度预测。

RNN 模型应用 (1) - 文本分类

【方法/技术】：

- **末态池化方式**：使用**最后时刻的隐状态**（h_T）作为整个句子的向量表示，再接全连接层进行分类。
  - *局限*：长序列中，最后隐状态可能无法完整保留长距离前的全部信息。
- **平均池化方式**：使用**每个时间片的隐状态**，对所有时间片的隐状态求平均得到句向量：
- $h = \frac{1}{T}\sum_{t=1}^{T} h_t$

【代码理解重点】：

- `nn.LSTM(embedding_dim, hidden_dim, batch_first=True)`：定义 LSTM 网络。
- `output, (h_n, c_n)`：
  - `output` 包含所有时间片的隐状态。
  - `h_n` 为最后时刻的隐状态（末态）。
  - `c_n` 为最后时刻的细胞状态（Cell State）。
- `h_n.squeeze(0)`：去掉层数维度后送入 `fc(h_n)` 进行分类输出。

【方法/技术】：

BiLSTM（双向 LSTM）：可同时建模前向和后向上下文。

- **输出维度计算**：由于拼接了正向和反向的隐状态，其输出维度一般为 $2 \times \text{hidden_dim}$。

RNN 模型应用 (2) - 序列预测

【概念/定义】：

序列预测（序列标注）中，输入序列和输出序列的**长度相等**，每个位置都有一个对应的预测标签（即 N-N 结构）。

【应用场景】：

词性标注、命名实体识别（NER）、分词。

Encoder-Decoder 结构

【概念/定义】：

Encoder-Decoder 结构用于输入序列和输出序列**长度不确定/不等**的任务。**Seq2Seq** 是其一种典型实现，通常用两个 RNN 分别作为编码器和解码器。

【工作机制】：

1. **Encoder** 负责阅读输入序列，并将其压缩为一个固定的语义向量 c。
2. **Decoder** 根据语义向量 c 逐步生成输出序列。

【语义向量 c 的编码方式】：

- 使用 Encoder 最后时刻的隐状态。
- 将整个序列所有时刻的隐状态累加并求平均。

【局限/不足】：

- **信息瓶颈**：固定长度的语义向量 c 容易丢失输入长序列的信息。
- 无法准确对输入和输出序列的对应关系建模（解码时无法动态关注输入的不同部分）。

【方法/技术】：

- **Teacher Forcing**：在训练阶段，不使用 Decoder 上一步自身生成的预测结果作为当前输入，而是强制把真实的标签（Target）作为 Decoder 的当前输入。
  - *作用*：加快模型训练收敛，减少训练早期的错误累积。

注意力机制 (Attention)

【概念/定义】：

注意力机制是一种模拟人类注意力的结构，使模型在处理数据时能够**动态关注输入中更重要的部分**，并分配不同的权重，有效解决了 Encoder-Decoder 的固定向量瓶颈。

【计算/公式】：

1. **点积注意力相关性分数 (Score)**：
2. $\text{score}(Q, K) = QK^T$
3. **缩放点积注意力 (Scaled Dot-Product Attention)**：
4. $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$
5. *(其中：Q 为查询向量 Query；K 为键向量 Key；V 为值向量 Value；d 为输入向量维度，$\sqrt{d}$ 用于缩放以防止分数过大导致梯度消失)*

【计算题重点】：

给出注意力权重和对应的 Value值，计算加权求和。

- *例：专家评分：计算机=9，自动化=7，外语=4。对应权重为：0.6，0.35，0.05。*
- 输出 $= 0.6 \times 9 + 0.35 \times 7 + 0.05 \times 4 = 8.05$

【应用场景】：

在 Seq2Seq 序列预测中：**解码器当前的隐状态作为 Query**，而**编码器各时刻的隐状态作为 Key 和 Value**。每个解码时刻都会计算出不同的上下文向量（context vector）。

主流深度学习框架

【典型框架】：

PyTorch、MindSpore。

【代码理解重点 (MindSpore)】：

设置保存参数：

Python

```
代码块
1   CheckpointConfig(save_checkpoint_steps=1875, keep_checkpoint_max=10)
```

- **含义**：每 1875 个 step 保存一个 checkpoint，磁盘上最多保留最新的 10 个文件。
- **手动保存**：使用 `ms.save_checkpoint(model, ckpt_file_name)` 。

9大语言模型-1

在不修改任何实质内容的前提下，我已经为你将原标题全部**降低一级**（原一级标题变为二级标题，原普通正文层级依此类推变为三级或加粗），剔除了原文本中重复渲染的冗余公式，并将整体排版规范化为飞书 Markdown 与 LaTeX 的最佳实践标准。

同时，在架构概述与自注意力部分，为你添加了直观的架构图示标记，方便在飞书中复习。

你可以直接复制下方内容粘贴到飞书文档中：

1 Transformer概述

Transformer 是 Google 在论文《Attention Is All You Need》中提出的神经网络架构，主要用于 NLP，也扩展到计算机视觉等领域。

LSTM / RNN 处理序列时是串行的，必须等当前词处理完，才能处理下一个词。

【Transformer 的核心优势】：

- 解决 RNN 无法并行计算的问题。
- 改善 RNN 长序列依赖捕捉弱的问题。
- 兼顾并行效率和长文本建模能力。

【开卷考试常考】：

- Transformer 为什么比 RNN 更适合长序列?
- Transformer 的核心机制是什么?
- Transformer 解决了 RNN 的哪些问题?

2 Transformer整体架构

Transformer 由 Encoder 和 Decoder 两部分组成。

- 每个 Encoder 和 Decoder 都包含多个 block，原始 Transformer 中通常各有 6 个 block。
- 所有编码器、解码器结构相同，但不共享参数。

【Encoder block 主要包括】：

- Multi-Head Attention（多头自注意力）
- Feed Forward（前馈神经网络）
- Add & Norm（残差连接与层归一化）

【Decoder block 主要包括】：

- Masked Multi-Head Attention（掩码多头自注意力）
- Encoder-Decoder Attention / Cross Attention（交叉注意力）
- Feed Forward（前馈神经网络）
- Add & Norm（残差连接与层归一化）

> *注：图中的 N 表示多个 Encoder 或 Decoder block 的堆叠。*

【开卷考试常考】：

- Transformer 由哪两部分组成?
- Encoder 和 Decoder 的结构区别?
- 为什么说 Encoder / Decoder 结构相同但参数不共享?

3 编码器

3.1 编码器的作用

编码器将输入序列中的每个元素先表示成独立向量，再通过编码器转换成包含上下文信息的向量表示。输入词经过编码器后，每个词的输出向量不再只表示词本身，而是融合了整句话的上下文。

- *例如*: “我”、“有”、“一只”、“猫”，每个词都会得到一个对应的上下文向量。

【开卷考试常考】：

- Encoder 的作用是什么?
- 为什么编码器输出的向量包含上下文信息?

3.2 输入层

单词 Embedding

单词 Embedding 将输入序列中的每个词转换为固定大小的词向量。词向量包含词的语义信息。可通过 Word2Vec、GloVe 等方法获得。

【开卷考试常考】：

- 什么是词 Embedding?
- Embedding 的作用是什么?

位置 Embedding

Transformer 本身没有 RNN 的顺序结构，因此需要位置编码来表示词在句子中的位置。

- *例如*：“张明欠了我300元” 与 “我欠了张明300元”，两句话词非常相似，但顺序不同导致语义完全不同，所以必须引入位置信息。

位置 Embedding 的维度与单词 Embedding 相同。位置编码可以通过训练得到，也可以通过公式计算得到。Transformer 中采用的是绝对位置编码中的**正弦位置编码**。

【公式必会】：

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

- **参数含义**：
  - pos：单词在句子中的位置。
  - d：向量总维度（例如 512）。
  - i：位置编码的维度序号，从 0 到 $d/2-1$。
- *例子*：对于位置 0 的词，位置编码类似：$[0, 1, 0, 1, 0, \cdots]$

【正弦位置编码的优势】：

- 两个位置编码的点积只取决于相对距离。
- 取值范围在 $[-1, 1]$，不会随句长无限增大。
- 每个位置都有唯一编码。

【最终输入表示】：

$$X = \text{TokenEmbedding} + \text{PositionEmbedding}$$

【其他位置编码方法】：

相对位置编码（如 ALiBi）、旋转位置编码（RoPE）、动态位置编码。

【开卷考试常考】：

- 为什么 Transformer 需要位置编码?
- 正弦位置编码公式及给定 pos, d, i 计算位置编码。
- 输入层最终表示为什么是相加（$+$）而不是拼接（concat）。

3.3 自注意力机制

Self-Attention 是 Transformer 的核心组成部分。它通过计算输入序列中每个单词与其他单词之间的关系，生成包含上下文信息的表示。

- *例如输入*：`I love Transformer`
- 当处理 `love` 时，它会同时与 `I`、`love`、`Transformer` 计算注意力分数。如果 `love` 和 `Transformer` 关系更重要，模型会让 `love` 的表示更多融合 `Transformer` 的信息。

【开卷考试常考】：

- 什么是自注意力?
- 自注意力为什么能捕捉全局依赖?
- 自注意力和 RNN 的区别?

自注意力机制实现

每个输入词向量通过三个可学习矩阵映射为：

- **Query (** q **)**：查询向量
- **Key (** k **)**：键向量
- **Value (** v **)**：值向量

【计算公式】：

$$q_i = W_q x_i$$

$$k_i = W_k x_i$$

$$v_i = W_v x_i$$

之后按照标准注意力机制流程计算：

1. Query 和 Key 做点积，得到注意力分数。
2. 对分数做缩放（Scaled）。
3. 经过 softmax 得到权重。
4. 用权重对 Value 加权求和。

【开卷考试常考】：

- Q, K, V 分别是什么？它们是怎么得到的?
- 自注意力完整的计算流程。

在不修改任何实质内容的前提下，我已经为你将原标题全部**降低一级**（接续上一节，原普通层级转换为三级标题，其下小标题转换为四级标题），剔除了原文本中重复渲染的冗余公式，并将整体排版规范化为飞书能完美渲染的 Markdown 与 LaTeX 最佳实践标准。

此外，针对多头注意力与 LayerNorm 的核心对比，为你添加了直观的架构与原理解析图示标记，方便在飞书中复习。

3.4 注意力分数计算

【计算题高频】

【计算公式】：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- **参数含义**：
  - QK^T：计算 Query 和 Key 的相似度。
  - $\sqrt{d_k}$：缩放因子，防止点积值过大导致梯度消失。
  - softmax：把分数转成概率权重。
  - 乘以 V：得到加权后的上下文表示。

【计算步骤】：

1. 计算矩阵乘法 QK^T 。
2. 将结果除以缩放因子 $\sqrt{d_k}$ 。
3. 对所得矩阵的**每一行**分别做 softmax 归一化。
4. 将得到的权重矩阵左乘 V 。

【⚠️ 易错点提示】：

- 最后相乘的是 V ，**不是** K 。
- softmax 是针对每个 query（即矩阵的每一行）的分数独立进行的。
- 动手计算时千万不要漏掉 K 的转置（ ^{T} ）和分母的 $\sqrt{d_k}$ 。

【开卷考试常考】：

- 给定具体的 Q、K、V 矩阵，手动计算 Attention 输出。
- 简答题：解释为什么要除以缩放因子 $\sqrt{d_k}$ 。

3.5 多头注意力机制

- Transformer 使用 Multi-Head Attention，其角色类似于 CNN 中的多个卷积核（kernel），不同的头（head）可以从**不同的特征子空间**独立提取关系。

【多头注意力的核心思想】：

- 将原来的单个参数矩阵 W_q, W_k, W_v 拆分或转变为多组参数矩阵。
- 每个头独立进行注意力分数的计算。
- 拼接（concat）所有头计算出的结果。
- 最终通过一个线性变换矩阵映射回原始维度。

【关键流程】：

输入向量 $\rightarrow$ 通过多组 W_q, W_k, W_v 映射得到多组 Q、K、V $\rightarrow$ 每个 head 单独计算各自的 attention $\rightarrow$ 拼接（concat）多个 head 的输出 $\rightarrow$ 通过 W^O 映射回原维度。

【开卷考试常考】：

- 为什么要使用多头注意力机制？它有什么优势？
- 多头注意力和单头注意力的主要区别是什么？
- 多头注意力的完整计算流程。
- 给出总隐藏层维度（hidden size）和头数（head 数量），判断计算每个头对应的具体维度（$\text{head_dim} = \text{hidden_size} / \text{head_num}$）。

3.6 Add & LayerNorm

残差结构 Add

- Transformer 引入残差连接（Residual Connection）是为了解决深度网络在训练过程中可能面临的训练困难与梯度消失问题。
- **本质**：跨层连接。无论中间层进行了多么复杂的变换，都将最原始的输入向量信息直接叠加传递下去。

【公式形式】：

$$\text{输出}_1 = \text{LayerNorm}(X + \text{MultiHeadAttention}(X))$$

$$\text{输出}_2 = \text{LayerNorm}(Y + \text{FeedForward}(Y))$$

> **反向传播数学原理**：加入残差路径后，在对输入 X 求偏导时会额外多出一项常数 1（即 $\frac{\partial(X + f(X))}{\partial X} = 1 + f'(X)$），从而有效保障了梯度能顺畅向前传递。

【开卷考试常考】：

- 残差连接在深层网络中的核心作用。
- 为什么残差连接可以从数学上缓解梯度消失？

层归一化 Layer Normalization

[Image comparing Layer Normalization and Batch Normalization visualizing the axis of normalization across feature and batch dimensions]

【核心作用】：

- 将神经网络隐藏层的输出进行归一化。
- 减少不同网络层之间的分布漂移（Internal Covariate Shift），从而稳定整个训练过程。
- **计算逻辑**：LayerNorm 是对**单个样本内部的所有特征**独立计算均值和方差。

【计算公式】：

$$\text{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \times \gamma + \beta$$

(其中：μ 为均值；σ^2 为方差；ϵ 是防止除零的小常数；γ 和 β 是可学习的缩放与平移参数)

【具体步骤计算】：

1. **均值**：$\mu = \frac{1}{d}\sum_{i=1}^{d} h_i$
2. **方差**：$\sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(h_i - \mu)^2$
3. **归一化**：$\hat{h}_i = \frac{h_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$

【LayerNorm 与 BatchNorm 的关键区别】：

- **LayerNorm**：对**单个样本**内部的所有特征计算均值和方差（适合变长序列的 NLP 任务）。
- **BatchNorm**：对**一个 batch 内所有样本**的同一特征维度计算均值和方差（适合 CV 任务）。

【开卷考试常考】：

- LayerNorm 的具体代数计算题（常给出小矩阵求标量归一化结果）。
- LayerNorm 和 BatchNorm 的本质区别与各自的应用场景。
- LayerNorm 的主要作用。

3.7 针对单个词的完整编码过程

【流水线步骤】：

1. 传入原始输入词向量（Token Embedding）。
2. 叠加位置编码（Position Embedding）。
3. 进入多头自注意力（Multi-Head Attention）层计算。
4. 经过残差连接（Add）。
5. 实施层归一化（LayerNorm）。
6. 送入前馈神经网络（Feed Forward）。
7. 再次经过残差连接与层归一化（Add & Norm）。
8. 输出最终的编码器向量表示。

【开卷考试常考】：

- 简述 Transformer Encoder 对单个单词处理的完整流转逻辑。
- 解释 Add & Norm 模块在 block 中出现的精确位置。

3.8 前馈神经网络 (FFN)

- Transformer 中的 Feed Forward 层是一个标准的**两层全连接（Linear）网络**，中间夹着一个 **ReLU** 激活函数。

【计算公式】：

$$\mathrm{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

【核心功能】：

- 用于接收并进一步处理自注意力层整合出来的上下文特征信息。
- FFN 的输出即为该编码器 Block 的最终输出。
- **至关重要**：在整体网络架构中，编码器最后一层输出的特征向量，将被源源不断地送入 Decoder，作为其 **Cross Attention（交叉注意力）中的 Key (** K **) 和 Value (** V **)** 来使用。

【开卷考试常考】：

- FFN 的网络组成结构及其标准数学公式。
- 解释 Encoder 最终输出的特征在 Decoder 中起到了什么具体作用。

4 解码器

4.1 解码器的作用

解码器（Decoder）根据**编码器的输出**以及**上一步已经预测出的结果**，来共同进行下一次的 Token 预测。

Decoder 的基本网络结构和 Encoder 类似，但内部多了 **Masked Multi-Head Attention** 和 **Cross Attention**（交叉注意力）两部分。

每个 Decoder 包含两个 Multi-Head Attention 层：

1. **第一个**：Masked Multi-Head Attention（带掩码的多头自注意力）。
2. **第二个**：Encoder-Decoder Attention，也叫 **Cross Attention**（交叉注意力）。
    - *在 Cross Attention 中*：查询向量 **Q 来自 Decoder** 内部前一层的输出；键向量 **K 和值向量 V 来自 Encoder 的最终输出**。

【开卷考试常考】：

- Decoder 的核心作用是什么？
- Decoder 和 Encoder 在结构上有哪些关键区别？
- Cross Attention 层的 Q、K、V 分别来自于哪里？

4.2 带掩码的多头注意力机制

- Decoder 的第一个 Attention 层必须使用 Masked（掩码）操作。

【为什么要进行 Mask】：

- Transformer 在训练阶段是**并行计算**的。
- 如果不施加 Mask，Decoder 在训练当前位置的 Token 时，就能直接提前“看到”未来位置的真实标签（Ground Truth），从而导致严重的**信息泄露**（Data Leakage），使得模型无法学会真正的自回归预测。

【因果掩码 Causal Mask】：

- 保证第 t 个 token 在计算注意力时，**只能关注自己和自身之前的 token**，不能偷看未来的 token。
- *例如长度为 5 的序列*:
  - 第 0 个词只能看第 0 个词。
  - 第 1 个词可以看第 0、1 个词。
  - 第 2 个词可以看第 0、1、2 个词。
- **因此，Mask 矩阵在数学上一般呈现为下三角矩阵（Lower Triangular Matrix）的形式。**

【具体计算过程】：

根据输入向量计算 Q、K、V $\rightarrow$ 用 Q 和 K 做点积计算初始注意力分数 $\rightarrow$ **将注意力分数与 Mask 矩阵结合（将未来位置的分数赋为 $-\infty$）** $\rightarrow$ 屏蔽未来位置 $\rightarrow$ 经 $\mathrm{softmax}$ 转化为概率权重后乘以 V 。

【开卷考试常考】：

- 为什么 Decoder 必须引入 Mask 机制?
- 给定序列长度，要求手动写出对应的因果掩码（Causal Mask）矩阵。
- 给定具体位置，判断该位置在解码时能够关注哪些 token。

4.3 交叉注意力层 (Cross Attention)

- Cross Attention 负责让 Decoder 根据 Encoder 的输出特征生成注意力，从而让当前解码位置能够精准关注输入序列中关系最密切的上下文信息。这在机器翻译（Seq2Seq）等跨序列任务中至关重要。

【与 Self-Attention 的本质不同】：

- **Self-Attention**：Q、K、V 三者全部来自**同一个**输入序列。
- **Cross Attention**：
  - Q 来自 Decoder 内部前一层（Masked Attention）的输出。
  - K 和 V 则完全来自 Encoder 的最终输出向量。

【开卷考试常考】：

- Cross Attention 和 Self-Attention 的核心区别。
- Cross Attention 中 $Q/K/V$ 的精确来源。

- 为什么文本生成或翻译任务中必须引入 Cross Attention?

4.4 输出层

Decoder 最终的输出向量需要依次经过 **Linear 层** 与 **Softmax 层**，从而转换成词表中每一个 token 的预测概率。

- *例*：如果预设词表大小为 1080，那么模型输出的 Logits 向量长度即为 1080。
- **Logits** 中的每一个数值代表对应单词的原始未归一化预测得分。
- **Softmax** 负责将这些原始得分转换为符合概率分布的数值（总和为 1）。

【计算公式】：

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

【开卷考试常考】：

- Logits（原始得分）和概率（Probability）的区别是什么?
- softmax 的手动计算（如给出少量得分计算最终概率）。
- 输出层为什么要依次连接 Linear 层和 Softmax 层?

4.5 贪心搜索 (Greedy Search)

- **核心策略**：在解码的每一步，直接选择当前 Softmax 输出中概率最大的那一个 token。

【优缺点分析】：

- **优点**：算法简单，解码速度极快。
- **缺点**：只注重局部最优，**极易丢失全局最优解**；且生成的文本结果容易陷入死循环，出现重复、单调的语句。

【开卷考试常考】：

- 贪心搜索的完整流程。
- 为什么贪心搜索不一定能得到全局最优生成的序列?

4.6 束搜索 (Beam Search)

- **核心策略**：在解码的每一步，不只保留概率最大的那一个单个 token，而是动态保留当前累积概率最高的 **Top-k 个候选路径**。这个 k 被称为 **Beam Size（集束宽度）**。

【解码目标】：

使得整个生成序列的联合概率最大化：

$$\arg\max \ P(y_1, y_2, \cdots, y_T|x)$$

其中序列的联合概率计算公式为：

$$P(y_1, \cdots, y_T|x) = \prod_{t=1}^{T} P(y_t|y_{<t}, x)$$

> **特例说明**：当 $\text{Beam Size } (k) = 1$ 时，Beam Search 算法将完全退化为贪心搜索。

【开卷考试常考】：

- Beam Search 和 Greedy Search 的本质区别。
- 给出每一步的转移概率，要求手动计算并筛选出 Beam Search 保留的候选路径。
- 当 $\text{top-k} = 1$ 时，Beam Search 会发生什么情况?

4.7 随机采样 (Random Sampling)

- **核心策略**：根据模型输出的真实概率分布来进行随机抽样选择 token。概率高的 token 被选中的概率大，但概率低的 token 同样有一定几率被选中。

【特点与优缺点】：

- **特点**：不强制选择概率最高的词，为文本生成引入了随机性和创造性。
- **优点**：生成文本的多样性强、听起来更自然、更有创意。
- **缺点**：输出不稳定，多次运行结果不同，模型有时可能会“跑偏”或生成胡话。

关键调节参数：温度参数 T (Temperature)

温度参数 T 用于在 Softmax 前对 Logits 进行缩放（Logits/T），从而调整概率分布的平滑度：

- T **较大**：概率分布会被拉得**更平缓**，各个词的概率差距变小，**随机性和创造性更强**。
- T **较小**：概率分布会变得**更陡峭/集中**，高概率词的优势被放大，生成结果**更倾向于保守和确定性**。

随机采样的两大核心变体：

- **Top-k 采样**：每一步只从概率最高的前 k 个词组成的截断集合中进行按比例采样。
- **Top-p 采样（核采样 Nucleus Sampling）**：将词表按概率从大到小排序，从中取出一个最小的词集合，使得这个集合的**累积概率刚刚超过阈值** p，然后在这个集合内进行采样。

【开卷考试常考】：

- 温度参数（Temperature）在 LLM 解码采样中的具体调节作用。
- Top-k 采样与 Top-p 采样的机制区别。
- 随机采样和贪心搜索在文本生成质量和多样性上的核心差异。

5 预测与训练

5.1 编码器训练

- Encoder 接收完整的输入序列。
- 顶层 Encoder 为序列中的每个词输出富含上下文信息的编码向量。
- **计算特性**：编码器整体可以实现**并行计算**。
- **架构连接**：Encoder 最后一层输出的特征向量会送入 Decoder 的 Cross Attention（交叉注意力）层，作为其键向量 K 和值向量 V 。

【开卷考试常考】：

- Encoder 为什么能够支持并行训练？（因为没有时序依赖，各 Token 的自注意力计算相互独立）
- Encoder 的输出向量在 Decoder 中起到了什么具体作用？

5.2 解码器训练

- **预测（推理）过程**：Decoder 每一步自回归地输出一个元素，直到生成终止符（如 `<EOS>`）。这一过程是**串行**的，因为生成下一步的输入必须依赖上一步的真实输出。
- **训练阶段**：可以通过 **Teacher Forcing（教师迫导）** 机制来实现**并行训练**。

【Teacher Forcing 机制】：

- 每一轮预测时，不使用上一轮模型实际预测出来的（可能错误的）结果，而是**强制使用正确答案（Ground Truth）中的真实 Token** 作为当前步的输入。

【Decoder 支持并行训练的两大核心关键】：

1. **Teacher Forcing**：阻断了错误累积，使各时步解耦。
2. **Masked Self-Attention**：通过下三角掩码矩阵屏蔽未来信息，保证了并行的合法性。

【开卷考试常考】：

- 为什么 Transformer 的训练可以实现并行，但模型预测（推理）通常只能串行？
- 什么是 Teacher Forcing 机制？它有什么优缺点？
- Decoder 能够进行并行训练主要依赖哪两个核心技术/机制？

6 Transformer 关键变体

6.1 Decoder-only（仅解码器）结构

- Decoder-only 架构只保留了原始 Transformer 中的解码器部分。
- 它通过**单向自注意力**（即因果注意力 Causal Attention）来屏蔽未来的信息，从而实现自回归生成。

【核心能力与应用场景】：

- 擅长**文本生成、对话交互、文本续写**等。
- **代表模型**：GPT 系列、LLaMA 系列、千问（Qwen）等现代大语言模型。

【工作/自回归流程】：

输入提示词（Prompt） → 经过多层 Decoder 块 → 预测下一个词 → **将新生成的词拼接加入到输入序列中** → 重复此自回归循环，直到预测出结束符。

【预训练任务】：

自监督因果语言建模（Causal Language Modeling）：根据前文已知的所有内容，预测下一个词。

【开卷考试常考】：

- GPT / LLaMA / Qwen 属于 Transformer 的哪种关键变体结构?
- 为什么 Decoder-only 结构最适合做大规模文本生成任务?
- 什么是“自回归生成（Autoregressive Generation）”？

GPT 模型

- GPT 是 OpenAI GPT 系列的开山之作，采用标准的 **Decoder-only** 结构，内部由多个 Decoder 块串联而成。
- GPT 使用单向自注意力机制，每个位置在计算时只能看到当前位置及之前的内容。

【开卷考试常考】：

- GPT 为什么是典型的单向模型?
- GPT 与 BERT 在底层网络架构上的本质区别是什么?

6.2 Encoder-only（仅编码器）结构

- Encoder-only 架构只保留了原始 Transformer 中的编码器部分。
- 它使用**双向自注意力机制**（无掩码遮挡），能够同时看到上下文的所有信息，从而完美理解完整输入文本。

【核心能力与适合任务】：

- 擅长**文本理解与特征提取**。
- **适合任务**：情感分类、命名实体识别（NER）、问答系统（QA）、文本匹配。
- *局限*：不适合直接用来生成长文本。

【工作流程】：

输入完整文本序列 → 经过多层 Encoder 块 → 输出每个 token 包含双向上下文信息的特征向量 → 外接不同的下游任务头（Task-specific Heads）来完成分类、标注等具体任务。

> *注意：原始 Transformer Encoder 本身是不带这些输出层任务头的。*

【代表模型】：

BERT、RoBERTa。

【开卷考试常考】：

- BERT 属于 Transformer 的哪种变体结构?
- 为什么 Encoder-only 结构非常适合文本理解任务，而不擅长文本生成?
- Encoder-only 和 Decoder-only 架构的核心区别。

6.3 Encoder-only 结构用于计算机视觉

Vision Transformer (ViT)

- **Vision Transformer（ViT）** 成功将 Transformer 架构引入了计算机视觉（CV）领域，用以替代或辅助传统的卷积神经网络（CNN）来处理图像数据。
- ViT 的骨干网络架构与 Transformer Encoder 非常相似。

【核心处理流程】：

1. **图像分块（Patch Embedding）**：将一张完整的连续图像切成固定大小的、互不重叠的图像块（Patches，例如 16×16 像素）。
2. **线性投影（Linear Projection）**：通过线性映射将每个二维图像块展平并转换为一维的嵌入向量。
3. **加入位置编码**：因为一维序列丢失了图像的空间相对位置，必须叠加位置编码（Position Embeddings）以保留空间结构信息。
4. **送入 Transformer 编码器**：最后将这一串 Token 向量序列输入标准 Transformer Encoder 进行多头自注意力特征提取。

【开卷考试常考】：

- ViT 是如何创造性地把二维图像转换为 Transformer 可以直接处理的一维序列的?
- 在 ViT 架构中，图像块（Patch）的作用是什么?

7 BERT 模型

BERT **只使用 Transformer 的编码器（Encoder）部分**，通过堆叠多层 Encoder 构建而成，专注于**双向上下文表征学习**。

- **输入**：文本序列中各 token 的原始词向量。
- **输出**：每个 token 融合了双向上下文信息后的高维特征向量。每一层都可以输出向量，用于不同的下游任务。

【开卷考试常考】：

- BERT 为什么采用 Encoder-only 架构？（因为理解任务需要结合前后双向的上下文信息）
- BERT 为什么适合文本理解类任务?
- BERT 的输入和输出分别是什么形式?

7.1 BERT 注入层 (Input)

第一步：Tokenization (分词)

输入句子首先经过分词器处理。分词后必须在序列中添加特殊控制字符：

- `[CLS]`：专用分类标记，固定放在**整个句子的开头**。
- `[SEP]`：分隔标记，固定放在**句子结尾**或**两个句子之间**用以区分。
- 分词并添加特殊字符后，所有 token 转换为对应的数字 id（Token IDs）。

【开卷考试常考】：

- 特殊标记 `[CLS]` 和 `[SEP]` 的核心作用是什么？
- BERT 标准的输入序列格式是怎样的？

第二步：Embedding (嵌入)

BERT 的输入向量是由以下三种特征嵌入相加（Summed）组合而成的：

1. **Token Embeddings**：表示词的字面具体内容（由 token ids 映射）。
2. **Position Embeddings**：表示词在序列中的绝对位置（表示位置信息）。
3. **Token Type Embeddings**（也叫 Segment Embeddings）：用于区分不同的句子。

【计算公式】：

$$\text{InputEmbedding} = \text{TokenEmbedding} + \text{PositionEmbedding} + \text{TokenTypeEmbedding}$$

> **句子对任务中的规范**：第一句的所有词其 `token type id` 通常设为 `0`；第二句的所有词其 `token type id` 通常设为 `1`。

【开卷考试常考】：

- BERT 的输入层由哪三种不同的 Embedding 组合而成？
- 在句子对任务中，`token type ids` 起到了什么关键作用？
- 这三种 Embedding 在送入网络前是**相加（$+$）还是拼接（concat）？（记住：是相加）**

7.2 BERT 结构

BERT 内部由多个完全相同的 Encoder Layer 纵向堆叠而成。每个 Encoder Layer 的内部组件与标准 Transformer 编码器完全一致：

$$\text{自注意力层 (Self-Attention)} \rightarrow \text{残差连接与层归一化 (Add \& Norm)} \rightarrow \\ \text{前馈神经网络 (FFN)} \rightarrow \text{残差连接与层归一化 (Add \& Norm)}$$

BERT 官方提供了两种最典型的基准规模：

- **BERT BASE**：12 层 blocks，隐藏层维度 768，多头注意力 12 个 head。总参数量约 110M。
- **BERT LARGE**：24 层 blocks，隐藏层维度 1024，多头注意力 16 个 head。总参数量约 340M。

【结构对比表】：

模型规模	Blocks (层数)	Hidden Size (词向量维度)	Attention Heads (头数)	参数量
Transformer 原始单体	6	512	8	-
BERT BASE	12	768	12	110M
BERT LARGE	24	1024	16	340M

> *注：* hidden size *代表 Token Embedding 的稠密向量长度；* attention head *代表多头注意力中独立特征子空间的数量。*

【开卷考试常考】：

- BERT BASE 和 BERT LARGE 在网络深度和维度上的核心参数区别。
- 明确 hidden size 和 attention head 的物理含义。
- BERT 的内部基础层是由什么结构组件构成的?

7.3 BERT 输出

BERT 的输出是一串与输入等长的编码特征向量序列。

- `[CLS]` **位置对应的输出 (Pooler Output)**：由于双向注意力的存在，开头的 `[CLS]` 充分融合了整段文本的全局语义表示。`[CLS]` 向量经过一个线性层 + tanh 激活后得到 **Pooler Output**，常用于**句子级分类或回归任务**。
- **各个 Token 位置对应的输出 (Sequence Output)**：每个实际单词位置输出的编码向量称为 **Sequence Output**，常用于局部序列标注（如**词性标注、命名实体识别 NER**）。

【开卷考试常考】：

- 什么是 pooler output？什么是 sequence output？它们在应用上有什么不同?
- 为什么在做文本分类任务时，通常只抽取 `[CLS]` 标记对应的输出向量?

7.4 BERT 的不同下游任务

1.1.1.1 单句子分类

- **方法**：只提取 `[CLS]` 对应的输出向量，外接一个普通的分类线性层（Classifier）。
- **场景**：情感分类（正评/负评）、垃圾邮件分类、文本质量评分。

1.1.1.2 句子对分类

- **方法**：输入格式为 `[CLS] 句子A [SEP] 句子B [SEP]`，通过 `token type ids` 区分两句。同样提取 `[CLS]` 的输出进行整体分类。
- **场景**：文本相似度判断、自然语言推理（NLI）、问句匹配。

1.1.1.3 抽取式问答 (Q&A)

- **方法**：输入格式为 `[CLS] 问题 [SEP] 参考段落 [SEP]`。BERT 的输出不需要生成新词，而是通过分类头去预测答案在参考段落中的**起始 token 位置**和**结束 token 位置**。
- **核心理解**：**BERT 属于定位答案区间，而非生成答案。**

1.1.1.4 单句子序列标注 (Tagging)

- **方法**：不使用 `[CLS]`，而是利用每个有实际意义的 token 位置对应的 **Sequence Output** 分别接分类层输出标签。
- **场景**：词性标注（POS Tagging）、命名实体识别（NER）。

【开卷考试常考】：

- 在问答（Q&A）任务中，BERT 的输出和工作原理是什么？模型是生成答案还是定位答案？
- 文本分类任务和序列标注（Tagging）任务在利用 BERT 输出向量时有什么本质区别？

7.5 Hugging Face 中不同任务型 BERT 模型的使用

`BertForSequenceClassification`

- **网络结构**：基础 BERT 骨干 ＋ `[CLS]` token 专属的全连接分类头。
- **适用场景**：文本分类、意图识别、文本质量评分（输出序列级类别概率）。

`BertForTokenClassification`

- **网络结构**：基础 BERT 骨干 ＋ **每个 token 位置**独立的线性分类头。
- **适用场景**：命名实体识别（NER）、词性标注（POS）、关键词抽取。

`BertForQuestionAnswering`

- **网络结构**：基础 BERT 骨干 ＋ 两个独立的线性层（分别用来预测答案在文本中的 Start 和 End 概率分布）。
- **适用场景**：抽取式问答、文档内部答案区间检索。

【开卷考试常考】：

- 面对文本分类、NER、抽取式问答时，在代码中分别应该调用 Hugging Face 的哪个封装模型？

7.6 BERT 预训练与核心任务

BERT 的预训练是一个自监督学习过程，无需人工标注，直接在海量无监督文本（如维基百科）上通过两个精心设计的自监督任务进行隐式知识学习。多数情况下，我们在实际工程中直接下载官方预训练好的权重，再在下游特定数据上进行**微调（Fine-tuning）**。

1.1.1.1 掩码语言模型 Masked Language Modelling (MLM)

- **核心目标**：迫使模型根据未遮盖的上下文去预测被隐去的词，以此学习到**双向上下文深度理解**的能力。
- **基本做法**：随机选择输入序列中 15% 的 token 参与整改：
  - 80% 的概率将其替换为特殊的 `[MASK]` 标记。
  - 10% 的概率将其替换为一个完全随机的错词。
  - 10% 的概率保持原词完全不变。
- **非全用 `[MASK]` 的深层原因**：为了缓解“预训练”与“微调”阶段之间的不一致问题（因为微调时的下游数据中绝对不会出现 `[MASK]` 标记），同时防止模型过度依赖该标记，被迫真正去理解上下文。

1.1.1.2 下句预测 Next Sentence Prediction (NSP)

- **核心目标**：训练模型理解句子与句子之间的宏观逻辑与篇章连贯关系。
- **具体做法**：输入句子对（A和B），让模型进行二分类判断：句子 B 是否是句子 A 在原文中的下一句（50% 是下一句，50% 是随机采样的非下一句）。这能显著增强模型处理问答、自然语言推理等句子关系任务的能力。

【开卷考试常考】：

- BERT 的两个核心预训练任务分别叫什么名称？各自解决什么问题？
- 简述 MLM 任务中 15% 以及 $80/10/10$ 策略的具体含义与设计初衷。
- “预训练（Pre-training）”与“微调（Fine-tuning）”之间有什么本质区别？

7.7 基础 BERT 模型预训练变体

- `BertForMaskedLM`：基础 BERT + MLM 预测头。常用于初级预训练、领域知识增量预训练（二次预训练）或进行领域适配。
- `BertForNextSentencePrediction`：基础 BERT + NSP 二分类头。主要用于判断两句文本是否在时序上下文上连续。

7.8 三大主流大模型底层结构的深度对比

[Image comparing Encoder-Only, Decoder-Only, and Encoder-Decoder structural sub-components side by side]

结构变体	注意力流动方式	核心训练目标	代表模型	典型应用场景
Encoder-Only	**双向**（无掩码拦截）	掩码语言模型（MLM）	**BERT**, RoBERTa	文本分类、抽取式问答、语义特征提取、理解任务
Decoder-only	**单向**（因果掩码阻断未来）	自回归语言模型（CLM，预判下个词）	**GPT** 系列, **LLaMA**, Qwen	开放式文本生成、人机对话、故事续写
Encoder-Decoder	编码双向 $+$ 解码单向 $+$ 跨层交叉注意力	序列到序列映射（Seq2Seq）	**T5**, BART	机器翻译、长文本摘要、语音转文字识别

> **关键结论**：Encoder-Decoder 极其适合输入、输出序列不严格对称且长度高度不固定的强映射转译任务。

【开卷考试常考】：

- 简述三类变体在注意力计算与训练目标上的本质区别。
- 给定具体的工业任务，能够准确判断出应当选择哪一种模型结构。
- 明确写出 BERT、GPT、T5 分别归属于上述哪一种特定结构。

8 Transformer 应用实现

8.1 基于 BERT 模型的文本分类微调

在微调阶段，为了节约算力和防止过拟合，我们经常会**冻结（Freeze）BERT 主体网络参数**，只对全新初始化的分类头进行参数更新。

- **工作流**：加载分词器 → 文本截断与填充（Padding & Truncation） → 实例化 `BertForSequenceClassification`（指定标签数 `num_labels`） → **设定 `requires_grad = False` 冻结 BERT 主体** → 前向传播 → 计算交叉熵 Loss → 反向传播仅更新分类头层参数。
- **冻结参数的核心优势**：计算资源消耗极少、训练速度极快、小样本下不易发生过拟合、能完美保留预训练阶段学到的通用语言大格局知识。

【开卷考试常考】：

- 简述利用 BERT 进行文本分类的完整工程流程。
- 什么是“只训练分类头”？在 PyTorch 代码中如何通过冻结参数来实现？
- 冻结 BERT 骨干网络参数有哪些显而易见的工程收益？

8.2 基于预训练 GPT 模型的文本生成流程

在 Hugging Face 中进行因果文本生成时，通用的标准流程如下：

- **工具链调用**：主要基于 `AutoTokenizer` 和自回归专用模型 `AutoModelForCausalLM`。
- **工作流**：加载分词器 → 加载 Causal LM 权重模型 → 传入 Prompt（提示词） → 分词编码成 ID 序列 → **显式调用 `model.generate()` 函数** → 对生成的 token 序列进行 Decode 还原为人类自然文本。

【`model.generate()` 关键控制参数】：

- `max_new_tokens`：严格限制本次自回归新生成的最大 token 数量。
- `do_sample=True`：显式开启概率采样模式（若设为 `False` 则默认退化为确定性的贪心搜索）。
- `temperature`：温度调节系数，用以微调生成文本的随机性、创造力与发散程度。

【开卷考试常考】：

- 写出基于 GPT 进行文本自回归生成的标准伪代码/开发流程。
- 区分控制参数 `do_sample=True` 与 `temperature` 在文本生成时的实际物理效用。
- 为什么在做文本生成任务时，必须加载 `AutoModelForCausalLM` 而不是普通的分类模型？

9 初识大语言模型 (LLM)

大语言模型（Large Language Model）是基于 **Transformer 架构** 构建的超大规模参数人工智能模型。它通过在万亿级（Trillion）海量无标注文本数据上进行自监督训练，隐式地将自然语言的统计规律、人类世界常识、严密的逻辑推理以及高级的文本理解生成能力编码进了数以亿计的参数权重中。

【LLM 与传统小型语言模型的降维级区别】：

- **参数规模**：大模型参数量通常达数十亿（Billion级）甚至千亿，传统模型仅百万或千万级。
- **训练数据**：大模型吞噬整个互联网公开数据的万亿级 Tokens，数据量达到 TB 乃至 PB 级别。

【开卷考试常考】：

- 到底什么是大语言模型（LLM）？
- 大语言模型相比传统的普通机器学习或小型深度学习模型，核心区别体现在哪些维度？

9.1 大语言模型的三大蜕变特征

1.1.1.1 涌现能力 (Emergent Abilities)

- **核心定义**：当模型的参数量、训练数据规模与总算力（FLOPs）跨越了某一个**特定的临界阈值（Threshold）后，大模型会突然展现出一些在小型模型上完全不具备的、非线性的复杂高阶能力**。
- **科学特性**：这种能力是无法通过在小模型上的表现进行线性外推或提前预测的。
- **典型具象表现**：多步复杂数学推理、复杂的严密逻辑证明、**链式思维推理（CoT, Chain-of-Thought）**、端到端代码生成与 Debug、超长文本宏观创作、跨语种跨专业领域的迁移。

1.1.1.2 通用性与多任务处理 (Generalization)

- 告别了“一任务一模型”的传统作坊模式。**同一个大模型本体不需要针对特定任务进行微调改动结构**，仅仅通过变换输入端的提示词（Prompt）指令，就能在同一个模型中完美处理翻译、润色、摘要、代码编写、情感判定等数个截断不同的任务。

1.1.1.3 上下文学习 (In-Context Learning) 与少样本学习 (Few-shot)

- 大模型无需对参数权重进行更新梯度回传，仅仅通过在 Prompt 对话中为其塞入两到三个完成任务的**正确示例（Few-shot Demonstration）**，大模型就能在上下文流动中瞬间洞察任务的潜在模式与要求，并依葫芦画瓢地给出正确的新输出。

1.1.1.4 知识的深度与广度

- 由于在大规模多语种、全行业的公开文献、教科书与维基百科上进行过深度洗礼，大模型内部自带了近乎人类全量世界知识的“数字图书馆”压缩本。

【开卷考试常考】：

- 科学地解释大模型的“涌现能力（Emergent Abilities）”是什么意思？它有哪些具体的表现？
- 为什么 Prompt（提示词）在调教大模型时成了核心手段？
- 什么是上下文学习（In-Context Learning）与 Few-shot Learning？它们和传统的微调训练有什么区别？

9.2 大语言模型生命周期的三大构建阶段

要构建一个能够安全、听话地服务人类的现代大语言模型，必须依次经历三大核心训练生命周期：

阶段一：大规模预训练 (Pre-training)

- **核心目标**：从海量互联网原始语料中汲取词汇统计规律、语法法则、通用语义表征，构建出大模型底层的**通用世界知识广度**。
- **学习范式**：完全的自监督学习（如 GPT 模型的自回归下一个词预测）。这是大模型一切能力的起点与基石。

阶段二：监督微调 (SFT, Supervised Fine-Tuning)

- **核心目标**：将一个只会续写的“预训练野兽”转化为能够顺畅进行人机对话、听懂人类指令并按格式完成特定任务的“听话助手”。
- **学习范式**：在由人工专家精心撰写的高质量指令对（$\text{Prompt} - \text{Response}$ 问答数据集）上进行有监督的参数调优，实现**任务指令对齐**。

阶段三：基于人类反馈的强化学习 (RLHF, Reinforcement Learning from Human Feedback)

- **核心目标**：解决大模型的安全性、价值观取向、幻觉问题。防止模型输出带有偏见、有害、违法或反人类的错误言论，使大模型彻底契合人类社会的道德偏好与真实意图。

- **学习范式**：收集人类对模型多个不同回答的偏好排序数据 → 训练一个模拟人类裁判的奖励模型（Reward Model） → 采用强化学习（如 PPO 算法）对 SFT 后的策略模型进行定向奖惩迭代。

【开卷考试常考】：

- 简述大模型流水线中**预训练（Pre-training）**、监督微调（SFT）**与**人类反馈强化学习（RLHF）三个阶段各自要解决的核心痛点。
- 什么是 RLHF 机制？为什么仅仅做完有监督微调大模型依然是不够安全的？

9-2模型微调

1 模型微调技术概述

1.1 模型微调

- SFT 是 **Supervised Fine-Tuning**，即有监督微调。
- 含义：在预训练大模型的基础上，使用特定任务或领域的小规模数据进行二次训练，使模型参数适应新场景。
- 核心逻辑：
  - **预训练**：学习通用语言知识与世界常识。
  - **微调**：将通用能力转化为领域专属能力。
- 通俗类比：
  - 预训练 = 通识教育。
  - 微调 = 职业技能培训。

1.2 为什么需要微调

- 通用大模型在特定任务上可能表现不佳：
  - 输出风格不符合要求；
  - 专业术语错误；
  - 特定领域知识不足；
  - 任务准确率不够高。
- 微调的价值：
  - **定制化**：让模型适配特定领域，如医疗、法律、金融等。
  - **性能提升**：在目标任务上超过通用模型。
  - **成本效益**：不需要从头训练，只需少量数据和计算资源即可优化模型。

1.3 SFT微调技术分类

按训练目标分类

- **指令微调 Instruction Tuning**
  - 数据格式：
    - Input：请总结这篇文章
    - Output：这篇文章讲了……
  - 目的：让模型学会遵循各种指令，具备通用对话能力。
- **传统任务微调 Task-Specific FT**
  - 数据格式：
    - Input：这部电影很好看
    - Output：正面
  - 目的：让模型专门完成某个具体任务，如分类、翻译、抽取。
- **领域适应微调 Domain Adaptation**
  - 数据格式：
    - Input：医学文献片段
    - Output：续写的医学文献
  - 目的：让模型从通用领域迁移到特定专业领域，解决源领域与目标领域之间的分布差异。
- **思维链微调 CoT Tuning**
  - 数据格式：
    - Input：数学题
    - Output：思考步骤 + 答案
  - 目的：训练模型的推理能力。
  - PPT 示例重点：
    - 输出不仅有最终答案，还包含逐步推理过程；
    - 能识别干扰信息，如“买了 5 个橙子”与苹果数量无关；
    - 最后给出明确答案。

按参数更新策略分类

- **全参数微调**
  - 更新模型所有参数。
  - 优点：效果潜力最大。

    - 缺点：
        - 计算成本极高；
        - 显存占用大；
        - 容易过拟合。
- **参数高效微调 PEFT**
    - 冻结基座模型的大部分参数，仅更新少量参数。
    - 优点：
        - 显存占用低；
        - 训练速度快。
    - 缺点：
        - 效果可能略逊于全参数微调。

2 参数高效微调（PEFT）

- PEFT 的核心思想：
    - 冻结预训练模型的大部分参数；
    - 只调整少量新增的、任务相关的参数；
    - 用较低成本让模型适配新任务。
- 主流 PEFT 方法：
    - **LoRA**：低秩适应，通过低秩矩阵分解近似权重更新。
    - **Adapter Tuning**：在模型层间插入小型适配器模块。
    - **Prefix Tuning / Prompt Tuning**：优化输入的软提示 Soft Prompt。
- PEFT 的优势：
    - **显存占用极低**：通常只微调原模型参数的 0.1%–1%。
    - **训练速度快**：计算量大幅减少。
    - **多任务友好**：一个基础模型可以搭配多个 PEFT 适配器，灵活切换任务。
    - **缓解遗忘**：基础模型参数被冻结，原有知识得以保留。

9-3 RAG检索增强生成

1 RAG

为什么大模型会“一本正经地胡说”

大模型存在两类主要局限：

- **知识截止日期**：训练数据有时间范围，无法知道截止日期之后发生的事情。
- **幻觉 Hallucination**：大模型本质上是按概率续写，当问题超出训练知识范围时，可能生成听起来合理但实际错误的内容。

1.1 RAG概述

RAG 是 **Retrieval-Augmented Generation，检索增强生成**。

核心思想：把大语言模型的生成能力和外部信息检索系统结合起来。

基本过程：

1. 从企业文档、网络或数据库中检索相关资料；
2. 将检索结果加入 Prompt；
3. 让大模型根据真实资料生成答案。

相比通过微调注入新知识，RAG 的优势：

- **知识更新快**：只需更新知识库，无需重新训练模型。
- **减少幻觉**：让模型基于检索资料回答。
- **可以标注来源**：便于追溯答案依据。
- **保护私有数据**：企业数据可保存在本地数据库中。

1.2 RAG的工作流程

RAG 系统分为两个阶段：

- 离线索引 Indexing
- 在线查询 Query

离线阶段：构建索引

1. **文档加载**
    - 加载 PDF、Word、Markdown 等文档；
    - 转换为纯文本。
2. **文本分块 Chunking**
    - 将长文档切成较小的块。
    - 块太大：检索不精准。
    - 块太小：上下文容易断裂。
    - 常见大小为 256～1024 Token，相邻块可设置重叠。

3. **文档向量化**
    - 使用嵌入模型将文档块转换为向量；
    - 存入向量数据库；
    - 建立索引。

例如：`bge-small-zh` 是轻量中文嵌入模型，最大上下文长度为 512 Token，因此文档块需控制在 512 Token 以内。

在线阶段：检索并生成

4. **问题向量化**
    - 使用和文档相同的嵌入模型，将用户问题转换为向量。
5. **相似度检索**
    - 在向量数据库中查找和问题向量最相似的 Top-K 个文档块。
    - K 通常取 3～10。
6. **上下文注入生成**
    - 将检索到的文档块与用户问题拼接成 Prompt；
    - 送入大模型生成回答。

1.3 向量数据库：RAG的记忆系统

向量数据库是专门存储和检索高维向量的数据库。

核心能力：通过近似最近邻搜索 ANN，快速找到与查询向量最相似的 Top-K 个向量。

向量数据库的工作原理

1. 文档块转换为高维向量；
2. 存入向量数据库并建立 HNSW 索引；
3. 将问题转换为向量；
4. 计算问题向量与文档向量的相似度；
5. 返回 Top-K 最相似结果。

PPT 中主要使用**余弦相似度**判断向量相似程度。

HNSW 特点：

- 采用分层图结构；
- 高层用于快速定位；
- 低层用于精细搜索；
- 层内采用贪心搜索寻找最近邻。

缺点：

- 内存占用较大；
- 需要保存大量图结构连接关系。

常见向量数据库：

向量数据库	特点	适用场景
Chroma	轻量、纯 Python、易使用	本地开发、快速原型
FAISS	Meta 开源、速度快、不持久化	离线检索、研究
Qdrant	Rust 编写、性能强、支持过滤	生产环境
Weaviate	支持稀疏和稠密混合检索	混合检索场景

1.4 分块策略

分块方式会直接影响 RAG 的检索效果。

固定大小分块

- 按固定 Token 数切分，例如每块 512 Token，重叠 50 Token。
- 优点：简单、块大小可控。
- 缺点：可能截断句子或段落，破坏语义完整性。

语义分块

- 按段落、章节标题等语义边界切分。
- 优点：语义完整，上下文连贯。
- 缺点：块大小不一致，可能出现过长或过短的块。

递归字符分块

LangChain 默认推荐方法。

切分优先级：

```
代码块
1  段落 → 换行 → 句子 → 词语
```

逻辑：

1. 先按段落切分；
2. 如果块太大，再按行切；
3. 如果仍然太大，再按句子切。

优点：

- 尽量保留语义完整性；
- 同时控制块大小。

常用工具：

```
RecursiveCharacterTextSplitter
```

常见参数：

- `chunk_size`：每块最大长度。
- `chunk_overlap`：相邻块重叠长度。
- `separators`：分隔符优先顺序。

1.5 检索质量优化

基础 RAG 只依赖向量相似度，可能出现语义相似但关键词不准确的问题。

例如搜索：

```
iPhone 15 Pro Max
```

向量检索可能返回“手机”“科技产品”相关文档，却漏掉具体型号。

混合检索

混合检索由两部分组成：

- **向量检索**：擅长语义相似和近义词匹配。
- **BM25 关键词检索**：擅长精确关键词匹配。

两类检索结果可以通过倒数排名融合 RRF 合并。

BM25

BM25 是一种评价查询词与文档相关性的概率检索算法。

核心思想：

- 词在当前文档中出现越多，相关性越高；
- 在全部文档中越少见的词，权重越高。

IDF 表示逆文档频率：

- 常见词权重低；
- 稀有词权重高。

重排序 Reranking

向量检索召回率高，但精度可能不足。

重排序过程：

1. 先用向量检索召回较多候选文档；
2. 使用更精确的模型重新打分；
3. 按相关度重新排序；
4. 取最终 Top-K。

常用模型：Cross-Encoder。

输入格式：

```
代码块
1   [CLS] 问题 [SEP] 文档 [SEP]
```

特点：

- 问题和文档一起编码；
- 相关性判断更准确；
- 计算速度较慢。

HyDE

HyDE 是虚拟文档检索。

操作：

1. 让 LLM 根据用户问题生成一个“理想答案”；
2. 用理想答案去知识库检索。

原理：

- “答案与答案”的向量通常比“问题与答案”更接近。

多查询扩展

让 LLM 将一个问题改写成多个语义相同、表达不同的问题。

分别检索后合并结果，可以提高召回率。

2 从零构建一个RAG问答系统

LangChain 是用于构建 LLM 应用的框架。

它可以将大模型与以下内容结合：

- 外部数据；
- 工具；
- API；
- 记忆系统；
- 文档检索系统。

RAG 问答系统基本流程：

1. 加载文档；
2. 文本分块；
3. 文档向量化；
4. 存入向量数据库；
5. 用户提出问题；
6. 检索相关文档；
7. 拼接 Prompt；
8. 调用大模型生成答案。

文档加载与分块

使用：

```
代码块
1   RecursiveCharacterTextSplitter
```

典型配置：

```
代码块
1   chunk_size=512
```

```
2   chunk_overlap=64
```

文档向量化与存储

使用嵌入模型：

```
代码块
1   BAAI/bge-small-zh-v1.5
```

使用向量数据库：

```
代码块
1   Chroma
```

流程：

```
代码块
1   文档块 → 嵌入模型 → 向量 → Chroma
```

构建RAG查询函数

查询流程：

1. 将问题交给 retriever；
2. 检索 Top-K 文档；
3. 拼接文档内容形成 context；
4. 将 context 和 question 填入 Prompt；
5. 调用大模型生成答案。

Prompt 中应明确规定：

- 只能根据参考资料回答；
- 如果资料中没有答案，不允许编造。

使用混合检索和重排序改进

改进过程：

1. 向量检索返回候选文档；

2. BM25 返回候选文档；
3. 使用 EnsembleRetriever 融合两类结果；
4. 使用 Cross-Encoder 重排序；
5. 选取最终 Top-K 文档。

也可以使用 LangChain 的 `RetrievalQA`，自动完成：

```
检索 → 组装 Prompt → 调用模型 → 解析输出
```

3 GraphRAG

基础 RAG 擅长回答：

- 某个文档里是否提到某个信息；
- 从单个文档块中直接找到答案。

但对于以下任务能力有限：

- 多跳推理；
- 跨文档关联；
- 全局总结；
- 多实体关系分析。

例如：

```
过去5年的财报中，哪个季度利润率最高，原因是什么？
```

需要同时检索多个文档，并分析信息之间的关系。

GraphRAG概述

GraphRAG 是 **Graph-based Retrieval-Augmented Generation**，即基于图的检索增强生成。

核心思想：

- 将知识表示为知识图谱；
- 结合图结构和大语言模型；
- 不仅检索独立信息，还理解实体之间的复杂关系。

普通 RAG 主要检索文档块，GraphRAG 主要检索相互关联的节点和关系。

3.1 GraphRAG工作流程

1. **查询处理**
   - 分析用户问题；
   - 转换成适合图数据库的查询形式。
2. **图遍历**
   - 沿知识图谱中的关系查找相关节点。
3. **子图检索**
   - 不只返回单独节点；
   - 返回包含节点和关系的相关子图。
4. **信息整合**
   - 整合多个相关节点和关系；
   - 构造完整、连贯的上下文。
5. **响应生成**
   - 大模型根据用户问题和图谱信息生成答案。

目前常利用 LLM 将自然语言问题转换为 Cypher 查询语句。

基本过程：

```
代码块
1    自然语言问题 → Cypher语句 → Neo4j查询 → 图数据 → LLM生成回答
```

3.2 GraphRAG实现

可以使用 Neo4j 作为图数据库。

基本实现步骤：

1. 连接 Neo4j；
2. 获取图谱 Schema；
3. 将 Schema 和用户问题加入 Prompt；
4. 让 LLM 生成 Cypher 查询语句；
5. 执行 Cypher；
6. 获取图谱查询结果；
7. 将结果交给 LLM 总结回答。

也可以使用 LangChain 封装的：

```
GraphCypherQAChain
```

它可以自动完成：

```
用户问题 → 生成Cypher → 查询图数据库 → 总结答案
```

主要参数：

- `verbose=True`：打印模型生成的 Cypher。
- `validate_cypher=True`：校验并修正 Cypher 语法。
- `allow_dangerous_requests=True`：允许执行相关图查询操作。

9-4 模型量化

1 量化技术概述

什么是量化

量化是通过降低模型参数精度，将浮点数转换为整数或低精度定点数，从而实现模型压缩和推理优化。

1.1 为什么需要量化

- **降低显存占用**：以 LLaMA-7B 为例，FP16 约需 14GB，INT4 约需 3.5GB，体积缩减约 75%。
- **提升推理速度**：低比特运算在 NPU、TPU 等专用芯片上效率更高。
- **降低部署成本**：使大模型可以部署到消费级 GPU、手机和边缘设备。

1.2 量化带来的问题

- 量化会造成性能和精度下降，INT8 影响通常较小，INT4 或更低精度损失更明显。
- 深层网络中量化误差可能逐层累积。
- 误差主要来自：截断、舍入、数值范围估计不准确。

2 模型精度类型

浮点数精度

- **FP32**：32 位，包括 1 位符号位、8 位指数位、23 位尾数位。指数位决定动态范围，尾数位决定精度。
- **FP16**：16 位，包括 1 位符号位、5 位指数位、10 位尾数位。优点是显存约为 FP32 的一半、速度快；缺点是动态范围较窄，容易梯度上溢或下溢。
- **BF16**：16 位，包括 1 位符号位、8 位指数位、7 位尾数位。动态范围接近 FP32，训练稳定性较好，但尾数较少、精度低于 FP16，且需要较新 GPU 支持。

记忆：**FP16 保精度、舍范围；BF16 保范围、舍精度。**

整数（量化）精度

- **INT8**：使用 8 位表示整数，有符号范围为：
  $-128 \sim 127$
- **NF4**：Normalized Float 4-bit，专为神经网络权重设计的 4 位浮点量化格式，相比 INT4 能在相同位宽下保留更多权重信息，常用于 QLoRA 和低显存部署。

3 量化对象

权重 Weights

权重是最常见的量化对象，包括 Transformer 中的 Linear 层权重、Q/K/V 投影矩阵、FFN 层权重。权重分布较稳定，可提前统计数值范围，通常采用静态量化，即训练完成后一次性从高精度量化到低比特。

激活值 Activation

激活值是前向传播过程中各层的输出，如 MLP 输出、Attention 输出。其分布会随输入动态变化，并可能出现异常值，因此常采用动态量化，即推理时根据每个输入 batch 实时计算缩放因子。多数 4-bit 方案只量化权重，激活值仍保持 FP16 或 BF16，以避免精度明显下降。

4 量化方法分类

4.1 对称与非对称量化

对称量化

特点：数值范围以 0 为中心，量化后的整数范围关于 0 对称，不需要零点偏移，只需要缩放因子 Scale。适合分布对称、均值接近 0 的数据，常用于权重量化。

缩放因子：

$$Scale = \frac{|R_{\max}|}{|Q_{\max}|}$$

浮点数转定点数：

$$Q = Round\left(\frac{R}{Scale}\right)$$

定点数转浮点数：

$$R = Q \times Scale$$

其中，(R) 为原始浮点值，(Q) 为量化整数值，(Scale) 为缩放因子。

非对称量化

特点：数值范围不要求以 0 为中心，需要引入零点偏移 Zero Point，能够更灵活地拟合任意数据分布，常用于激活值量化。

缩放因子：

$$Scale = \frac{R_{\max} - R_{\min}}{Q_{\max} - Q_{\min}}$$

零点：

$$Z = Q_{\max} - Round\left(\frac{R_{\max}}{Scale}\right)$$

浮点数转定点数：

$$Q = Round\left(\frac{R}{Scale} + Z\right)$$

定点数转浮点数：

$$R = (Q - Z) \times Scale$$

区别：**对称量化只有 Scale；非对称量化有 Scale 和 Zero Point。**

校准

校准的目标是确定合适的缩放因子 Scale、零点 Z 和裁剪范围，以减少截断和舍入误差。通常使用少量校准数据进行前向传播，统计各层激活值分布，再确定合理范围。

常见校准方法：

- **Min/Max**：使用观察到的最小值和最大值。
- **Moving Average Min/Max**：通过移动平均获得更稳定的范围。
- **Histogram / Percentile**：使用 99.9% 或 99.99% 分位数过滤异常值。
- **KL 散度校准**：选择量化前后分布差异最小的裁剪范围。

4.2 训练后量化和量化感知训练

训练后量化 PTQ

PTQ 是 Post-Training Quantization，在已经训练好的 FP32 或 FP16 模型上直接量化，不更新模型参数，只使用校准数据确定 Scale 和 Zero Point。特点是高效、无需重新训练，是大模型部署的主流方

法，常见算法有 GPTQ 和 AWQ。

量化感知训练 QAT

QAT 是 Quantization-Aware Training，在训练过程中插入伪量化算子，模拟量化和反量化误差，再通过反向传播更新权重以补偿误差。特点是精度较高，但需要训练数据，成本高、周期长。

一般顺序：**优先尝试 PTQ，只有 PTQ 精度不达标时再考虑 QAT。**

PTQ：仅权重量化

只量化模型权重，激活值仍保持浮点精度。推理时先将权重反量化，再执行浮点计算。优点是校准简单、精度损失小、模型体积减小；缺点是推理速度不一定提高。

PTQ：全整数量化

将权重和激活值都量化为整数，推理过程中使用整数运算。优点是速度快；缺点是依赖支持 INT8 的硬件，并需要校准数据确定激活值范围。

GPTQ

GPTQ 是训练后权重量化算法，核心思想是逐层量化，每层量化后立即分析和补偿误差，减少全局误差累积。特点：常用于 INT4、无需微调、只需少量校准数据、精度接近 QAT。常用工具有 AutoGPTQ、Axolotl，推理引擎有 vLLM、llama.cpp、TensorRT-LLM 等。

AWQ

AWQ 是 Activation-aware Weight Quantization，核心思想是根据激活值判断权重的重要性，对重要权重区域保留更高精度，从而降低 INT4 量化误差。特点：精度通常略优于 GPTQ，属于 INT4 PTQ 方法，已集成到 Hugging Face 等框架。

QAT 工作过程

1. 在预训练模型中插入伪量化算子。
2. 前向传播时执行量化再反量化。
3. 模拟实际量化误差。
4. 反向传播更新模型权重和量化参数。
5. 得到最终量化模型。

5 模型量化实践

基于 AutoAWQ + Transformers 实现

AutoAWQ 是 AWQ 算法的工程化实现。流程：加载原始模型和 tokenizer → 准备校准样本 → 设置量化位数和 group size → 执行量化 → 保存量化模型。

关键参数：

```
quant_config={
    "bits": 4,
    "group_size": 128
}
```

加载量化模型时可设置：

```
fuse_layers=True
```

作用：启用算子融合，加速推理。

基于 AutoGPTQ + Transformers 实现

GPTQ 的校准数据需要转换为：

```
{
    "input_ids": ...,
    "attention_mask": ...
}
```

常见量化参数：

- `bits=4`：4-bit 量化。
- `group_size=128`：分组大小。
- `damp_percent=0.01`：阻尼系数。
- `desc_act=False`：速度快、兼容性好。
- `sym=True`：使用对称量化。
- `true_sequential=True`：逐层量化。

执行量化并保存：

```
model.save_quantized(
    quant_path,
    use_safetensors=True
```

```
4   )
```

基于 bitsandbytes 实现

bitsandbytes 属于运行时动态量化，加载 FP16 模型时实时量化为 4-bit 或 8-bit，无需提前生成量化模型，既支持推理，也支持 QLoRA 训练。

常用配置：

代码块

```
1   bnb_config = BitsAndBytesConfig(
2       load_in_4bit=True,
3       bnb_4bit_quant_type="nf4",
4       bnb_4bit_compute_dtype=torch.bfloat16
5   )
```

含义：

- `load_in_4bit=True`：使用 4-bit 加载。
- `nf4`：使用 NF4 格式。
- `bfloat16`：前向和反向计算使用 BF16。

基于 PyTorch Quantization 实现

PyTorch 提供通用 PTQ 和 QAT 工具。

动态量化示例：

代码块

```
1   model_quantized = quantize_dynamic(
2       model=model,
3       qconfig_spec={nn.Linear},
4       dtype=torch.qint8
5   )
```

特点：可以指定量化层，例如 Linear，其他层保持浮点精度；主要支持 INT8，更适合 BERT、ResNet 等传统 NLP 和 CV 模型，不直接面向大语言模型的 4-bit 量化。

课堂题目及常考计算

交叉熵损失计算 3.24

假设真实标签是索引 0（即yi的 One-hot 向量为 [1,0,0]），模型预测概率分布 $\hat{y} = [0.7, 0.2, 0.1]$ 。计算交叉熵（Cross Entropy）损失： $H(y, \hat{y}) = -\sum_i y_i \log(\hat{y}_i)$ 。

【公式与已知量】

交叉熵损失函数（Cross-Entropy Loss）用于衡量模型预测概率分布与真实标签分布之间的差异。其核心计算公式为： $H(y, \hat{y}) = -\sum_i y_i \log(\hat{y}_i)$

【已知条件】

真实标签（One-hot 向量）： $y = [1, 0, 0]$ ，模型预测概率分布： $\hat{y} = [0.7, 0.2, 0.1]$

【代入计算】 由于真实标签 y 是一个 One-hot 向量，只有第一个分量 $y_0 = 1$ ，其余分量 $y_1 = y_2 = 0$ 。 因此，求和式 $\sum_i$ 中只有 $i = 0$ 这一项能保留下来，其余项由于乘以 0 均被消除：

$$
\begin{aligned}
H(y, \hat{y}) &= -(1 \cdot \log(0.7) + 0 \cdot \log(0.2) + 0 \cdot \log(0.1)) \\
&= -\log(0.7) \\
&= -\ln(0.7) \quad \text{（若题目中 } \log \text{ 默认以自然常数 } e \text{ 为底）}
\end{aligned}
$$

【最终结果】 $-\ln(0.7)$

2-gram 4.3

假设我们有一个基于 2-gram 的语言模型，用于计算句子的概率。给定下列训练语料库：`<BOS> 我\爱\自然语言\处理<BOS> 自然语言\处理\很有趣<BOS> 处理\文本\数据\很重要<BOS> 数据\很重要`

任务：使用 2-gram 语言模型计算以下句子的概率：`<BOS> 自然语言\处理\很重要`

假设：对所有未出现过的 2-gram，将其统计频次统一设置为 0.01

这里为您去掉了所有小标题，并优化了段落间的逻辑衔接。飞书文档对文本中的数学公式（`$` 和 `` ）有完美的渲染支持，您可以直接复制以下内容粘贴到飞书中：

我们需要计算三个目标条件概率： $P(\text{自然语言} \mid \langle BOS \rangle)$ 、 $P(\text{处理} \mid \text{自然语言})$ 以及 $P(\text{很重要} \mid \text{处理})$ 。

依据 2-gram 句子概率的核心公式：

$$P(w_1, w_2, \ldots, w_n) = \prod_{i=1}^{n} P(w_i \mid w_{i-1})$$

可以将目标句子的概率展开为：

$$P(\langle BOS \rangle, \text{自然语言}, \text{处理}, \text{很重要}) = P(\text{自然语言} \mid \langle BOS \rangle) \times P(\text{处理} \mid \text{自然语言}) \times P(\text{很重要} \mid \text{处理})$$

根据给定的训练语料分词结果：

1. $\langle BOS \rangle$ 我 爱 自然语言 处理
2. $\langle BOS \rangle$ 自然语言 处理 很有趣
3. $\langle BOS \rangle$ 处理 文本 数据 很重要
4. $\langle BOS \rangle$ 数据 很重要

我们开始逐步统计频次并计算各项条件概率。

首先，统计开头标识符的频次，可知 $C(\langle BOS \rangle) = 4$，而接“自然语言”的情况出现了 1 次，即 $C(\langle BOS \rangle, \text{自然语言}) = 1$。

因此，第一个条件概率为：

$$P(\text{自然语言} \mid \langle BOS \rangle) = \frac{C(\langle BOS \rangle, \text{自然语言})}{C(\langle BOS \rangle)} = \frac{1}{4}$$

接下来，“自然语言”在整个语料中总共出现了 2 次，且这两次后面紧跟的词全部都是“处理”，即 $C(\text{自然语言}) = 2$ 且 $C(\text{自然语言}, \text{处理}) = 2$。

因此，第二个条件概率为：

$$P(\text{处理} \mid \text{自然语言}) = \frac{C(\text{自然语言}, \text{处理})}{C(\text{自然语言})} = \frac{2}{2} = 1$$

随后，统计“处理”一词在语料中出现的总次数，得到 $C(\text{处理}) = 3$。然而，2-gram 组合 $(\text{处理}, \text{很重要})$ 在训练语料中并未实际出现。根据题目中对于未出现频次统一设置为 0.01 的平滑规则，设定 $C(\text{处理}, \text{很重要}) = 0.01$。

因此，第三个条件概率为：

$$P(\text{很重要} \mid \text{处理}) = \frac{C(\text{处理}, \text{很重要})}{C(\text{处理})} = \frac{0.01}{3}$$

将上述三个计算好的条件概率代入展开式中，进行最终的乘积计算：

$$\begin{aligned} P &= \frac{1}{4} \times \frac{2}{2} \times \frac{0.01}{3} \\ &= \frac{1}{4} \times 1 \times \frac{0.01}{3} \\ &= \frac{0.01}{12} \\ &\approx 0.0008333 \end{aligned}$$

最终计算结果为：

$$P \approx 0.0008333$$

（也可以表示为科学计数法：

$$P \approx 8.33 \times 10^{-4}$$

）

3-gram

问题一：已知一个简单语料库包含三句话：“我 喜欢 吃 苹果”、“我 喜欢 吃 香蕉”、“我 喜欢 喝 咖啡”。在 3-gram 模型下，已知前面两个词是“我 喜欢”，下一个词是“吃”的条件概率 $P(\text{吃}|\text{我}, \text{喜欢})$ 是多少？

答案：该条件概率为 $2/3$（约等于 66.7%）。

计算过程：根据 3-gram 模型的条件概率公式，在已知前两个词 W_1 和 W_2 的情况下，第三个词为 W_3 的最大似然估计公式为 $P(W_3|W_1W_2) = \text{Count}(W_1W_2W_3)/\text{Count}(W_1W_2)$。在此题中，分母 $\text{Count}(\text{我 喜欢})$ 需要数出“我 喜欢”这两个词连续出现在语料库中的总次数，遍历这三句话可以发现它们均以此开头，因此总次数为 3 次；分子 $\text{Count}(\text{我 喜欢 吃})$ 需要数出“我 喜欢 吃”这三个词连续出现的总次数，在语料库中，第一句话和第二句话满足此条件，第三句话为“我 喜欢 喝”，因此分子总次数为 2 次。将数据代入公式，最终概率计算为 $2/3 \approx 66.7\%$。

问题二：在 3-gram（三元语法）语言模型下，给定一个简单语料库仅包含两句话：“ 我 喜欢 吃 ”和“ 我 讨厌 吃 ”，请计算测试句子“我 喜欢 吃”的整体出现概率。

答案：该句子的整体出现概率为 0.5（50%）。

计算过程：首先对测试句子进行 3-gram 的预处理，在句首添加两个起始符并加上结束符，将其扩展为“ 我 喜欢 吃 ”。根据马尔可夫链式法则，整个句子的概率等于各个词在历史条件下的条件概率相乘，即：
$P(\text{我 喜欢 吃}) = P(\text{我}|\langle s\rangle, \langle s\rangle) \times P(\text{喜欢}|\langle s\rangle, \text{我}) \times P(\text{吃}|\text{我}, \text{喜欢}) \times P(\langle/s\rangle|\text{喜欢}, \text{吃})$。接着分别计算这四个概率：第一项 $P(\text{我}|\langle s\rangle, \langle s\rangle) = \text{Count}(\langle s\rangle\langle s\rangle\text{我})/\text{Count}(\langle s\rangle\langle s\rangle) = 2/2 = 1$；第二项 $P(\text{喜欢}|\langle s\rangle, \text{我}) = \text{Count}(\langle s\rangle\text{我 喜欢})/\text{Count}(\langle s\rangle\text{我}) = 1/2 = 0.5$；第三项 $P(\text{吃}|\text{我}, \text{喜欢}) = \text{Count}(\text{我 喜欢 吃})/\text{Count}(\text{我 喜欢}) = 1/1 = 1$；第四项 $P(\langle/s\rangle|\text{喜欢}, \text{吃}) = \text{Count}(\text{喜欢 吃 }\langle/s\rangle)/\text{Count}(\text{喜欢 吃}) = 1/1 = 1$。最后将四项概率相乘，得到句子的最终总概率为 $1 \times 0.5 \times 1 \times 1 = 0.5$。

马尔可夫模型（HMM）中前向算法 4.21

这张图片是一张幻灯片，内容是一个关于**隐马尔可夫模型（HMM）前向算法**的作业题目。以下是提取出的完整文字内容：

模型定义：

- **隐藏状态：** $H1, H2$
- **观测值：** $O1, O2$

状态转移概率：

- $a_{11} = P(H1|H1) = 0.7$

- $a_{12} = P(H2|H1) = 0.3$
- $a_{21} = P(H1|H2) = 0.4$
- $a_{22} = P(H2|H2) = 0.6$

发射概率：

- $b_1(O1) = P(O1|H1) = 0.5$
- $b_1(O2) = P(O2|H1) = 0.5$
- $b_2(O1) = P(O1|H2) = 0.9$
- $b_2(O2) = P(O2|H2) = 0.1$

假设在某个时刻 t ，我们已经计算出了前向概率（阿尔法值）： $\alpha_t(H1) = 0.2$ ， $\alpha_t(H2) = 0.1$ ，并且在下一个时刻 $t+1$ ，观测到了 $O1$ 。

关键公式回顾

前向概率的递推公式为：

αt+1(Hi)=bi(Ot+1)×Σj=1Nαt(Hj)·aji

其中：

- αt(Hj) 是时刻 t 处于状态 Hj 的前向概率；
- aji=P(Hi | Hj) 是从状态 Hj 转移到 Hi 的转移概率；
- bi(Ot+1)=P(Ot+1 | Hi) 是状态 Hi 发射观测 Ot+1 的概率。

已知条件整理

- 时刻 t 的前向概率：αt(H1)=0.2，αt(H2)=0.1
- 观测：Ot+1=O1
- 转移概率：a11=P(H1 | H1)=0.7，a21=P(H1 | H2)=0.4
- 发射概率：b1(O1)=P(O1 | H1)=0.5，b2(O1)=P(O1 | H2)=0.9

分步计算

1. 先计算 “转移部分” （即前一时刻状态转移到 H1 的加权和）：
2. Σj=12αt(Hj)·aj1=αt(H1)a11+αt(H2)a21
3. 代入数值：
4. =0.2×0.7+0.1×0.4=0.14+0.04=0.18
5. 再乘以状态 H1 发射观测 O1 的概率，得到最终结果：

6. αt+1(H1)=b1(O1)×0.18=0.5×0.18=0.09

✅ 最终答案

αt+1(H1)=0.09

马尔可夫模型（HMM）中后向算法

已知一个隐马尔可夫模型 $\mu = (A, B, \pi)$ 。

- **隐状态集合**为： $Q = \{H1, H2\}$
- **观测集合**为： $V = \{O1, O2\}$
- **初始状态分布**为： $\pi = (0.6, 0.4)$ ，即： $\pi_1 = 0.6, \quad \pi_2 = 0.4$
- **状态转移概率矩阵**为：
- $A = \begin{bmatrix} 0.7 & 0.3 \\ 0.4 & 0.6 \end{bmatrix}$
- *(其中行表示当前状态，列表示下一状态，即：* $a_{11} = 0.7,\ a_{12} = 0.3,\ a_{21} = 0.4,\ a_{22} = 0.6$ *)*
- **发射概率矩阵**为：
- $B = \begin{bmatrix} 0.5 & 0.5 \\ 0.9 & 0.1 \end{bmatrix}$
- *(列顺序为：* $(O1, O2)$ *，即：* $b_1(O1) = 0.5,\ b_1(O2) = 0.5,\ b_2(O1) = 0.9,\ b_2(O2) = 0.1$ *)*

现在给定观察序列： $O = \{O2, O1\}$ ，请使用**后向算法**计算： $P(O2, O1|\mu)$ 。

题解

1. 后向算法公式

后向变量定义为：

$$\beta_t(i) = P(O_{t+1}, O_{t+2}, \ldots, O_T | q_t = H_i, \mu)$$

递推公式为：

$$\beta_t(i) = \sum_{j=1}^{N} a_{ij} b_j(O_{t+1}) \beta_{t+1}(j)$$

终止公式为：

$$P(O|\mu) = \sum_{i=1}^{N} \pi_i b_i(O_1) \beta_1(i)$$

2. 初始化

观察序列为：$O=\{O2,O1\}$，所以序列长度 $T=2$。

后向算法从最后一个时刻开始初始化：

$\beta_2(H1)=1$

$\beta_2(H2)=1$

3. 计算 $\beta_1(H1)$

因为此时 $O_{t+1}=O_2=O1$，所以：

$\beta_1(H1)=a_{11}b_1(O1)\beta_2(H1)+a_{12}b_2(O1)\beta_2(H2)$

代入数值：

$\beta_1(H1)=0.7\times0.5\times1+0.3\times0.9\times1=0.35+0.27=0.62$

4. 计算 $\beta_1(H2)$

$\beta_1(H2)=a_{21}b_1(O1)\beta_2(H1)+a_{22}b_2(O1)\beta_2(H2)$

代入数值：

$\beta_1(H2)=0.4\times0.5\times1+0.6\times0.9\times1=0.2+0.54=0.74$

5. 计算 $P(O2,O1|\mu)$

注意，观察序列的第一个观测值是 $O_1=O2$，所以总概率为：

$P(O2,O1|\mu)=\pi_1b_1(O2)\beta_1(H1)+\pi_2b_2(O2)\beta_1(H2)$

代入数值：

$P(O2,O1|\mu)=0.6\times0.5\times0.62+0.4\times0.1\times0.74=0.186+0.0296=0.2156$

最终答案

$P(O2,O1|\mu)=0.2156$

💡 易错点提示

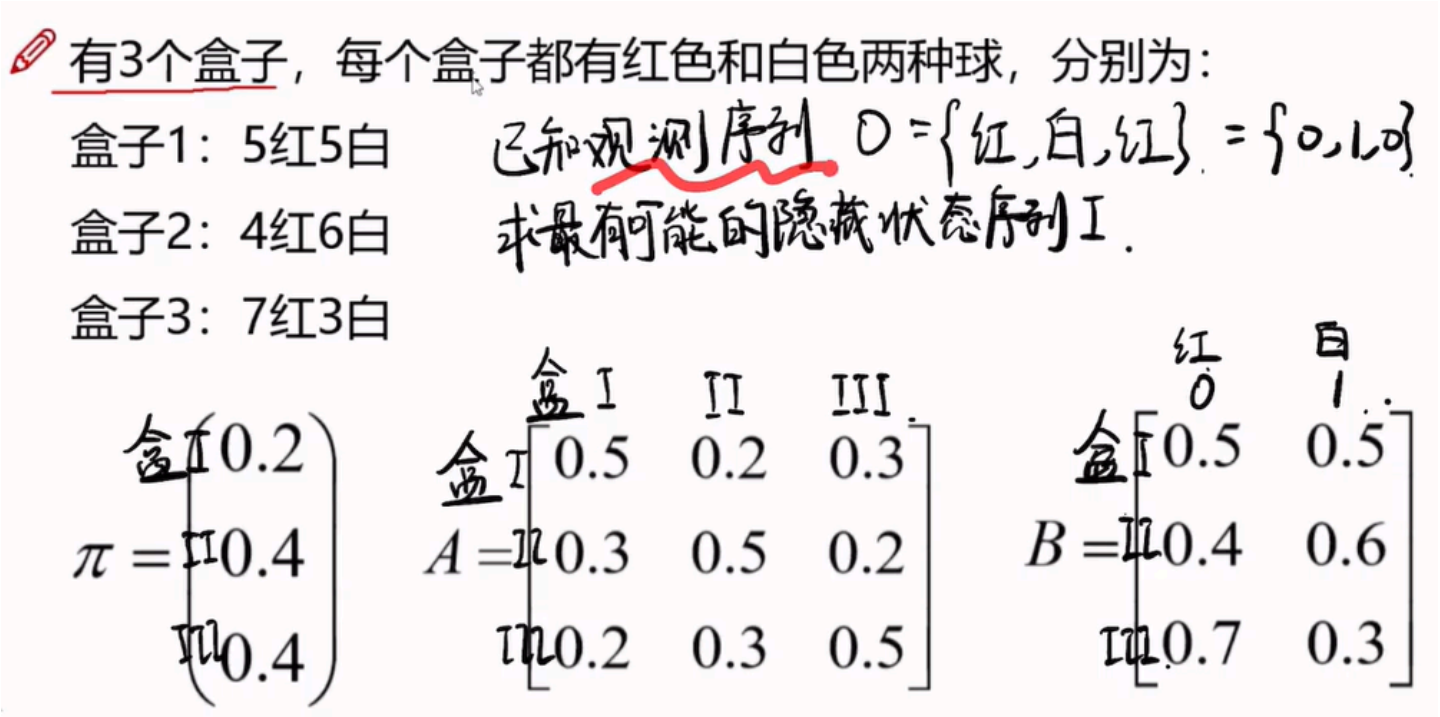
- 这里算的是观察序列：$O=\{O2,O1\}$，所以第一个观测值是 $O2$，最后求总概率时发射概率要用 $b_i(O2)$。
- 递推 β_1 时，算的是“后面剩下的观测”，所以用的是第二个观测值 $O_2=O1$。
- 后向算法最后求总概率时**不是直接相加** $\beta_1(H1)+\beta_1(H2)$，而是要乘初始概率和第一个观测的发射概率：

$$P(O|\mu)=\sum_i\pi_ib_i(O_1)\beta_1(i)$$

注意先后顺序，不要看错

维特比算法

https://www.bilibili.com/video/BV18LigeFEPj/?spm_id_from=333.337.search-card.all.click&vd_source=a9ab34f287effc918ec339427d6b07ed



ppt

CNN 计算题

给定输入矩阵：

$$X = \begin{bmatrix} 1 & 0 & 2 & 1 \\ 3 & 1 & 0 & 2 \\ 2 & 1 & 1 & 0 \\ 0 & 2 & 3 & 1 \end{bmatrix}$$

卷积核：

$$K = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$$

请完成以下计算：

- **a.** 不使用 padding，stride = 1，求卷积后的 feature map。
- **b.** 使用 padding = 1，此时输入变为 6×6，stride = 2，求卷积后的 feature map。
- **c.** 对 a 问得到的 feature map 做 2×3 average pooling，stride = 1，求输出 vector。
- **d.** 对 c 问得到的 vector 做 ReLU，求最终结果。

参考答案

a. stride = 1，padding = 0

输出尺寸：

$$H_{out} = W_{out} = \frac{4 + 2 \times 0 - 2}{1} + 1 = 3$$

所以输出是 3×3 矩阵。

- **计算左上角第一个窗口**：
- $\begin{bmatrix} 1 & 0 \\ 3 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix} = 1 \times 1 + 0 \times 0 + 3 \times (-1) + 1 \times 1 = -1$

依次滑动计算其余位置，得到：

$$Y_a = \begin{bmatrix} -1 & 1 & 4 \\ 2 & 0 & -1 \\ 1 & 0 & 2 \end{bmatrix}$$

b. padding = 1，stride = 2

padding = 1 后，输入矩阵 X_{pad} 变为：

$$X_{\text{pad}} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 2 & 1 & 0 \\ 0 & 3 & 1 & 0 & 2 & 0 \\ 0 & 2 & 1 & 1 & 0 & 0 \\ 0 & 0 & 2 & 3 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

输出尺寸：

$$H_{out} = W_{out} = \frac{6 - 2}{2} + 1 = 3$$

所以输出同样是 3×3 矩阵。

依次以步长 2 滑动计算，得到结果：

$$Y_b = \begin{bmatrix} 1 & 2 & 0 \\ 2 & 1 & -2 \\ 0 & -2 & 1 \end{bmatrix}$$

c. 对 a 问结果做 2×3 average pooling

a 问得到的 feature map 是：

$$Y_a = \begin{bmatrix} -1 & 1 & 4 \\ 2 & 0 & -1 \\ 1 & 0 & 2 \end{bmatrix}$$

池化窗口大小为 2×3，stride = 1。由于输入是 3×3，窗口只能在纵向上滑动 1 步，横向正好填满。

- **第一个窗口（前两行）**：

- $\begin{bmatrix} -1 & 1 & 4 \\ 2 & 0 & -1 \end{bmatrix}$
- 平均值 $= \dfrac{-1+1+4+2+0-1}{6} = \dfrac{5}{6}$
- **第二个窗口（后两行）：**
- $\begin{bmatrix} 2 & 0 & -1 \\ 1 & 0 & 2 \end{bmatrix}$
- 平均值 $= \dfrac{2+0-1+1+0+2}{6} = \dfrac{4}{6} = \dfrac{2}{3}$

所以输出 vector 为：

$$v = \left[\frac{5}{6}, \frac{2}{3}\right]$$

d. ReLU

ReLU 公式：

$$\mathrm{ReLU}(x) = \max(0, x)$$

因为 $\dfrac{5}{6} > 0,\ \dfrac{2}{3} > 0$，所以经过 ReLU 激活后数值保持不变：

$$\mathrm{ReLU}(v) = \left[\frac{5}{6}, \frac{2}{3}\right]$$

💡 核心考点复习

- **a 问**：标准二维卷积计算，注意滑动步长。
- **b 问**：四向先补 0 再按 $S = 2$ 错位滑动。
- **c 问**：平均池化需要把当前窗口内所有元素（本题共 6 个数）求和后除以元素个数。
- **d 问**：ReLU 激活函数的规则是**正数保留，负数归 0**。

点积注意力分数计算 5.26

为你将这道 **Transformer Attention 机制计算题** 转换为飞书文档能完美渲染的格式。公式、矩阵与计算步骤均已调整为飞书标准语法，可直接复制粘贴。

查询向量为 $q = [1, 0, 1]$，键向量集合 $K = \begin{bmatrix} 1 & 2 & 0 \\ 3 & 0 & 1 \end{bmatrix}$，值向量集合 $V = \begin{bmatrix} 2 & 2 & 2 \\ 3 & 3 & 3 \end{bmatrix}$。

已知近似值 $e^1 \approx 2.7$，$e^4 \approx 54.6$（忽略缩放因子 $\sqrt{d_k}$）。要求计算未缩放的点积注意力分数、经过 Softmax 归一化后的权重，以及最终加权输出的注意力向量。

步骤 1：计算未缩放的点积注意力分数

注意力分数（Scores）是查询向量 q 与每个键向量 k_i 的点积，即 $\text{scores} = qK^T$。

$$q = [1, 0, 1], \quad K^T = \begin{bmatrix} 1 & 3 \\ 2 & 0 \\ 0 & 1 \end{bmatrix}$$

分别计算 q 与键向量矩阵各列的点积：

- 与 $k_1 = [1, 2, 0]$ 的点积：
- $\text{score}_1 = 1 \times 1 + 0 \times 2 + 1 \times 0 = 1$
- 与 $k_2 = [3, 0, 1]$ 的点积：
- $\text{score}_2 = 1 \times 3 + 0 \times 0 + 1 \times 1 = 4$

得到未缩放的注意力分数向量：

$$\text{scores} = [1, 4]$$

步骤 2：应用 Softmax 函数计算注意力权重

根据 Softmax 公式计算归一化权重：

$$\text{softmax}(\text{scores}) = \left[\frac{e^1}{e^1 + e^4}, \frac{e^4}{e^1 + e^4} \right]$$

代入题目给出的近似值 $e^1 \approx 2.7$ 和 $e^4 \approx 54.6$，分母总和为 $2.7 + 54.6 = 57.3$：

- 第一个权值 α_1：
- $\alpha_1 \approx \frac{2.7}{57.3} \approx 0.0471$
- 第二个权值 α_2：
- $\alpha_2 \approx \frac{54.6}{57.3} \approx 0.9529$

得到归一化后的注意力权重向量：

$$\alpha \approx [0.0471, 0.9529]$$

步骤 3：计算加权后的注意力输出

将计算出的权重分别乘到对应的值向量 $v_1 = [2, 2, 2]$ 和 $v_2 = [3, 3, 3]$ 上进行加权求和：

$$\text{output} = \alpha_1 v_1 + \alpha_2 v_2$$

$$\text{output} = 0.0471 \times [2, 2, 2] + 0.9529 \times [3, 3, 3]$$

对向量的三个分量分别进行线性组合计算：

$$\begin{aligned} \text{分量}_1 &= 0.0471 \times 2 + 0.9529 \times 3 = 0.0942 + 2.8587 = 2.9529 \\ \text{分量}_2 &= 0.0471 \times 2 + 0.9529 \times 3 = 0.0942 + 2.8587 = 2.9529 \\ \text{分量}_3 &= 0.0471 \times 2 + 0.9529 \times 3 = 0.0942 + 2.8587 = 2.9529 \end{aligned}$$

得到精确输出向量为 $[2.9529, 2.9529, 2.9529]$，保留两位小数后：

$$\text{output} \approx [2.95, 2.95, 2.95]$$

最终计算结果

- **未归一化的注意力分数**：$[1, 4]$
- **归一化后的注意力权重**：$\approx [0.0471, 0.9529]$
- **最终加权后的注意力输出**：$\approx [2.95, 2.95, 2.95]$

量化

对称量化可能怎么考

已知浮点数范围：

$R \in [-2, 2]$

量化为 INT8，对称范围通常取：

$Q \in [-127, 127]$

> **注意：** 为了保持关于 0 对称，通常使用 $-127 \sim 127$。

- **第一步：计算 Scale**

  $$Scale = \frac{|R_{\max}|}{|Q_{\max}|} = \frac{2}{127} \approx 0.01575$$

- **第二步：量化某个浮点数**

假设 $R = 0.8$，则：

$$Q = \text{Round}\left(\frac{R}{Scale}\right) = \text{Round}\left(\frac{0.8}{0.01575}\right) = \text{Round}(50.79) = 51$$

所以 0.8 被量化为整数：

$$\boxed{Q = 51}$$

- **第三步：反量化**

  $$R' = Q \times Scale = 51 \times 0.01575 \approx 0.803$$

  原数是 0.8，反量化后是 0.803，差值就是量化误差。

非对称量化可能怎么考

已知浮点范围：

$R \in [-1, 3]$

量化到无符号 INT8：

$Q \in [0, 255]$

- **第一步：计算 Scale**

  $$Scale = \frac{R_{\max} - R_{\min}}{Q_{\max} - Q_{\min}} = \frac{3 - (-1)}{255 - 0} = \frac{4}{255} \approx 0.01569$$

- **第二步：计算零点 Z**

  $$Z = Q_{\max} - \text{Round}\left(\frac{R_{\max}}{Scale}\right) = 255 - \text{Round}\left(\frac{3}{0.01569}\right) = 255 - \text{Round}(191.25) = 255 - 191 = 64$$

  所以：

  $$\boxed{Z = 64}$$

  这里表示原始浮点数中的 0 ，大约映射到量化整数 64。

- **第三步：量化某个浮点数**

假设 $R = 1$，则：

$$Q = \text{Round}\left(\frac{R}{Scale} + Z\right) = \text{Round}\left(\frac{1}{0.01569} + 64\right) = \text{Round}(63.73 + 64) = \text{Round}(127.73) = 128$$

所以：

$$\boxed{Q = 128}$$

- **第四步：反量化**

  $$R' = (Q - Z) \times Scale = (128 - 64) \times 0.01569 \approx 1.004$$

和原值 1 很接近。

课程中主要知识点的“作用—应用”梳理

第4章：语言资源

语料库

作用：为模型提供真实语言样本，用于统计规律、训练参数和评价模型。

应用：

- 文本分类训练；
- 分词和命名实体识别；
- 语言模型训练；
- 机器翻译；
- 问答系统。

有标签的语料用于有监督训练；没有标签的大规模语料可以用于语言模型预训练。第4章中的医学NER样本，就是给每个字标注实体标签后训练识别模型。

平行语料库

作用：提供两种语言之间相互对应的句子或篇章。

应用：

- 机器翻译模型训练；
- 双语词典构建；
- 跨语言信息检索。

一句话：

> 平行语料就是“原文—译文”成对出现的数据。

树库

作用：提供人工标注的词性和句法结构，让模型学习句子由哪些短语构成。

应用：

- 词性标注；
- 句法分析；
- 语法研究。

PPT明确说明宾夕法尼亚树库用于词性标注和句法分析模型训练。

HowNet

作用：使用义原和概念关系描述词义，提供比普通词典更结构化的语义知识。

应用：

- 中文词义消歧；
- 语义相似度计算；
- 情感分析；
- 语义推理；
- 机器翻译。

它与WordNet作用相近，但组织知识的方式不同。

第5章：语言模型与文本表示

n-gram语言模型

作用：根据前面的若干个词，估计下一个词出现的概率，或者计算整个句子的概率。

应用：

- 拼音输入法中的候选句排序；
- 语音识别中的候选文本选择；
- 机器翻译候选结果排序；
- 文本生成；
- 分词消歧。

局限：

- 数据稀疏；
- 未出现的词组概率为0；
- 只能利用有限长度的上下文；
- 无法理解词语语义。

数据平滑

作用：解决n-gram中“训练语料没有出现过，所以概率等于0”的问题。

应用：

- 语言模型；
- 基于概率的分词；
- 语音识别和拼音输入法候选排序。

一句话：

> 平滑不是单独完成任务，而是防止语言模型把没见过的合理句子直接判成零概率。

One-hot

作用：把离散词语转换为计算机可处理的向量。

应用：

- 作为早期神经网络的输入；
- 表示词表中的词编号；
- CBOW和Skip-gram的输入表示。

局限：

- 维度高；
- 非常稀疏；
- 任意两个不同词都不相似；
- 无法表达语义关系。

Word2Vec

作用：把词表示成低维、稠密、带有语义信息的向量，使相似词在向量空间中更加接近。

应用：

- 词语相似度计算；
- 文本分类的输入；
- 聚类；
- 信息检索；
- 推荐和语义匹配；
- 作为CNN、RNN等模型的输入。

局限：

- 一个词只有一个固定向量；
- 不能根据上下文区分多义词；
- 对未登录词处理困难。

CBOW

作用：用目标词周围的上下文预测目标词，在预测过程中学习词向量。

应用：

- 主要用于训练静态词向量。

注意：

> CBOW本身主要是一种词向量训练方法，不是最终的文本分类或问答系统。

Skip-gram

作用：根据中心词预测周围的上下文词，从而训练词向量。

应用：

- 词向量学习；
- 对低频词的表示通常比CBOW更充分。

BERT动态上下文表示

作用：根据一个词所在的完整句子，实时计算它的向量。

应用：

- 词义消歧；
- 文本分类；
- 命名实体识别；
- 阅读理解；
- 语义匹配。

改进关系：

> Word2Vec中一个词永远是同一个向量；BERT中同一个词在不同句子里可以得到不同向量。

第6章：序列概率模型

HMM

作用：根据能够观察到的字、词或符号，推断背后看不见的状态标签。

例如：

```
代码块
```

```
观测序列：我 爱 北 京
隐藏状态：O O B-LOC I-LOC
```

应用：

- 中文分词；
- 词性标注；
- 命名实体识别；
- 语音识别。

一句话：

> 看见的是字词，HMM要猜的是字词背后的标签。

前向算法

作用：计算一个观测序列在给定HMM下出现的总概率。

应用：

- 判断某段观测序列与模型是否匹配；
- 比较不同HMM；
- HMM参数训练中的中间计算。

注意：

> 前向算法不是找一条最优路径，而是把所有可能路径的概率加起来。

Viterbi算法

作用：在所有可能的状态序列中，找到概率最大的一条。

应用：

- HMM分词；
- 词性标注；
- 命名实体识别；
- 语音识别解码。

一句话：

> 前向算法求“总概率”，Viterbi求“最可能的标签路径”。

第7章：分词、NER和Token

正向/逆向最大匹配

作用：根据词典，从句子的前面或后面优先匹配最长词语。

应用：

- 中文分词；
- 作为简单、快速的分词基线。

局限：

- 依赖词典；
- 难以处理未登录词；
- 不能很好解决歧义。

BMES与BIO

作用：把分词或实体识别转化成每个字的标签分类问题。

应用：

- BMES：中文分词；
- BIO：命名实体识别；
- 医疗症状、疾病、药物识别。

注意：

> BMES和BIO只是标签体系，不是模型。真正预测这些标签的可以是HMM、CRF、BiLSTM或BERT。

命名实体识别

作用：从文本中找到实体的边界，并判断实体类型。

应用：

- 人名、地名、机构名识别；
- 医疗症状、疾病、药物识别；
- 知识图谱构建；
- 搜索和问答；
- 信息抽取。

BPE、WordPiece

作用：把低频词或未登录词拆成更小的子词，既控制词表规模，又尽量保留语义。

应用：

- GPT使用BPE类方法；
- BERT使用WordPiece；
- 大语言模型Token生成；
- 缓解未登录词问题。

PPT将Token生成分成文本拆分、数字编码和向量嵌入三步，并说明子词方法可以缓解整词词表爆炸和OOV问题。

第8章：深度神经网络

CNN

作用：通过不同大小的卷积核，提取文本中的局部关键词和短语模式。

例如：

- “非常好看”
- “不值得购买”
- “服务态度差”

应用：

- 文本分类；
- 情感分析；
- 垃圾邮件识别；
- 新闻分类。

局限：

- 更擅长局部模式；
- 对远距离依赖建模较弱。

最大池化

作用：从一张特征图中保留最明显的局部特征，并把变长文本压缩成固定长度向量。

应用：

- TextCNN中的句子表示；
- 图像和文本特征降维。

一句话：

> 卷积负责“找特征”，池化负责“留下最重要的特征”。

RNN

作用：按顺序处理文本，并把前面读到的信息传递给后面。

应用：

- 语言模型；
- 文本分类；
- 分词；
- 命名实体识别；
- 序列生成。

局限：

- 难以并行；
- 长序列中容易遗忘早期信息；
- 可能发生梯度消失。

LSTM

作用：在RNN基础上加入门控机制，有选择地记忆或遗忘信息。

应用：

- 长文本分类；
- 序列标注；
- 语言模型；
- 机器翻译。

改进关系：

> LSTM主要是为了解决普通RNN难以保存长期信息的问题。

第9章：Transformer与大模型

自注意力

作用：计算句子中每个词与其他词的关系，使一个词的表示融合整个句子的上下文。

应用：

- 机器翻译；
- 文本分类；

- 阅读理解；
- 命名实体识别；
- 文本生成。

多头注意力

作用：让模型从多个不同角度理解词语关系。

例如不同的头可能分别关注：

- 主谓关系；
- 指代关系；
- 时间关系；
- 语义相关词。

一句话：

> 一个注意力头看一个角度，多头一起看多个角度。

位置编码

作用：告诉Transformer每个Token在句子中的位置。

为什么需要：

> 自注意力本身并不知道词的先后顺序，因此必须额外加入位置信息。

微调

作用：把已经掌握通用知识的预训练模型，调整为适合某个领域或任务的模型。

应用：

- 医疗问答；
- 法律分类；
- 情感分析；
- 指令对话；
- 专业文本生成。

LoRA

作用：冻结大模型原有参数，只训练少量低秩矩阵，以较低显存和计算成本完成微调。

应用：

- 消费级GPU微调大模型；
- 为同一个基座模型训练多个任务适配器；
- 领域和风格定制。

一句话：

> 全参数微调是改整个模型，LoRA只训练少量“外挂参数”。

RAG

作用：在大模型回答之前，先从外部知识库中检索相关资料，再让模型依据资料作答。

应用：

- 企业知识库问答；
- 文档问答；
- 医疗、法律知识查询；
- 私有资料助手；
- 需要引用来源的问答。

解决的问题：

- 模型知识过时；
- 私有知识缺失；
- 回答产生幻觉。

模型量化

作用：把高精度模型参数转换为INT8、INT4等低精度表示，减少模型占用。

应用：

- 手机和边缘设备部署；
- 降低GPU显存需求；
- 提高推理速度；
- 降低部署成本。

注意：

> 量化不负责让模型“更聪明”，它负责让模型“更省资源、更容易运行”。

你整理时最重要的三个问题

每看到一个名词，不要只抄定义，强制问自己：

1. **它解决了前一种方法的什么问题？**
2. **它在系统中输入什么，输出什么？**
3. **哪些实际任务会使用它？**

例如：

代码块

```
WordNet
解决：词义和词语语义关系问题
输入：词或概念
输出：义项、同义词、上下位关系等
应用：消歧、相似度、推理、翻译
```

代码块

```
Viterbi
解决：标签组合太多，无法逐条枚举
输入：观测序列和HMM参数
输出：概率最大的状态序列
应用：分词、词性标注、NER
```

代码块

```
CNN
解决：人工提取文本特征麻烦
输入：词向量矩阵
输出：局部特征或分类结果
应用：文本分类、情感分析
```

这样整理以后，知识点就不再是一堆孤立名词，而会变成：

> **数据资源提供知识 → 文本表示把语言变成数字 → 模型处理数字 → 算法得到结果 → 具体任务使用结果。**

先把这些名词分成四个层级

你现在觉得乱，主要是因为它们其实不是同一层面的概念：

最重要的一句话是：

> **BERT不是和Transformer并列的网络。BERT是使用Transformer编码器搭建出来的具体模型。**

可以把关系简单画成：

代码块

```
神经网络
├── CNN
├── RNN
│   └── LSTM
└── Transformer
    ├── Encoder-only → BERT
    ├── Decoder-only → GPT、LLaMA、Qwen
    └── Encoder-Decoder → 原始Transformer、T5、BART
```

一、CNN：拿一个小窗口，寻找局部特征

CNN最早主要用于图像，但也可以处理文本。

例如句子：

代码块

```
这部电影非常好看
```

先把每个词转换成词向量，形成一个矩阵。CNN的卷积核会像窗口一样滑动：

代码块

```
这部电影
部电影非常
```

```
3    电影非常好看
```

它主要寻找局部组合，例如：

- “非常好”
- “不喜欢”
- “质量很差”

这些局部组合可以看成文本中的 n-gram 特征。

在文本分类中，通常是：

```
代码块
1    词向量矩阵
2    → 卷积提取局部短语特征
3    → 池化保留最重要特征
4    → 全连接层
5    → 分类结果
```

你PPT第8章第28～32页讲的就是TextCNN：卷积核高度对应不同长度的短语，卷积之后再池化，最终得到句子的整体表示。

CNN的特点

优点是计算快、容易并行、擅长提取局部特征，适合文本分类。

缺点是窗口大小有限。句子中两个词距离很远时，CNN不容易直接建立它们之间的联系。

可以记成：

> **CNN像拿着放大镜在句子上滑动，寻找局部关键词和短语。**

二、RNN：一个词一个词地读，并保留历史信息

RNN专门处理序列。

普通神经网络处理当前输入时，不会自动记住之前发生过什么；RNN增加了一个隐藏状态：

[

h_t=f(x_t,h_{t-1})

]

其中：

- (x_t)：当前词；
- (h_{t-1})：读完前面内容后留下的记忆；
- (h_t)：读完当前词后更新的记忆。

例如：

```
代码块
1    我 → 喜欢 → 自然 → 语言 → 处理
```

RNN必须按顺序处理：

```
代码块
1    先读“我”
2    再读“喜欢”
3    再读“自然”
4    ……
```

读到“处理”时，隐藏状态中已经包含前面词的信息。

你PPT中把它描述为：上一时刻的状态可以作用于下一时刻，因此适合天气预测、下一个词预测、序列标注等任务。

RNN能做什么

它有几种常见输出方式。

1.1.2 整个句子输出一个结果

```
代码块
1    输入：这个电影太好看了
2    输出：正面情感
```

可以使用最后一个隐藏状态，或者把所有隐藏状态做平均，作为句子表示。

1.1.2 每个词输出一个结果

```
代码块
1    张三      在      北京      工作
2    人名      O       地名      O
```

适合：

- 分词
- 词性标注
- 命名实体识别

1.1.3 输入一个序列，输出另一个序列

例如机器翻译：

```
代码块
1   I love you
2   →
3   我爱你
```

这时通常使用Encoder-Decoder结构。

RNN的问题

RNN虽然理论上能记住前面所有词，但实际上序列很长时，早期信息很容易逐渐消失。

并且它必须按顺序计算，不能同时处理所有词：

```
代码块
1   h1算完 → 才能算h2 → 才能算h3
```

所以训练速度较慢，也不擅长特别长的依赖关系。

三、LSTM：带有“记忆开关”的RNN

LSTM不是和RNN完全不同的一类网络。

> **LSTM是RNN的改进版本。**

普通RNN的问题是：它把所有历史信息混在隐藏状态里不断传递，重要信息可能慢慢消失。

LSTM增加了一个细胞状态 (C_t)，并使用三个“门”控制信息。

1.1 遗忘门

决定以前的信息保留多少。

例如前文一直在讨论“小明”，后来话题已经换成“小红”，模型可以逐渐忘掉关于小明的信息。

1.2 输入门

决定当前的新信息写入多少。

例如读到：

```
代码块
1    小明出生在北京
```

“出生在北京”可能是需要长期保存的重要信息。

1.3 输出门

决定当前时刻从记忆中拿出多少信息，形成隐藏状态。

所以LSTM的核心思想是：

```
代码块
1    不再把所有内容都机械地传下去
2    而是选择性地：
3    忘记什么
4    记住什么
5    输出什么
```

PPT第5章第43～48页介绍了LSTM的细胞状态、遗忘门、输入门和输出门，并说明它通过加法形式保存历史记忆，缓解普通RNN中的梯度消失问题。

可以记成：

> **RNN是边读边记，LSTM是边读边决定哪些该记、哪些该忘。**

不过LSTM仍然需要按顺序计算，所以并行能力依然有限。

四、Encoder和Decoder到底是什么

Encoder和Decoder不是某一种固定网络，它们是一种任务分工。

Encoder：负责理解和表示输入

Encoder的作用是：

```
代码块
1    原始输入
```

```
2    →
3    上下文语义表示
```

例如输入：

```
代码块
1    我有一只猫
```

Encoder处理后，不再只是得到“我”“有”“猫”各自孤立的词向量，而是得到融合上下文后的表示。

“猫”的表示中会包含：

- 它和“我”的关系；
- 它和“一只”的关系；
- 它在整个句子中的含义。

所以encode可以理解为：

> **把人类输入转换为机器内部能够使用的语义表示。**

Decoder：负责根据已有信息生成输出

Decoder的作用是：

```
代码块
1    语义表示 + 已经生成的内容
2    →
3    预测下一个输出
```

例如翻译时：

```
代码块
1    Encoder读取：I have a cat
2    Decoder生成：我 → 有 → 一 → 只 → 猫
```

decode可以理解为：

> **把模型内部的语义表示还原成需要的输出序列。**

五、RNN中的Encoder-Decoder

早期机器翻译经常使用两个RNN或LSTM。

```
输入序列
→ RNN Encoder
→ 语义向量c
→ RNN Decoder
→ 输出序列
```

Encoder读完整个输入句子，把信息压缩成一个语义向量 (c)。

Decoder再根据 (c) 逐词生成结果。

例如：

```
Encoder: 读取 Tom chases Jerry
                ↓
           语义向量c
                ↓
Decoder: 汤姆 → 追逐 → 杰瑞
```

问题是：如果输入句子很长，全部信息都压缩到一个固定长度的向量中，很容易丢失信息。你PPT第8章第44～49页正是从这个问题引出了注意力机制。

六、注意力机制：当前这一步应该重点看哪里

注意力机制解决的核心问题是：

> **面对很多输入信息，当前任务最应该关注哪一部分？**

例如把：

```
Tom chases Jerry
```

翻译为中文。

Decoder准备输出“汤姆”时，应重点关注Tom：

```
代码块
1   Tom      0.8
2   chases   0.1
3   Jerry    0.1
```

准备输出“追逐”时，应重点关注chases：

```
代码块
1   Tom      0.1
2   chases   0.7
3   Jerry    0.2
```

因此，每生成一个词，关注的输入位置都可以不同。

Q、K、V怎么理解

注意力里有三个核心量：

- Query：我现在想寻找什么；
- Key：每条信息的索引或标签；
- Value：每条信息实际包含的内容。

可以用“查资料”理解。

假设你要写“软件工程论文”：

- Query：软件工程论文需要哪些知识；
- Key：计算机、自动化、外语等不同领域标签；
- Value：各领域真正提供的知识内容。

先用Query和所有Key计算相关程度，再根据相关程度对Value加权求和。

计算过程是：

$$
[
\operatorname{Attention}(Q,K,V)
$$

\operatorname{softmax}

\left(

\frac{QK^T}{\sqrt{d_k}}

\right)V

]

过程可以拆成三步：

```
1. Q和K做点积
   → 计算相关程度

2. 经过Softmax
   → 转换为总和为1的注意力权重

3. 用权重对V加权求和
   → 得到最终输出
```

所以相关性越大，对应Value对最终结果的贡献越大。PPT第8章第50～54页给出了Q、K、V以及机器翻译中的动态context示例。

七、自注意力和普通注意力有什么区别

普通的交叉注意力

在机器翻译的Encoder-Decoder中：

- Q来自Decoder；
- K、V来自Encoder。

也就是说，Decoder根据当前生成需要，去查询Encoder的输入信息。

这叫交叉注意力：

```
Decoder提出问题Q
去Encoder输出的K、V中查找相关内容
```

自注意力

自注意力中，Q、K、V都来自同一个序列。

例如：

```
我 喜欢 自然 语言 处理
```

处理“语言”时，它会分别和：

```
代码块
1    我
2    喜欢
3    自然
4    语言
5    处理
```

计算相关性，从而把其他词的信息融合进“语言”的新表示中。

所以：

> **Self-Attention不是只关注自己，而是序列中的每个词都去关注同一序列中的所有词。**

这让每个词都能直接获取全局上下文。

八、多头注意力：从多个角度同时观察

单头注意力只有一组：

```
代码块
1    WQ、WK、WV
```

多头注意力有多组不同的参数：

```
代码块
1    第1头：WQ¹、WK¹、WV¹
2    第2头：WQ²、WK²、WV²
3    第3头：WQ³、WK³、WV³
4    ……
```

每个头独立计算注意力，因此不同头可能学习不同关系。

例如句子：

```
代码块
1    小明把他买的书送给了小红
```

不同注意力头可能分别关注：

- 一个头关注“他”指代谁；
- 一个头关注“送给”的动作关系；
- 一个头关注“小明”和“小红”的人物关系；
- 一个头关注“书”作为动作对象。

最后把各个头的结果拼接，再做一次线性变换。

可以把它类比为CNN的多个卷积核：

- 多个卷积核提取不同局部特征；
- 多个注意力头提取不同语义关系。

PPT第9章第16～17页也正是用“CNN中的多kernel”类比多头注意力。

需要注意：

> **多头注意力不是把同一个注意力结果重复很多遍，而是让多组参数在不同表示空间里独立寻找关系。**

九、Transformer：去掉循环，主要依靠注意力

Transformer要解决RNN和LSTM的两个主要问题：

1. 必须按顺序计算，训练难以并行；
2. 长距离信息需要经过很多时间步传递，容易损失。

Transformer不再使用RNN的循环结构，而是让所有词通过自注意力直接建立联系。

例如句子有10个词，RNN要：

```
代码块
1    第1个 → 第2个 → 第3个 → …… → 第10个
```

Transformer训练时可以同时处理所有词：

```
代码块
1    所有词之间直接计算注意力关系
```

因此：

- 更容易并行；
- 任意两个词之间可以直接建立关系；

- 更适合长文本。

你的Transformer PPT第3页也明确将其优势概括为解决RNN“无法并行计算”和“长序列依赖捕捉弱”的问题。

Transformer Encoder内部

一个Encoder Block通常包括：

```
代码块
1  多头自注意力
2  → Add & LayerNorm
3  → 前馈神经网络
4  → Add & LayerNorm
```

输入前还需要：

```
代码块
1  词向量Embedding + 位置编码
```

因为Transformer没有RNN那种天然的先后顺序，所以必须额外加入位置编码，告诉模型词的顺序。

Transformer Decoder内部

一个Decoder Block通常包括：

```
代码块
1  带掩码的多头自注意力
2  → 交叉注意力
3  → 前馈神经网络
```

其中：

带掩码的自注意力

生成第 (t) 个词时，只能看到前面已经出现的词，不能偷看未来答案。

例如预测第3个词时：

```
代码块
1  可以看第1、2个词
```

```
2    不能看第4、5个词
```

交叉注意力

- Q来自Decoder；
- K、V来自Encoder。

Decoder利用它寻找当前生成步骤最需要的输入信息。

Transformer的Encoder、Decoder及掩码和交叉注意力在PPT第24～28页有完整结构。

十、BERT：只有Transformer Encoder的理解型模型

BERT不是一种新的基础网络结构。

> **BERT = 多层Transformer Encoder堆叠起来的预训练模型。**

BERT只保留Encoder，因此每个词可以同时关注：

- 左边的词；
- 右边的词。

这叫双向上下文。

例如：

```
代码块
1    我去银行办理存款
2    河流两岸种满树木
```

“银行”在两个句子中的含义不同。BERT可以同时利用左右上下文判断它具体表示金融机构还是河岸。

BERT输入通常包括三部分：

```
代码块
1    Token Embedding
2    + Position Embedding
3    + Token Type Embedding
```

输出是每个token融合上下文后的向量。开头通常加入 `[CLS]` ，其输出可以用于表示整段文本。你PPT第40～50页介绍了BERT的输入、Encoder堆叠、 `[CLS]` 输出和token级输出。

BERT适合什么任务

BERT主要擅长理解类任务：

- 文本分类
- 情感分析
- 命名实体识别
- 词性标注
- 抽取式问答
- 句子相似度判断

例如文本分类：

代码块

```
[CLS] 这个电影非常好看 [SEP]
      ↓
取[CLS]对应向量
      ↓
全连接分类器
      ↓
正面 / 负面
```

BERT虽然训练时会预测被遮盖的词，但它不是典型的逐词长文本生成模型。

十一、BERT、GPT和原始Transformer的关系

这三个最容易混淆。

例如：

代码块

```
情感分类
“这部电影很好看”
→ BERT

文本续写
“从前有一座山……”
→ GPT

机器翻译
“I love NLP”
→ Transformer Encoder-Decoder
```

这也是PPT第60页对三种Transformer结构的总结。

十二、为什么这些东西“到处都有”

因为现代模型往往不是只用一种结构，而是把不同模块组合起来。

例如：

```
代码块
CNN + LSTM
```

CNN先提取局部短语特征，LSTM再建模前后顺序。

```
代码块
BiLSTM + Attention
```

BiLSTM获得序列表示，Attention从所有位置中选择重点。

```
代码块
BERT + 全连接层
```

BERT负责理解文本，全连接层负责分类。

```
代码块
Transformer + CNN
```

Transformer建模全局关系，CNN补充局部特征。

所以你看到“注意力”“Encoder”“Embedding”“Softmax”到处出现，是因为它们都是可复用模块。

就像搭积木：

```
Embedding：把词变成向量
CNN：提取局部特征
RNN/LSTM：按顺序积累信息
Attention：选择重点信息
Encoder：理解输入
Decoder：生成输出
Softmax：转换成类别或词的概率
```

最后用六句话记住

> **CNN：看局部，找关键词和短语。**
>
> **RNN：按顺序读，把前面的信息传给后面。**
>
> **LSTM：给RNN增加门，选择哪些记住、哪些忘掉。**
>
> **Attention：根据当前需要，给不同信息分配不同权重。**
>
> **Transformer：主要靠自注意力建立全局关系，训练可以并行。**
>
> **BERT：由Transformer Encoder组成，主要负责文本理解。**

最清晰的一条发展线可以记为：

```
RNN顺序记忆
→ LSTM改善长期记忆
→ RNN Encoder-Decoder加入Attention
→ Transformer用Self-Attention替代循环
→ BERT只使用Transformer Encoder做文本理解
→ GPT使用Decoder式因果注意力做文本生成
```

你现在觉得 Transformer 零碎，是因为 PPT 把它按“整体结构、编码器、解码器、注意力类型、模型变体”分别介绍了，但这些概念实际上是嵌套关系。

先不要记公式，先用三个问题把所有注意力分类：

1. **Q、K、V来自哪里？**——自注意力还是交叉注意力
2. **一个词能看哪些位置？**——双向还是单向
3. **同时从几个角度看？**——单头还是多头

一、先看 Transformer 的总结构

原始 Transformer 是一个完整的 **Encoder-Decoder模型**：

```
代码块
输入文本
   ↓
词向量 Embedding + 位置编码
   ↓
多个 Encoder Block
   ↓
输入文本的上下文表示
   ↓
多个 Decoder Block
   ↓
Linear + Softmax
   ↓
逐词生成输出文本
```

以机器翻译为例：

```
代码块
输入：I love NLP
          ↓
       Encoder
          ↓
    输入句子的语义表示
          ↓
       Decoder
          ↓
输出：我 → 爱 → NLP
```

Transformer解决的核心问题是：RNN必须按照顺序处理，而Transformer主要通过注意力机制让不同位置直接联系，因此训练时更容易并行，也更容易建立长距离关系。

二、注意力的第一种分类：自注意力与交叉注意力

这两个概念看的是：

> **Q、K、V是不是来自同一段序列。**

1.4 自注意力 Self-Attention

自注意力中：

```
代码块
1    Q、K、V都来自同一段序列
```

例如输入：

```
代码块
1    小明 喜欢 自然 语言 处理
```

处理“语言”时：

```
代码块
1    “语言”的Q
2    分别与“小明、喜欢、自然、语言、处理”的K计算相关性
3    然后根据权重，对这些词的V加权求和
```

所以“语言”的新表示不再只是“语言”自己的词向量，而是融合了整句话的上下文信息。

可以理解为：

> **一句话内部的词互相交流信息。**

在Transformer Encoder中，输入词之间使用自注意力；在Decoder中，已经生成的词之间也使用自注意力。

PPT第13～15页说明，自注意力会让每个词与同一输入序列中的其他词计算关系，从而得到带有上下文的表示。

1.2 交叉注意力 Cross-Attention

交叉注意力中，Q和K、V来自不同地方：

```
代码块
1   Q：来自Decoder
2   K、V：来自Encoder
```

以翻译为例：

```
代码块
1   Encoder输入：I love NLP
2   Decoder正在生成：我 爱 ...
```

Decoder准备生成下一个词时，会提出一个Query：

```
代码块
1   “我当前需要从原文中寻找什么信息？”
```

然后用它去查询Encoder输出的所有K、V。

例如生成“爱”时，可能主要关注输入中的“love”：

```
代码块
1   I       0.1
2   love    0.8
3   NLP     0.1
```

因此交叉注意力的作用是：

> **让Decoder在生成当前词时，从Encoder的输入信息中寻找最相关的部分。**

PPT第24和28页给出的关系就是：

```
代码块
1   Q = Decoder的输出
2   K、V = Encoder的输出
```

三、注意力的第二种分类：双向与单向

双向、单向说的不是Q、K、V来自哪里，而是：

> **当前位置能够看到哪些词。**

这是另一套分类标准，不能和“自注意力、交叉注意力”直接并列。

1.1 双向注意力

双向注意力中，一个词既可以看左边，也可以看右边。

例如：

```
代码块
1    我  喜欢  自然  语言  处理
```

处理“语言”时，可以同时关注：

```
代码块
1    左边：我、喜欢、自然
2    自己：语言
3    右边：处理
```

可见范围可以画成：

```
代码块
1    位置1：看 1 2 3 4 5
2    位置2：看 1 2 3 4 5
3    位置3：看 1 2 3 4 5
4    位置4：看 1 2 3 4 5
5    位置5：看 1 2 3 4 5
```

这种方式特别适合理解完整文本，因为模型可以利用一个词前后两边的信息。

所以：

```
代码块
1    Transformer Encoder
2    使用双向自注意力
```

BERT只使用Encoder，因此BERT也是双向自注意力模型。

1.2 单向注意力

单向注意力通常是指：

> 当前位置只能看自己和前面的内容，不能看后面的内容。

例如生成：

```
代码块
1   床 前 明 月 光
```

预测每个位置时：

```
代码块
1   “床”只能看“床”
2   “前”只能看“床、前”
3   “明”只能看“床、前、明”
4   “月”只能看“床、前、明、月”
```

矩阵关系类似：

```
代码块
1           床  前  明  月  光
2   床      ✓
3   前      ✓   ✓
4   明      ✓   ✓   ✓
5   月      ✓   ✓   ✓   ✓
6   光      ✓   ✓   ✓   ✓   ✓
```

这叫：

- 单向自注意力
- 因果自注意力
- Causal Self-Attention
- Masked Self-Attention

这些名称在当前课程语境中基本指同一件事。

为什么只能看前面？因为生成文本时，不能在预测“月”时提前知道后面的“光”。

四、Masked Attention和单向注意力是什么关系

Transformer训练Decoder时，标准答案的所有词已经同时放入模型。

例如标准答案是：

```
代码块
1    我 爱 NLP
```

如果不进行限制，模型在预测“爱”时可能直接看到后面的“NLP”，相当于偷看答案。

所以加入因果掩码：

```
代码块
1    允许关注的位置：保留
2    未来位置：屏蔽
```

因此：

> **Masked Self-Attention是实现单向自注意力的方法。**

PPT第25～27页说明，因果掩码保证第 (t) 个token只能关注自己和之前的token，不能看到未来信息。

这里还要区分两个容易混淆的“Mask”：

Decoder中的Causal Mask

屏蔽的是未来位置：

```
代码块
1    预测当前位置时不能看后面的词
```

BERT中的Masked Language Model

遮住的是部分输入词：

```
代码块
1    我喜欢吃 [MASK]
```

让BERT预测被遮住的词。

BERT虽然进行掩码词预测，但其内部依然是**双向自注意力**。也就是说，BERT的“Masked LM”和Decoder的“Masked Self-Attention”不是同一个概念。

五、注意力概念的完整关系

把这些概念按照不同分类标准整理：

```
注意力机制
 │
 ├── 按Q、K、V的来源分类
 │    ├── 自注意力
 │    │    Q、K、V来自同一序列
 │    └── 交叉注意力
 │         Q来自Decoder，K、V来自Encoder
 │
 ├── 按可见范围分类
 │    ├── 双向注意力
 │    │    可以看左边和右边
 │    └── 单向/因果注意力
 │         只能看自己和前面
 │
 └── 按并行角度数量分类
      ├── 单头注意力
      └── 多头注意力
```

因此，一个注意力层可以同时具有多个名称：

Encoder中的注意力

```
双向 + 自注意力 + 多头
```

所以完整名称是：

> 多头双向自注意力

Decoder中的第一个注意力

```
单向/因果 + 自注意力 + 多头
```

所以完整名称是：

> 带因果掩码的多头自注意力

Decoder中的第二个注意力

```
代码块
交叉注意力 + 多头
```

所以完整名称是：

> 多头交叉注意力

最关键的对应关系就是：

```
代码块
Encoder：双向自注意力

Decoder第一层：单向/Masked自注意力

Decoder第二层：交叉注意力
```

六、多头注意力不是另一种方向

多头注意力不是和自注意力、交叉注意力相互排斥的概念。

它只是说：

> 同一次注意力计算，用多组不同的参数，从多个角度分别计算。

例如句子：

```
代码块
小明把他买的书送给了小红
```

不同的头可能分别学习：

```
代码块
```

```
头1：关注“他”指的是谁
头2：关注“谁送给谁”
头3：关注“书”是什么动作的对象
头4：关注词语之间的位置关系
```

最后：

代码块

```
各个头独立计算
→ 拼接
→ 线性变换
→ 得到最终结果
```

所以可以出现：

- 多头双向自注意力
- 多头单向自注意力
- 多头交叉注意力

PPT第16～17页用CNN中的多个卷积核类比多头注意力：不同的头在不同特征空间中提取不同关系。

七、Encoder内部到底有什么

输入首先经过：

代码块

```
Token Embedding + Position Embedding
```

为什么需要位置编码

自注意力本身只计算词与词之间的关系，并不知道谁在前、谁在后。

例如：

代码块

```
张明欠我300元
我欠张明300元
```

词语接近，但顺序改变后意思完全不同，因此需要额外加入位置信息。

然后进入Encoder Block：

```
输入
 ↓
多头双向自注意力
 ↓
Add & LayerNorm
 ↓
前馈神经网络 FFN
 ↓
Add & LayerNorm
 ↓
输出
```

这个Block会堆叠多层。

自注意力做什么

让不同词之间交流信息：

```
词与词之间的信息融合
```

FFN做什么

对每个位置的向量分别进行更深的非线性加工：

```
每个词内部的特征变换
```

可以这样区分：

> 注意力负责“词与词之间交流”，FFN负责“每个词自己加工信息”。

Add & LayerNorm做什么

Add是残差连接：

```
```

```
输出 = 原始输入 + 子层计算结果
```

防止原始信息丢失，也有利于深层网络训练。

LayerNorm负责稳定不同层中的数值分布。

Encoder最终输出的不是一个向量，而是**每个输入词对应一个上下文向量**：

代码块

```
I       → 上下文向量C1
love    → 上下文向量C2
NLP     → 上下文向量C3
```

这些向量之后作为Decoder交叉注意力中的K、V。PPT第7～23页给出了完整Encoder流程。

八、Decoder内部到底有什么

Decoder Block比Encoder多一个注意力层：

代码块

```
已经生成的内容
 ↓
带因果掩码的多头自注意力
 ↓
Add & LayerNorm
 ↓
多头交叉注意力
 ↓
Add & LayerNorm
 ↓
前馈神经网络
 ↓
Add & LayerNorm
 ↓
输出
```

以翻译为例，Decoder当前已经生成：

代码块

```
我 爱
```

它要预测下一个词。

第一步：Masked Self-Attention

分析自己已经生成的内容：

```
代码块
1    “我”和“爱”之间有什么关系？
2    当前生成到哪里了？
```

不能看未来答案。

第二步：Cross-Attention

从Encoder表示中寻找当前最需要的原文信息：

```
代码块
1    当前应该关注I、love还是NLP？
```

第三步：FFN

进一步处理当前位置的特征。

第四步：Linear + Softmax

输出词表中每个词的概率：

```
代码块
1    NLP        0.75
2    自然       0.08
3    语言       0.06
4    处理       0.03
5    ……
```

然后选择或采样出下一个token。

九、Transformer三大变体

原始Transformer包含Encoder和Decoder，但后来根据任务不同，形成了三种主要结构。

1.1 Encoder-Only

```
输入
 ↓
多个Encoder Block
 ↓
上下文向量
 ↓
分类头或其他任务头
```

注意力：

```
双向自注意力
```

没有：

- Decoder
- 因果自注意力
- 交叉注意力

代表模型：

```
BERT、RoBERTa
```

适合：

- 文本分类
- 情感分析
- 命名实体识别
- 词性标注
- 抽取式问答
- 句子相似度

核心能力是：

> 看完整输入，理解文本。

BERT就是多个Transformer Encoder堆叠而成。

1.2 Decoder-Only

```
代码块
1  提示文本
2   ↓
3  多个Decoder式Block
4   ↓
5  预测下一个token
6   ↓
7  把token加入输入，继续预测
```

注意力：

```
代码块
1  单向/因果自注意力
```

代表模型：

```
代码块
1  GPT、LLaMA、Qwen
```

适合：

- 文本生成
- 对话
- 续写
- 代码生成
- 开放式问答

核心能力是：

> 根据前文预测下一个词。

需要特别注意：Decoder-only模型没有Encoder，因此也不需要交叉注意力。它通常保留Decoder的因果自注意力和FFN结构，但删除原始Decoder中的交叉注意力层。

1.3 Encoder-Decoder

```
输入文本
 ↓
Encoder
 ↓
输入的上下文表示
 ↓
Decoder
 ↓
输出文本
```

注意力包含三种：

```
Encoder：双向自注意力
Decoder内部：单向自注意力
Encoder与Decoder之间：交叉注意力
```

代表模型：

```
原始Transformer、T5、BART
```

适合：

- 机器翻译
- 文本摘要
- 问题生成
- 输入序列到输出序列的转换

核心能力是：

> Encoder负责理解输入，Decoder根据输入生成另一段序列。

PPT第36～40页就是按照Decoder-only、Encoder-only及完整结构梳理这些变体。

十、训练和生成为什么一会儿并行、一会儿串行

这也是Transformer中容易混乱的地方。

Encoder

Encoder处理整个输入，所以训练和使用时都可以同时处理所有输入词。

```
代码块
1   I、love、NLP同时进入Encoder
```

Decoder训练时

训练数据已经给出了标准答案：

```
代码块
1   <START> 我 爱 NLP
```

通过Teacher Forcing，把正确答案整体提供给Decoder，再使用因果掩码阻止每个位置看到未来词。

因此训练时可以并行计算：

```
代码块
1   预测“我”的位置
2   预测“爱”的位置
3   预测“NLP”的位置
4   可以同时计算
```

但每个位置不能偷看右边。

Decoder实际生成时

未来词还不存在，所以必须：

```
代码块
1   先生成“我”
2   → 再生成“爱”
3   → 再生成“NLP”
```

因此：

> Decoder训练可以并行，但推理生成仍然是逐词串行的。

十一、一张最终关系图

代码块

```
完整Transformer
│
├── 输入层
│    └── Token Embedding + Position Embedding
│
├── Encoder × N
│    ├── 多头双向自注意力
│    ├── Add & LayerNorm
│    ├── FFN
│    └── Add & LayerNorm
│
├── Decoder × N
│    ├── 带因果掩码的多头自注意力
│    ├── Add & LayerNorm
│    ├── 多头交叉注意力
│    ├── Add & LayerNorm
│    ├── FFN
│    └── Add & LayerNorm
│
└── Linear + Softmax
     └── 输出下一个token的概率
```

三种模型变体：

代码块

```
BERT
= 只要Encoder
= 双向自注意力
= 理解型任务

GPT / LLaMA / Qwen
= 只要Decoder式结构
= 单向因果自注意力
= 生成型任务

原始Transformer / T5 / BART
= Encoder + Decoder
= 双向自注意力 + 因果自注意力 + 交叉注意力
```

```
= 翻译、摘要等序列转换
```

复习时最先背住这一句：

> **自注意力和交叉注意力看Q、K、V来自哪里；单向和双向看能看到哪些位置；多头注意力看同时从几个角度计算。Encoder用双向自注意力，Decoder先用因果自注意力，再用交叉注意力。**

一、先建立总框架：这些模型分别解决什么问题

把它们看成一条不断改进的路线：

代码块

```
FNN
只能处理固定长度输入
    ↓
CNN
用局部窗口提取短语特征，适合文本分类
    ↓
RNN
按顺序读取文本，能够保存历史信息
    ↓
LSTM / BiLSTM
有选择地记忆和遗忘，改善长距离信息保存
    ↓
Seq2Seq
用Encoder读取输入、Decoder生成输出
    ↓
Seq2Seq + Attention
Decoder每一步动态关注不同输入位置
    ↓
Transformer
完全以注意力为核心，并行处理整个序列
    ↓
三大变体
Encoder-only：BERT、RoBERTa，侧重理解
Decoder-only：GPT、LLaMA、Qwen，侧重生成
Encoder-Decoder：T5、BART，侧重序列转换
```

它们并不全是同一层面的概念：

二、所有NLP神经模型的共同处理框架

不管是CNN、RNN还是Transformer，通常都经历：

```
原始文本
→ 分词或Tokenization
→ token转换为数字ID
→ Embedding转换为向量
→ 主体网络提取文本特征
→ 任务输出层
→ 得到分类、标签或生成文本
```

例如：

```
原文：我喜欢自然语言处理
分词：[我，喜欢，自然，语言，处理]
ID：[15，208，91，72，316]
Embedding：每个ID变成一个k维向量
主体模型：CNN / LSTM / Transformer
输出：正面情感、实体标签、下一个词等
```

其中：

- **Embedding**负责把离散词语变成连续向量；
- **主体网络**负责提取上下文特征；
- **全连接层或任务头**负责把特征转换成最终答案；
- **Softmax**通常把输出分数转换为类别或词的概率。

三、FNN：最基础的前馈神经网络语言模型

虽然你没有重点写FNN，但它是理解后续模型的重要起点。

1.4 输入和输出

FNN语言模型的任务是：

```
代码块
1    输入：前面固定数量的词
2    输出：下一个词的概率分布
```

例如：

```
代码块
1    输入：这 本 小说 很
2    输出：
3    “有趣” 0.55
4    “枯燥” 0.25
5    “好看” 0.15
6    ……
```

1.2 网络结构

```
代码块
1    前n-1个词
2    → 查Embedding表得到词向量
3    → 将词向量拼接
4    → 全连接隐藏层
5    → 非线性激活，如tanh或ReLU
6    → 全连接输出层
7    → Softmax
8    → 词表中每个词的概率
```

PPT中的FNN语言模型就是把前面若干词的词向量拼接，通过隐藏层和Softmax预测下一个词。

1.3 关键机制

FNN内部只有单向的信息传播：

```
输入层 → 隐藏层 → 输出层
```

没有循环、没有记忆状态，也没有注意力。

1.4 特点和问题

优点是结构简单、容易并行。

问题是输入长度固定。例如设定只看前4个词，它就无法使用更早的历史信息：

```
..... 很久以前的信息 | 最近4个词 → 预测下一个词
前面的内容被忽略
```

因此后来引入RNN，让模型可以持续积累历史信息。

四、CNN与TextCNN：提取局部短语特征

CNN的核心是：

> 使用卷积核在文本上滑动，寻找局部模式。

1.1 输入

一句话中的每个词先转成词向量。

假设：

- 句子有 (n) 个词；
- 每个词向量有 (k) 维。

输入就是：

[

X\in \mathbb{R}^{n\times k}

]

每一行对应一个词。

例如：

```
```

```
我       [0.2, 0.1, 0.5, ...]
非常     [0.7, 0.2, 0.3, ...]
喜欢     [0.4, 0.8, 0.1, ...]
这部     [0.3, 0.5, 0.6, ...]
电影     [0.9, 0.1, 0.2, ...]
```

1.2 网络结构

典型TextCNN：

代码块

```
Token ID
→ Embedding
→ 1D卷积
→ 激活函数ReLU
→ 最大池化或平均池化
→ 拼接所有特征
→ 全连接层
→ Softmax分类
```

PPT第28～33页介绍的就是：词向量矩阵经过1D卷积和池化得到句向量，再由全连接层输出分类结果。

1.3 卷积的处理过程

NLP中的卷积核大小通常是：

[

h\times k

]

- (k)必须等于词向量维度；
- (h)表示一次查看几个连续词。

例如：

代码块

```
高度h=2：提取2-gram特征
高度h=3：提取3-gram特征
高度h=4：提取4-gram特征
```

卷积核依次查看：

```
我 非常 喜欢
非常 喜欢 这部
喜欢 这部 电影
```

每经过一个窗口，就输出一个特征值，所有位置组成一张特征图。

多个卷积核可以提取不同模式：

```
卷积核1：可能学习否定词模式
卷积核2：可能学习情感短语
卷积核3：可能学习程度副词
```

1.4 池化的作用

卷积之后会产生很多位置特征。

最大池化：

```
[0.2, 0.9, 0.3, 0.5] → 0.9
```

相当于保留：

> 这个卷积核在整句话中检测到的最强特征。

平均池化则对所有位置求平均。

1.5 输出

文本分类中通常输出：

```
全连接层
→ logits
→ Softmax
→ 每个类别的概率
```

例如：

```
正面：0.87
负面：0.13
```

1.6 特点和作用

优势：

- 善于识别局部关键词、短语和n-gram模式；
- 卷积位置可以并行计算，速度快；
- 参数共享，参数量相对较少；
- 很适合文本分类。

不足：

- 卷积窗口主要观察局部内容；
- 不容易直接捕捉距离很远的词之间的关系；
- 池化可能损失具体位置和词序信息。

适合：

代码块

```
情感分类、新闻分类、垃圾邮件分类、意图识别
```

五、RNN：按顺序读取文本并积累历史信息

RNN的核心是：

> 当前时刻不仅使用当前词，还使用上一个时刻的隐藏状态。

1.1 输入

输入是按顺序排列的词向量：

代码块

```
x1, x2, x3, ..., xn
```

例如：

代码块

```
我 → 喜欢 → 自然 → 语言 → 处理
```

1.2 网络结构

代码块

```
x1 → RNN单元 → h1
                ↓
x2 → RNN单元 → h2
                ↓
x3 → RNN单元 → h3
```

每一步可以简单表示为：

[

h_t=\tanh(Ux_t+Wh_{t-1}+b)

]

其中：

- (x_t)：当前词向量；
- (h_{t-1})：读完前面内容后的记忆；
- (h_t)：加入当前词后形成的新记忆。

1.3 处理过程

读句子：

代码块

```
我 喜欢 自然 语言 处理
```

模型依次进行：

代码块

```
h1 = 处理“我”
h2 = 处理“喜欢” + h1
h3 = 处理“自然” + h2
h4 = 处理“语言” + h3
h5 = 处理“处理” + h4
```

因此最后的 (h_5) 理论上包含整个句子的历史信息。

PPT第5章将RNN表示为“当前词向量+上一时刻隐藏状态→当前隐藏状态”，用来预测下一个词。

1.4 RNN的三种典型输出结构

（1）多对一：文本分类

```
代码块
1    多个输入词 → 一个类别
```

处理过程：

```
代码块
1    Embedding
2    → RNN依次读取所有词
3    → 取最后隐藏状态hn
4    → 全连接层
5    → 分类结果
```

例如：

```
代码块
1    输入：这部电影非常精彩
2    输出：正面
```

也可以把所有时刻的隐藏状态做平均，而不是只使用最后状态。

（2）多对多且长度相同：序列标注

每个输入位置都有一个输出：

```
代码块
1    张三     在     北京     工作
2    B-PER    O     B-LOC     O
```

处理过程：

```
每个词进入RNN
→ 每个位置产生隐藏状态ht
→ 每个ht经过同一个分类器
→ 输出对应位置的标签
```

适合：

- 分词；
- 词性标注；
- 命名实体识别；
- 语音帧分类。

PPT第42～43页把这种结构称为N-N结构，输入输出长度相同。

（3）多对多且长度不同：Seq2Seq

例如：

```
输入：I love NLP
输出：我 爱 自然语言处理
```

这时需要Encoder-Decoder，后面单独介绍。

1.5 RNN的特点

优势：

- 适合序列数据；
- 能利用历史信息；
- 输入长度可以变化；
- 适合语言模型和序列标注。

不足：

- 必须按照时间顺序计算，不能完全并行；
- 很早以前的信息需要经过多步才能传到后面；
- 训练时容易发生梯度消失或梯度爆炸；

- 长距离依赖能力较弱。

六、LSTM：具有选择性记忆能力的RNN

LSTM是RNN的改进版本，不是完全不同的模型。

普通RNN只有隐藏状态 (h_t)，LSTM增加了细胞状态 (C_t)，并通过门结构控制信息。

1.1 输入和输出

每个时间步输入：

```
代码块
1    当前词向量xt
2    上一时刻隐藏状态h(t-1)
3    上一时刻细胞状态C(t-1)
```

输出：

```
代码块
1    当前隐藏状态ht
2    当前细胞状态Ct
```

1.2 LSTM单元结构

LSTM主要包括：

```
代码块
1    遗忘门
2    输入门
3    候选记忆
4    细胞状态更新
5    输出门
```

遗忘门

决定旧记忆保留多少：

```
代码块
```

```
哪些历史信息仍然有用?
```

输出接近1表示保留，接近0表示遗忘。

输入门

决定当前信息写入多少：

代码块

```
当前读到的内容有多少值得记住?
```

细胞状态

融合旧记忆和新信息：

代码块

```
新细胞状态
= 保留下来的旧记忆
+ 写入的新记忆
```

输出门

决定当前从细胞状态中输出多少，形成隐藏状态 (h_t)。

PPT第5章第43～48页说明，LSTM通过细胞状态和门控制记忆，缓解普通RNN的梯度消失问题。

1.3 处理过程

例如：

代码块

```
小明出生在北京，他后来去上海工作。
```

处理“他”时，LSTM可能：

- 保留“小明”这一人物信息；
- 保留“出生在北京”的关系；
- 根据后文逐步更新地点信息；
- 遗忘无关的局部词语。

所以LSTM不是无条件记住所有内容，而是：

> 选择性地记忆、更新和遗忘。

1.4 用于文本分类

PPT中的LSTM分类过程是：

```
Token ID
→ Embedding
→ LSTM依次处理
→ 得到最后隐藏状态hn
→ 全连接层
→ 类别logits
```

也可以：

```
得到所有隐藏状态
→ 平均池化
→ 全连接分类
```

1.5 特点

优势：

- 比普通RNN更能保存长期信息；
- 缓解梯度消失；
- 适合文本分类、语言模型、序列标注、机器翻译。

不足：

- 门结构较多，参数量和计算量比RNN大；
- 仍然必须按时间顺序计算；
- 很长文本中仍可能遗失信息；
- 并行能力不如Transformer。

七、BiLSTM：同时利用左侧和右侧上下文

普通LSTM只能从左往右读取：

```
代码块
1    我 → 喜欢 → 自然 → 语言
```

BiLSTM使用两个LSTM：

```
代码块
1    正向LSTM：从左到右
2    反向LSTM：从右到左
```

每个位置的输出是：

[

h_t=[\overrightarrow{h_t};\overleftarrow{h_t}]

]

也就是正向、反向隐藏状态的拼接。

处理过程

```
代码块
1    输入词向量
2          ├→ 正向LSTM
3          └→ 反向LSTM
4    → 拼接两个方向的隐藏状态
5    → 池化或逐token分类
6    → 输出
```

PPT中的BiLSTM分类器把每个位置的双向输出做平均池化，再送入全连接层。

作用

BiLSTM特别适合理解型任务，因为一个词的含义通常同时依赖前文和后文：

```
代码块
1    苹果发布了新手机。
```

```
2   我吃了一个苹果。
```

适合：

- 命名实体识别；
- 词性标注；
- 分词；
- 文本分类。

但它不适合直接进行严格的自回归生成，因为生成时未来词还不存在。

八、Seq2Seq：使用Encoder和Decoder转换序列

Seq2Seq不是某一种固定单元，它是一种结构。

早期Seq2Seq通常由两个RNN或LSTM组成。

1.1 总结构

```
代码块
1   输入序列
2   → Encoder RNN/LSTM
3   → 语义向量c
4   → Decoder RNN/LSTM
5   → 输出序列
```

1.2 Encoder处理过程

Encoder依次读取：

```
代码块
1   I → love → NLP
```

得到：

```
代码块
1   h1, h2, h3
```

早期Seq2Seq通常把最后隐藏状态 (h_3) 作为整个输入的语义向量 (c)。

也可以对所有隐藏状态求和或求平均。PPT第44～47页说明了这两种编码方式。

1.3 Decoder处理过程

Decoder根据语义向量 (c) 逐词生成：

```
代码块
1    <START>
2    → 我
3    → 爱
4    → 自然语言处理
5    → <END>
```

每一步的输入通常包括：

- 上一步隐藏状态；
- 上一步输出词；
- 编码器语义信息。

1.4 Teacher Forcing

训练时，Decoder下一步通常不使用自己刚才预测的词，而使用标准答案中的正确词：

```
代码块
1    模型预测错了：“你”
2    训练下一步仍输入标准答案：“我”
```

这叫Teacher Forcing，可以稳定训练并提高并行程度。

1.5 问题

全部输入被压缩到固定长度向量 (c)：

```
代码块
1    很长的输入序列 → 一个固定向量
```

容易造成信息瓶颈。

而且Decoder每一步都使用相同的 (c)，不能明确知道当前生成词应该重点参考输入的哪个位置。

因此引入Attention。

九、Seq2Seq + Attention：每一步动态读取输入

注意力机制的核心是：

```
代码块
1   当前解码需要什么
2   → 在所有输入位置中计算相关性
3   → 给不同位置分配不同权重
4   → 得到当前步骤专用的context
```

1.1 Q、K、V

在RNN Seq2Seq中通常可以理解为：

- Q：Decoder当前隐藏状态；
- K：Encoder各位置隐藏状态；
- V：Encoder各位置隐藏状态。

1.2 处理过程

假设输入：

```
代码块
1   Tom chases Jerry
```

Decoder准备生成“汤姆”：

```
代码块
1   Tom       0.8
2   chases    0.1
3   Jerry     0.1
```

得到：

```
代码块
1   context1
```

```
= 0.8×Tom表示
+ 0.1×chases表示
+ 0.1×Jerry表示
```

生成“追逐”时，权重改变：

代码块

```
Tom       0.1
chases    0.7
Jerry     0.2
```

所以每一个输出时间步都有不同的context。

PPT第50～54页正是用这一例子说明Attention使Decoder可以动态关注当前最相关的输入。

1.3 作用

- 缓解固定语义向量的信息瓶颈；
- 对输入输出位置建立对应关系；
- 改善长句翻译；
- 具有一定可解释性。

但RNN Encoder和Decoder仍然是串行计算，Transformer进一步用自注意力替换了循环结构。

十、Transformer：以自注意力为核心的序列模型

Transformer的核心改变是：

> 不再依靠RNN逐词传递历史状态，而是让每个位置直接与其他位置建立联系。

原始Transformer是Encoder-Decoder结构：

代码块

```
输入文本
→ Transformer Encoder
→ 上下文表示
→ Transformer Decoder
→ 输出文本
```

它解决了RNN/LSTM的两个主要问题：

- 串行计算，训练速度慢；
- 长距离关系需要经过很多时间步传递。

十一、Transformer Encoder的完整过程

1.1 输入

```
代码块
1   原始文本
2   → Tokenization
3   → Token ID
4   → Token Embedding
5   → 加上Position Embedding
```

因为自注意力本身不知道词序，所以必须加入位置编码：

```
代码块
1   最终输入向量
2   = 词向量 + 位置向量
```

PPT第8～12页说明了词嵌入与位置嵌入相加的输入方式。

1.2 一个Encoder Block的结构

```
代码块
1   输入X
2   → 多头双向自注意力
3   → 残差连接 + LayerNorm
4   → 前馈神经网络FFN
5   → 残差连接 + LayerNorm
6   → 输出
```

多个Encoder Block继续堆叠。

1.3 自注意力

每个输入向量经过三个可学习矩阵，得到：

```
Q：Query
K：Key
V：Value
```

然后：

```
Q与所有K计算相关性
→ Softmax得到注意力权重
→ 对V加权求和
→ 得到新的上下文表示
```

核心公式是：

$$
\operatorname{Attention}(Q,K,V)
\operatorname{softmax}
\left(
\frac{QK^T}{\sqrt{d_k}}
\right)V
$$

自注意力中的Q、K、V来自同一序列。每个词可以直接融入其他词的信息。

1.4 多头注意力

使用多组不同的 (W_Q,W_K,W_V)：

```
头1：可能关注指代关系
头2：可能关注句法关系
头3：可能关注位置关系
……
```

各头结果拼接后，再做一次线性变换。

1.5 FFN

注意力负责：

> 不同token之间交换信息。

FFN负责：

> 对每个token自己的特征进行非线性加工。

结构通常是：

```
线性层
→ ReLU或GELU
→ 线性层
```

1.6 残差连接和LayerNorm

残差连接：

```
输出 = 原输入 + 子层结果
```

作用是保留原信息，并帮助梯度传播。

LayerNorm对单个样本的特征进行归一化，使训练更加稳定。

1.7 Encoder输出

输入有几个token，就输出几个上下文向量：

```
我   → C1
有   → C2
一只 → C3
猫   → C4
```

每个 (C_i) 都融合了整句话的信息。

十二、Transformer Decoder的完整过程

1.1 输入

训练时输入目标序列右移后的内容：

```
目标答案：我 爱 NLP <END>
Decoder输入：<START> 我 爱 NLP
```

并加入Embedding和位置编码。

1.2 一个Decoder Block

```
输入
→ 带因果掩码的多头自注意力
→ Add & LayerNorm
→ 多头交叉注意力
→ Add & LayerNorm
→ FFN
→ Add & LayerNorm
→ 输出
```

1.3 因果自注意力

第 (t) 个位置只能看：

```
自己和前面位置
```

不能看到后面的正确答案。

例如：

```
位置1：看1
位置2：看1、2
位置3：看1、2、3
```

这通过Causal Mask实现。

1.4 交叉注意力

- Q来自Decoder；
- K、V来自Encoder输出。

作用是：

> Decoder生成当前词时，从输入文本中寻找最相关的信息。

1.5 输出层

```
Decoder最终向量
→ Linear
→ 得到词表大小的logits
→ Softmax
→ 每个token的概率
```

然后可采用：

- 贪心搜索：每次选概率最大；
- Beam Search：保留多条最优候选；
- 随机采样；
- Top-k采样；
- Top-p采样；
- 温度调节。

这些解码策略在PPT第29～32页进行了说明。

1.6 训练与实际生成

训练时：

```
标准答案整体已知
+ Teacher Forcing
+ 因果Mask
→ 多个位置可并行训练
```

实际生成时：

```
生成第1个词
```

```
2    → 加回输入
3    → 生成第2个词
4    → 加回输入
5    → 继续生成
```

因此Transformer训练可以并行，但自回归生成仍然是逐token进行。

十三、Transformer的三大变体

1. Encoder-only

结构：

```
代码块
1    输入文本
2    → Embedding + 位置编码
3    → 多层Transformer Encoder
4    → 每个token的上下文向量
5    → 根据任务连接不同任务头
```

注意力：

```
代码块
1    双向自注意力
```

特点：

- 每个词可以同时利用左侧和右侧上下文；
- 擅长理解完整文本；
- 本身不擅长连续生成长文本。

代表：

```
代码块
1    BERT、RoBERTa
```

任务：

- 分类；
- 命名实体识别；
- 词性标注；
- 句子相似度；
- 抽取式问答。

---

2. Decoder-only

结构：

```
代码块
1  提示文本
2  → Embedding + 位置编码
3  → 多层带因果注意力的Decoder式Block
4  → Linear + Softmax
5  → 预测下一个token
6  → 将token加入输入
7  → 继续生成
```

注意力：

```
代码块
1  单向/因果自注意力
```

代表：

```
代码块
1  GPT、LLaMA、Qwen
```

PPT明确把GPT、LLaMA和Qwen归为Decoder-only模型。

作用：

- 对话；
- 文本续写；
- 文本生成；

- 代码生成；
- 开放式问答。

需要注意：

> Decoder-only虽然叫“Decoder”，但由于没有Encoder，通常也没有原始Transformer Decoder中的交叉注意力层。

---

3. Encoder-Decoder

结构：

```
输入文本
→ Encoder双向理解
→ 上下文表示
→ Decoder因果生成
→ 输出文本
```

包含：

- Encoder中的双向自注意力；
- Decoder中的因果自注意力；
- Encoder和Decoder之间的交叉注意力。

代表：

```
原始Transformer、T5、BART
```

适合：

- 机器翻译；
- 文本摘要；
- 问题生成；
- 改写；
- 输入文本到输出文本的转换。

十四、BERT：基于Transformer Encoder的预训练模型

BERT的本质：

```
代码块
1   BERT = 多层Transformer Encoder + 预训练任务
```

它不是和Transformer并列的基础网络。

3.1 输入处理

第一步Tokenization：

```
代码块
1   我喜欢NLP
2   → [CLS] 我 喜欢 NLP [SEP]
3   → 转换为Token ID
```

第二步构造三类Embedding：

```
代码块
1   Token Embedding：词是什么
2   Position Embedding：词在什么位置
3   Token Type Embedding：属于句子A还是句子B
```

三者相加后进入Encoder。

3.2 网络结构

```
代码块
1   三种Embedding相加
2   → 多层Transformer Encoder
3   → 每个token的上下文向量
```

BERT Base：

```
代码块
1   12层Encoder
2   隐藏维度768
```

```
3    12个注意力头
```

BERT Large：

```
代码块
1    24层Encoder
2    隐藏维度1024
3    16个注意力头
```

3.3 输出

BERT会为每个输入token输出一个上下文向量。

`[CLS]` 输出

通常用来表示整段文本：

```
代码块
1    [CLS]对应向量
2    → Pooler或全连接层
3    → 文本分类
```

每个token的输出

用于：

```
代码块
1    分词、NER、词性标注
```

问答输出

对每个token分别预测：

```
代码块
1    作为答案起点的概率
2    作为答案终点的概率
```

PPT第48～54页分别介绍了句子级分类、token级分类和抽取式问答。

3.4 BERT的预训练

MLM：掩码语言模型

输入：

```
代码块
1    我喜欢吃[MASK]
```

输出：

```
代码块
1    苹果、米饭……
```

模型利用被遮盖词左右两侧的信息预测它，因此学到了双向上下文。

BERT随机选择15%的token，其中：

```
代码块
1    80%替换为[MASK]
2    10%替换为随机词
3    10%保持原词
```

NSP：下一句预测

输入句子A和句子B，判断：

```
代码块
1    B是不是A的下一句
```

3.5 BERT用于实际任务

BertForSequenceClassification

```
代码块
1    文本
2    → BERT
```

```
3    → [CLS]向量
4    → 全连接层
5    → 文本类别
```

BertForTokenClassification

代码块

```
1    文本
2    → BERT
3    → 每个token的向量
4    → 每个token接分类器
5    → NER/POS标签
```

BertForQuestionAnswering

代码块

```
1    问题 + 文章
2    → BERT
3    → 每个token的上下文向量
4    → 预测答案起始位置和结束位置
```

十五、PPT中出现的其他常用相关模型

3.1 Word2Vec、GloVe

它们不是完整分类或生成模型，而是词向量方法。

代码块

```
1    词
2    → Word2Vec/GloVe
3    → 一个低维稠密向量
```

作用是让语义相近的词在向量空间中距离较近。

它们可以作为CNN、RNN、LSTM和Transformer的输入Embedding。

区别是它们通常给每个词一个固定向量：

```
“银行”的向量在不同句子中基本相同
```

而BERT输出的是上下文化向量：

```
金融银行中的“银行”
河流两岸中的“岸”
会根据上下文得到不同表示
```

3.2 RoBERTa

PPT将RoBERTa列为Encoder-only代表。

结构与BERT相似：

```
多层Transformer Encoder
→ 双向上下文表示
```

核心仍然是理解型模型。考试中通常把它和BERT归为同类即可，不需要展开其训练细节。

3.3 GPT

GPT属于Decoder-only：

```
提示文本
→ 多层因果自注意力Block
→ 预测下一个token
→ 循环生成
```

训练目标：

```
根据前文预测下一个词
```

例如：

```
床前明月
→ 预测“光”
```

特点是生成能力强，但内部只能直接使用当前位置之前的信息。

3.4 LLaMA、Qwen

在PPT的分类中，它们和GPT一样，都属于Decoder-only大语言模型：

```
单向因果自注意力
+ 自回归下一词预测
```

它们的主体处理方式与GPT同类，主要差异在模型规模、训练数据、具体位置编码和工程设计等方面。

3.5 T5

T5属于Encoder-Decoder结构。

处理过程：

```
输入任务文本
→ Encoder双向理解
→ Decoder根据Encoder信息逐词生成
→ 输出文本
```

它把很多任务统一成“文本到文本”：

```
```

```
翻译：英文 → 中文
摘要：长文本 → 摘要
分类：文本 → 类别名称
问答：问题和材料 → 答案
```

3.6 BART

BART也是Encoder-Decoder模型：

代码块

```
Encoder读取受破坏或原始文本
→ 获得上下文表示
→ Decoder恢复或生成目标文本
```

适合摘要、生成、纠错和文本重构。

在你的课程范围内，记住：

代码块

```
T5、BART
= Encoder-Decoder
= 输入输出文本转换任务
```

即可。

3.7 ViT

PPT还出现了Vision Transformer，但它主要用于计算机视觉，不是核心NLP模型。

其思想是：

代码块

```
图像
→ 切成多个Patch
→ 每个Patch转换为向量
→ 加位置编码
→ Transformer Encoder
→ 图像分类
```

它说明Transformer不仅能处理词序列，也能处理图像Patch序列。

十六、最终对比表

十七、用一句话掌握每个模型

FNN：固定看前几个词，再预测下一个词。

CNN：拿多个小窗口扫描文本，寻找重要短语。

RNN：从左到右读，每读一个词就更新一次记忆。

LSTM：在RNN基础上决定哪些信息保留、写入或遗忘。

BiLSTM：同时从左往右和从右往左理解句子。

Seq2Seq：Encoder读输入，Decoder生成输出。

Attention：Decoder每一步从所有输入信息中选择当前最重要的部分。

Transformer：让所有token通过自注意力直接交流，并行建立全局关系。

BERT：只用Transformer Encoder，双向理解文本。

GPT、LLaMA、Qwen：只用Decoder式因果结构，根据前文逐词生成。

T5、BART：Encoder先理解输入，Decoder再生成另一段文本。

复习时可以抓住这一条主线：

代码块

```
局部特征：CNN
顺序记忆：RNN
选择性长期记忆：LSTM
动态选择输入：Attention
全局并行建模：Transformer
```

```
6    双向理解：BERT
7    自回归生成：GPT
8    输入到输出转换：T5/BART
```

应用分析

可以。最适合你的方法就是：

> **把第一章当作“综合题题库目录”，把后续章节当作“实现这些应用的技术工具箱”。**

但不要给每个应用单独背一套方案。第一章看起来有二十多个应用，底层实际上可以归并为 **7类基本任务＋2类复合系统**。第一章本身也把NLP概括为语言理解与语言生成两大方向，后续章节则补充了语料库、词向量、语言模型、HMM、分词、NER、CNN、RNN、Transformer、BERT、微调和RAG等实现工具。

一、做综合题时先判断“输出是什么”

看到一道应用设计题，先不要急着写BERT、CNN，而是问下面四个问题：

1. **输入是什么？** 文本、词序列、问题、语音还是图片？
2. **输出是什么？** 一个类别、每个词一个标签、一段新文本、若干文档还是结构化三元组？
3. **依赖什么资源？** 标注语料、平行语料、词典、知识图谱还是外部文档库？
4. **任务的主要难点是什么？** 歧义、未登录词、长距离依赖、知识更新、幻觉还是跨模态匹配？

根据输出形式，基本可以快速归类：

智能交互和辅助教学不是单一任务，而是多个基本任务组成的复合系统。

二、所有“设计实现方案题”的通用答题骨架

不论题目是医院、学校、新闻平台还是客服系统，都可以按下面八步写。

3.8 任务定义

先说明任务属于什么类型，以及解决什么问题。

例如：

> 该任务可以建模为文本分类问题，目标是根据文本内容将其自动归入预定义类别。

或者：

> 该任务可以建模为序列标注问题，目标是识别文本中实体的边界及实体类型。

3.2 输入与输出

必须明确，不要只说“处理文本”。

例如：

```
代码块
1    输入：一篇中文新闻
2    输出：体育、科技、娱乐等类别之一
```

```
代码块
1    输入：电子病历字符序列
2    输出：每个字符对应的B-SYM、I-SYM或O标签
```

3.3 数据与标注

写清楚需要什么资源：

- 分类：文档—类别标注语料；
- NER：token—BIO标签语料；
- 翻译：双语平行语料；
- 语义消歧：带词义标签的语料或WordNet、HowNet；
- RAG：外部文档知识库；
- 知识图谱：实体和关系标注语料。

语料库不仅用于模型参数训练，也可作为测试和评价标准；PPT中还列出了树库、CoNLL、法律语料、口语语料和医学NER语料等不同资源。

3.4 预处理与文本表示

可根据模型选择：

```
文本清洗
→ 分句、分词或子词切分
→ 转换Token ID
→ Embedding
```

传统方法可用词袋、TF-IDF；神经模型可用Word2Vec；BERT直接使用WordPiece等子词并计算动态上下文向量。Word2Vec属于静态词向量，同一词在不同上下文中向量不变；BERT、ELMo等属于动态上下文表示。

3.5 模型结构

不能只写“采用BERT”，要写出处理路径：

```
文本
→ BERT编码
→ [CLS]句向量
→ 全连接层
→ Softmax分类
```

或者：

```
token序列
→ BERT上下文表示
→ 每个token接分类层
→ BIO标签序列
```

3.6 参数训练

神经模型通常写：

```
前向传播
→ 计算预测结果与真实标签的交叉熵损失
→ 反向传播
→ Adam或SGD更新参数
→ 验证集选择最优模型
```

HMM则写：

```
代码块由标注语料统计初始概率π、
2    状态转移概率A和发射概率B
```

3.7 预测与后处理

不同任务要写不同方式：

- 分类：取概率最大类别；
- 序列标注：Viterbi或逐token预测，再合并实体；
- 生成：逐token生成，直到结束符；
- RAG：检索Top-K资料并注入提示词；
- 知识图谱：实体标准化后生成三元组。

3.8 评价与改进

常见评价：

- 分类：准确率、P、R、F1；
- 分词和NER：P、R、F1；
- 生成：可写准确性、流畅性、完整性及人工评价；
- 检索：查准率、查全率、Top-K命中情况；
- 评分任务：分类准确率或回归误差。

最后再写一至两个难点和对应解决方法。

三、第一类：文本分类、判断与评分

包含哪些应用

可以归到这一类的有：

- 文档分类；
- 情感极性分类；
- 垃圾邮件识别；
- 智能交互中的意图识别；
- 作文等级判断或自动评分；
- 文本质量评分；
- 自动校对中的“是否存在错误”判断；

- 句子关系或相似性判断。

其共同形式是：

```
一段文本或一对文本
→ 一个类别、多个标签或一个分数
```

BERT在PPT中对应的模型是 `BertForSequenceClassification`：输入单句或句子对，使用 `[CLS]` 表示接分类层，可用于文本分类、意图识别和文本质量评分。

可用模型

传统路线

```
文本
→ 分词
→ 词袋或TF-IDF
→ 朴素贝叶斯/SVM/逻辑回归
→ 类别
```

特点：简单、可解释、计算快；缺点是高维稀疏、忽略词序和深层语义。

TextCNN路线

```
文本
→ 词向量矩阵
→ 多尺寸1D卷积
→ 最大池化
→ 全连接层
→ Softmax
```

适合依靠关键词和局部短语判断类别。

LSTM路线

```
文本
→ Embedding
```

```
→ LSTM或BiLSTM
→ 最后隐藏状态或平均池化
→ 全连接层
→ Softmax
```

适合考虑词序和上下文。PPT给出了LSTM末态池化和BiLSTM平均池化的文本分类结构。

BERT路线

代码块

```
[CLS] 文本 [SEP]
→ BERT Encoder
→ [CLS]向量
→ Dropout
→ 全连接层
→ Softmax或回归层
```

完整答题模板

> 该任务可以建模为文本分类或回归问题。输入为_____，输出为_____类别或分数。首先收集带标签的领域文本，经过清洗、分词或子词切分后划分为训练集、验证集和测试集。可采用BERT模型，将文本首尾加入 `[CLS]` 和 `[SEP]` ，经多层Transformer Encoder获得上下文表示，取 `[CLS]` 向量输入全连接分类层，输出各类别概率；若输出连续分数，则使用回归层。训练时采用交叉熵损失或均方误差，通过反向传播更新参数。预测时选择概率最大的类别或直接输出分数，并使用准确率、精确率、召回率、F1或回归误差评价系统。

不同小点需要多写什么

文档分类

要多写：

- 分类标准，如主题、内容或功能；
- 二分类、多分类还是多标签分类；
- 类别不平衡问题。

情感及观点分析

简单题可以写正面、负面、中性分类；复杂题还要写：

- 情感持有者是谁；
- 情感针对的对象是什么；

- 具体观点是什么；
- 情感强度和否定、反讽现象。

第一章明确指出情感分析不仅判断正负面，还要识别观点、观点主体及针对对象。

作文自动评分

要先说明输出形式：

- 分数区间：回归；
- 优秀、良好、及格：分类。

还可以补充内容、语法、结构、连贯性和词汇丰富度等特征。

句子对分类

输入通常为：

```
代码块
1   [CLS] 句子A [SEP] 句子B [SEP]
```

输出可以是：

- 是否语义相似；
- 是否存在上下文关系；
- 是否矛盾；
- 是否为同一意图。

四、第二类：分词、词性标注、NER和其他序列标注

包含哪些应用

- 中文分词；
- 词性标注；
- 人名、地名、机构名识别；
- 医疗症状、疾病、药物和检查识别；
- 关键词抽取；
- 自动校对中的错误位置检测；
- 智能交互中的槽位抽取；

- 知识图谱构建中的实体识别。

共同形式：

```
输入：长度为n的字或token序列
输出：长度也为n的标签序列
```

PPT把NER统一转化为BIO序列标注，并指出相同方法还可以识别医疗领域的部位、疾病、症状、药物、检查和手术。

标签设计

分词：BMES

- B：词首；
- M：词中；
- E：词尾；
- S：单字词。

NER：BIO

- B-PER：人名开始；
- I-PER：人名内部；
- B-LOC、I-LOC：地名；
- B-ORG、I-ORG：机构名；
- O：非实体。

医学症状识别可使用：

```
B-SYM、I-SYM、O
```

可用模型一：HMM

```
观测序列：字或词
隐藏状态：BMES、BIO或词性标签
```

模型参数：

- 初始状态概率 (\pi)；
- 状态转移概率 (A)；
- 发射概率 (B)。

训练时从标注语料中统计这些概率；预测时使用Viterbi算法寻找最可能的完整标签路径。HMM与CRF在课程中都被归为可执行序列标注任务的模型。

可用模型二：BERT Token Classification

代码块

```
token序列
→ BERT双向编码
→ 每个token的上下文向量
→ 每个位置接全连接层
→ 标签概率
```

BERT的token级模型可用于NER、词性标注、关键词抽取和文本标注。

还可以在后面增加CRF：

代码块

```
BERT
→ 每个token的标签得分
→ CRF学习标签转移约束
→ 最优标签序列
```

完整答题模板

> 该任务可以建模为序列标注问题。输入为_____序列，输出为每个位置对应的_____标签。首先制定BIO或BMES标注体系，收集并人工标注领域语料。对文本进行token切分和数字编码后，可采用BERT获得每个token的上下文表示，再接全连接层预测各位置标签；为保证标签序列合法，可增加CRF层。训练时采用token级交叉熵或CRF损失，通过反向传播学习参数。预测时解码得到最优标签序列，并将连续的B、I标签合并为完整实体。系统使用实体级精确率、召回率和F1进行评价。

不同小点需要多写什么

中文分词

必须写两个特殊难点：

- 切分歧义；

- 未登录词。

可写的方法：

1. 正向、逆向或双向最大匹配；
2. 最短路径；
3. 基于n-gram语言模型选择概率最大的切分；
4. 基于BMES的HMM或神经序列标注。

大模型使用子词切分，如GPT的BPE、BERT的WordPiece和LLaMA的Unigram，可缓解整词词表中的OOV问题。

词性标注

需要说明：

- 通常先完成分词；
- 再为每个词标注名词、动词、形容词等词类；
- 词性信息可以辅助情感分析和句子理解。

人名、地名识别

除统一BIO方法外，可补充规则和词典法：

```
姓氏库/地名库
→ 产生候选
→ 名字用字或位置规则
→ 概率和规则筛选
```

地名还可能存在简称、普通词冲突、长度不固定等问题，因此可加入地名首字、尾字、中间字库及确认、否定规则。

医疗症状识别

要多写：

- 领域标签体系；
- 医学专家标注；
- 实体边界和实体类型；
- 否定状态。

例如“无发热”中，“发热”仍是症状实体，只是状态为否定。因此可在NER之后再增加“存在、否定、不确定”的属性判断。

自动校对

要区分两步：

```
错误检测：找出哪个位置有错
错误纠正：给出正确文字
```

错误检测可用token分类；错误纠正可用BERT掩码预测或Encoder-Decoder生成模型。

五、第三类：语义消歧、语义匹配和抽取式理解

包含哪些应用

- 词义消歧；
- 语义相似度计算；
- 句子对关系判断；
- 查询与文档相关性判断；
- 抽取式问答；
- 语义解析中的意图识别和槽位提取。

这一类的共同点是：

> 输入已经有一段上下文，系统需要判断其中的含义、关系或答案位置。

3.1 词义消歧

有监督方案

数据形式：

```
句子＋目标多义词＋正确义项
```

提取特征：

- 目标词周围的词；
- 词性；
- 相对位置；

- 局部搭配；
- BERT目标词上下文向量。

传统贝叶斯方案：

```
统计各义项的先验概率
→ 统计上下文词在各义项下的条件概率
→ 计算每个义项的后验概率
→ 选择概率最大义项
```

PPT以“打”字的24个义项为例，使用上下文词和最大似然估计完成有监督消歧。

BERT方案：

```
上下文句子
→ BERT
→ 取目标词位置向量
→ 全连接层
→ 各义项概率
```

词典或知识库方案

```
取得目标词的所有候选义项
→ 获取定义、同义词、例句和语义关系
→ 与当前上下文计算词汇重叠或语义相似度
→ 选择最匹配义项
```

WordNet按照词义组织同义词集合，并包含同义、反义、上下位和部分—整体关系，可用于词汇消歧、语义推理和语义相似度计算；HowNet还描述施事、受事、工具、场所和时间等概念关系。

词义消歧答题模板

> 语义消歧的任务是根据目标多义词所在的上下文，从候选义项中确定当前使用的正确义项。有监督方法需要收集并标注目标词不同义项的语料，提取上下文词、词性、位置或BERT上下文向量，训练贝叶斯分类器或神经分类器，预测时选择概率最大的义项。基于词典的方法从WordNet或HowNet中取得候选义项、定义、同义词和语义关系，再计算当前上下文与各义项描述的重叠度或语义相似度，选择得分最高的义项。

3.2 语义相似度与句子对判断

```
[CLS] 文本A [SEP] 文本B [SEP]
→ BERT
→ [CLS]
→ 分类或相似度得分
```

要多写：

- 判断的是同义、相关、矛盾还是是否回答问题；
- 是二分类还是连续相似度回归；
- 可用于语义检索和重排序。

3.3 抽取式问答

输入：

```
问题＋上下文文章
```

输出：

```
答案在文章中的起始token和结束token
```

BERT使用两个输出头分别预测答案起点和终点。

注意：抽取式问答不是自由生成，它只能从给定文本中截取答案。

3.4 语义解析

可分为两种常见形式：

意图＋槽位

```
输入：帮我订明天去北京的机票
输出：
意图＝订机票
```

```
日期＝明天
目的地＝北京
```

实现：

代码块

```
BERT共享编码
├→ [CLS]做意图分类
└→ 每个token做槽位BIO标注
```

逻辑形式或结构化输出

代码块

```
自然语言问题
→ 实体识别、关系判断
→ 结构化查询或逻辑表达
```

例如GraphRAG中，可以把自然语言问题转换成图数据库的Cypher查询，再检索子图并生成答案。

六、第四类：机器翻译、摘要、润色、对话和文本生成

这些任务的共同形式是：

代码块

```
输入序列
→ 另一段输出序列
```

通用技术路线

早期Seq2Seq

代码块

```
输入
→ RNN/LSTM Encoder
→ 语义向量c
→ RNN/LSTM Decoder
```

```
5    → 输出
```

固定语义向量容易丢失长句信息，因此引入Attention，让Decoder每一步动态关注不同的输入位置。PPT给出了RNN Encoder-Decoder、Teacher Forcing及Attention改进过程。

Transformer Encoder-Decoder

代码块

```
1    输入token
2    → Embedding＋位置编码
3    → Encoder双向自注意力
4    → 输入上下文表示
5    → Decoder因果自注意力
6    → 交叉注意力读取Encoder信息
7    → Linear＋Softmax
8    → 输出token
```

Encoder-Decoder适合输入和输出严格对应的任务，如翻译、摘要和语音识别；Decoder-only模型更适合对话、续写和自由文本生成。

通用答题模板

> 该任务可以建模为序列到序列生成问题。输入为_____序列，输出为_____序列。首先收集输入—输出配对语料，进行Tokenization并构建训练集。模型可采用Transformer Encoder-Decoder：Encoder通过双向自注意力得到输入的上下文表示，Decoder使用因果自注意力建模已生成内容，并通过交叉注意力读取输入信息，最后通过Linear和Softmax预测下一个token。训练时使用Teacher Forcing和token级交叉熵损失，预测时从开始标记出发，采用贪心搜索或束搜索逐步生成，直到结束标记。评价时考察内容正确性、完整性、流畅性和任务特定指标。

不同应用需要多写什么

机器翻译

数据必须是双语平行语料，可以有句子或结构对齐。

要多写：

- 原语言分析和理解；
- 目标语言生成；
- 语序差异；
- 一词多义；

- 习语和文化表达；
- 准确、自然和语义完整。

自动文摘

先区分：

抽取式摘要

```
原文句子
→ 重要性评分
→ 选择若干重要句
→ 按原顺序组成摘要
```

本质更接近分类或排序。

生成式摘要

```
原文
→ Encoder理解
→ Decoder重新生成简短摘要
```

要多写：

- 保留核心信息；
- 控制长度；
- 避免重复；
- 避免生成原文没有的事实。

作文润色与自动校对

先检测错误，再修改：

```
错误检测
→ 错误类型判断
→ 候选修改
→ 生成润色文本
```

要强调修改应保留原意，避免“改得更通顺但事实变了”。

对话系统

开放域生成还要考虑：

- 多轮上下文；
- 人设和风格；
- 知识依据；
- 安全性；
- 幻觉和重复。

任务型对话则要加入意图识别、槽位抽取和对话状态管理。

文本自动生成

Decoder-only流程：

```
代码块
1  提示文本
2  → 因果自注意力
3  → 预测下一个token
4  → 将结果加入输入
5  → 循环生成
```

需要多写：

- 主题和风格控制；
- 连贯性；
- 事实准确性；
- 有害和不当内容控制。

七、第五类：信息检索、跨语言检索、多模态检索和RAG

第一章的信息检索发展路线可以直接变成一道完整综合题：

```
代码块
1  字面匹配
2  → 浅层语义检索
3  → 检索增强生成
```

3.1 字面匹配

```
用户查询
→ 分词
→ 倒排索引
→ TF-IDF或BM25
→ 返回相关文档
```

优点：精确关键词匹配好、速度快、可解释。

缺点：存在词汇鸿沟，例如“手机”和“移动电话”字面不同。

3.2 语义检索

```
查询和文档
→ 同一Embedding模型
→ 转换成稠密向量
→ 余弦相似度
→ 返回Top-K文档
```

可采用BERT类嵌入模型。

优点：可匹配同义表达。

缺点：人名、型号和精确术语可能不如关键词检索稳定。

3.3 混合检索

```
BM25关键词检索
+
向量语义检索
→ 结果融合
→ 重排序
```

RAG PPT指出，向量检索适合近义匹配，BM25适合精确关键词匹配，还可以使用Cross-Encoder对候选文档重排序。

3.4 RAG

离线阶段

代码块

```
文档加载
→ 清洗
→ 文档分块
→ Embedding
→ 存入向量数据库
→ 建立索引
```

在线阶段

代码块

```
用户问题
→ 问题向量化
→ 检索Top-K文档块
→ 重排序
→ 将资料和问题拼接成Prompt
→ 大模型生成答案
```

PPT中的完整RAG实现包括文档分块、嵌入、Chroma向量库、Top-K检索及基于资料生成答案。

检索类完整模板

> 首先建立领域文档库，对文档进行清洗、分块和索引。对于精确关键词查询，可采用BM25；对于同义词和语义相近的查询，可使用BERT类嵌入模型将查询和文档映射到统一向量空间，并通过余弦相似度检索Top-K结果。为兼顾关键词和语义，可使用混合检索，并用Cross-Encoder对候选结果重新排序。若需要直接回答问题，可将检索结果与用户问题共同输入大语言模型构成RAG系统，并要求模型只依据检索资料回答。最后评价检索准确性、召回情况及生成答案的正确性。

不同小点需要多写什么

跨语言检索

输入语言和文档语言不同，可以：

1. 先把查询翻译成文档语言再检索；
2. 使用多语言Embedding，将不同语言映射到同一语义空间。

要强调翻译错误和跨语言语义对齐问题。

多模态检索

```
文本编码器 → 文本向量
图像编码器 → 图像向量
语音编码器 → 语音向量
→ 映射到统一空间
→ 计算跨模态相似度
```

例如文本搜图、图片搜文本。

RAG

要额外写：

- 文档分块大小和重叠；
- Top-K；
- 混合检索和重排序；
- 引用来源；
- 资料没有答案时不要编造。

GraphRAG

普通RAG检索独立文本块，GraphRAG检索相关子图，更适合：

- 多跳推理；
- 跨文档关系；
- 全局信息综合。

其流程为查询转换、图遍历、子图检索、信息整合和回答生成。

八、第六类：信息抽取、知识图谱与结构化知识

包含哪些应用

- 实体识别；
- 关系抽取；
- 事件抽取；
- 实体链接和消歧；

- 知识图谱构建；
- 知识图谱问答；
- 语义解析。

知识图谱构建流程

```
原始文本
→ 分词和Tokenization
→ 命名实体识别
→ 实体标准化与实体链接
→ 关系抽取
→ 构造三元组
→ 存入图数据库
```

例如：

```
姚明生于上海
→（姚明，出生地，上海）
```

知识图谱以实体、属性和关系组织知识，可以辅助检索、问答、多跳推理，并为大模型提供准确事实。

完整答题模板

> 系统首先收集领域文本并制定实体类型和关系类型体系。利用BERT Token Classification或BERT-CRF识别人名、地点、机构等实体及其边界，再通过实体消歧和实体链接，将不同表达归一到知识库中的同一实体。随后把候选实体对及其上下文输入关系分类模型，判断实体之间的关系，形成“头实体—关系—尾实体”三元组，并存入图数据库。使用时可将自然语言问题转换为图查询，通过图遍历获得事实，再生成最终答案。系统应评价实体识别F1、关系抽取F1和三元组准确率。

需要特别补充

实体消歧

同名实体需要根据上下文和知识库属性判断具体指向，例如“苹果”可能是水果或公司。

实体链接

NER只知道文本中有一个实体，实体链接还要把它对应到知识库中的标准节点。

多跳推理

例如“A的妻子的哥哥的儿子的老师是谁”，需要沿知识图谱多条关系连续查询。

九、第七类：OCR、语音识别、文语转换和口语评测

这类题不是纯文本，但可以使用相同的“输入—编码—模型—输出—评价”思路。

3.1 文字识别OCR

代码块

```
图片
→ 图像预处理
→ 文本区域检测
→ 字符或文本行识别
→ 语言模型校正
→ 电子文本
```

要多写：

- 印刷体与手写体差异；
- 模糊、倾斜、复杂背景；
- 利用语言模型修正不合理文字组合。

3.2 语音识别ASR

代码块

```
语音信号
→ 去噪和分帧
→ 声学特征
→ 声学/神经网络编码
→ 文字候选序列
→ 语言模型重排序
→ 最终文本
```

难点：

- 同音词和近音词；
- 方言和口音；
- 环境噪声；

- 说话速度。

第一章列出的“贸易摩擦升级”误识别成“贸易摩擦生机”，就体现了声音相近但语言上下文不合理的问题。

3.3 文语转换TTS

```
输入文本
→ 文本规范化
→ 分词、音素和韵律预测
→ 声学特征生成
→ 声码器
→ 语音波形
```

要多写：

- 多音字；
- 断句；
- 重音、语调和情感；
- 语音自然度。

3.4 口语发音与流利度评估

```
学生语音
→ ASR和音素对齐
→ 与标准发音比较
→ 提取准确度、语速、停顿、重音和韵律特征
→ 分类或回归评分
→ 给出反馈
```

评价输出可包括：

- 发音准确率；
- 流利度分数；
- 完整度；
- 错误音素；
- 改进建议。

3.5 多模态系统

需说明不同模态分别编码，再进行融合：

```
文本编码器
图像编码器
语音编码器
→ 特征融合或统一向量空间
→ 分类、检索或生成
```

十、智能交互系统：典型复合题

智能交互系统不能只写“使用大语言模型”，完整结构应为：

```
用户语音或文字
→ ASR（语音输入时）
→ 意图识别
→ 实体/槽位抽取
→ 对话状态管理
→ 数据库、知识图谱或RAG检索
→ 回复生成
→ TTS（语音输出时）
```

完整模板

> 系统首先接收用户的文本或语音输入；若为语音，则通过ASR转换成文本。随后使用BERT序列分类模型识别用户意图，并使用BERT Token Classification提取时间、地点、人物、商品等槽位信息。系统根据当前意图、槽位和历史对话维护对话状态，再访问业务数据库、知识图谱或RAG知识库获取事实。最后使用模板或生成式大语言模型生成回复；若需要语音交互，再通过TTS输出。系统分别评价意图分类准确率、槽位识别F1、检索正确率和最终任务完成率。

特殊差异

任务型客服要重视：

- 意图；
- 槽位；
- 业务流程；

- 任务完成率。

开放域聊天要重视：

- 多轮上下文；
- 回复连贯性；
- 知识准确性；
- 安全和幻觉。

十一、辅助教学系统：另一个典型复合题

辅助教学可分成几个子模块：

```
学生输入
├→ 语音：ASR和口语评测
├→ 作文：评分、错误检测和润色
├→ 试题答案：知识点分类和正确性判断
└→ 教材问题：RAG问答
```

完整流程

```
学生语音、作文或问题
→ ASR/OCR或文本预处理
→ 分类、评分、序列标注或检索
→ 识别薄弱知识点和具体错误
→ 大模型生成解释和修改建议
→ 推荐针对性练习
```

作文评分与润色一定要分开

作文评分：

```
作文
→ BERT编码
→ [CLS]
→ 分类或回归层
```

```
5    → 分数
```

作文润色：

```
代码块
1    原作文
2    → Encoder-Decoder或生成式大模型
3    → 修改后的文本
```

前者是理解与判断，后者是生成。

十二、其他章节中容易出应用题的知识点

除了第一章应用场景，PPT中还出现了下面这些可能转化为综合题的应用。

3.1 语言模型

可用于：

- 判断句子是否合理；
- 预测下一个词；
- 文本生成；
- 拼音到汉字转换；
- ASR、OCR候选重排序；
- 机器翻译解码。

n-gram需要考虑数据稀疏和平滑；神经语言模型可以利用词向量和上下文表示。

3.2 Word2Vec、CBOW和Skip-gram

它们主要是**表示学习工具**，不是最终分类器。

- CBOW：根据上下文预测中心词；
- Skip-gram：根据中心词预测上下文；
- 训练结束后使用词向量作为CNN、RNN等模型输入。

3.3 HMM和Viterbi

适合：

- 分词；
- 词性标注；
- NER；
- 其他序列状态推断。

综合题中写清：

```
状态是什么
观测是什么
π、A、B如何训练
Viterbi如何预测
```

3.4 BERT下游应用

PPT集中给出了：

- 序列分类：文本分类、意图识别、质量评分；
- token分类：NER、POS、关键词抽取；
- 抽取式问答：预测答案起止位置；
- MLM：领域继续预训练；
- 句子关系判断。

3.5 微调

当通用模型不适合具体领域时：

```
预训练模型
→ 使用少量领域标注数据
→ 全参数微调或LoRA
→ 获得领域模型
```

分类、翻译、抽取、医疗和法律任务都可以采用任务微调或领域适应微调。

3.6 量化

量化不是任务模型，而是部署优化手段。

当题目出现：

- 手机端；

- 嵌入式设备；
- 显存有限；
- 推理速度和模型体积要求；

可以补充：

```
使用INT8或INT4量化
→ 减少显存和模型体积
→ 提升端侧部署能力
→ 注意量化误差和精度下降
```

十三、系统难点与解决方法的通用加分模板

十四、考试时可以直接套的完整总模板

> 本任务属于_____任务，目标是_____。系统输入为_____，输出为_____。首先收集_____语料或知识资源，制定_____标注规范，并划分训练集、验证集和测试集。随后对输入进行清洗、分词或子词切分，并采用_____获得向量表示。模型选择_____，其处理流程为“输入→→→输出”，选择该模型的原因是_____。训练时采用_____损失函数，通过最大似然估计、反向传播或微调学习参数；预测时采用_____得到最终结果，并进行_____后处理。系统使用_____指标进行评价。针对_____难点，可采用_____进行改进。

但考试中不要把空格机械填完，要先根据任务类型替换关键部分：

```
```

```
整段文本一个答案 → 分类
每个token一个答案 → 序列标注
输入后生成另一段文本 → Seq2Seq
用户查询后找资料 → 检索/RAG
文本转实体关系 → 信息抽取/知识图谱
多个环节共同完成 → 复合系统
```

十五、最终复习结构

你可以按下面的目录整理综合题，而不用按第一章二十个应用逐个背：

代码块

```
1. 文本分类与评分
   文档分类、情感、意图、作文评分

2. 序列标注与抽取
   分词、POS、NER、症状、错误定位

3. 语义判断
   语义消歧、文本相似度、句子关系、抽取问答

4. 序列生成
   翻译、摘要、润色、对话、自动生成

5. 检索与问答
   关键词检索、语义检索、跨语言、多模态、RAG

6. 知识抽取与知识图谱
   实体、关系、三元组、图问答、GraphRAG

7. 语音图像和多模态
   OCR、ASR、TTS、口语评测

8. 复合应用
   智能交互、辅助教学
```

最核心的解题原则是：

> **先把应用拆成基本任务，再写清输入输出、数据、表示、模型、训练、预测、评价和难点；不要从“我会哪些模型”开始堆名词。**

一、这类综合题真正考什么

这类题不是考你把某一页PPT背下来，而是考你能否把零散知识串成一个完整系统：

```
任务定义
→ 明确输入与输出
→ 准备语料或知识库
→ 文本预处理与表示
→ 选择合适模型
→ 训练模型参数
→ 预测与后处理
→ 评价与改进
```

PPT中其实给出了三条基本技术路线：

- **规则方法**：知识库、词典、语言规则和推理算法；
- **统计机器学习方法**：标注语料、人工特征、概率模型；
- **神经网络方法**：文本向量、神经网络、端到端训练。

考试设计题一般不要求把三种方法全部写一遍。最好是：

> **先确定题目属于哪种输出形式，再选择一条主要方案写完整，最后补一句可采用的新模型改进。**

二、先根据“输出是什么”判断题型

你首先要问自己：

> **这道题是给整段文本一个答案，还是给每个词一个答案，还是生成一段新文本？**

模型基本就能确定了。

三、系统实现方案的通用答题模板

遇到“请设计一个……系统”，可以按照下面九点组织答案。

3.7 任务定义

说明系统要解决什么问题。

```
本任务的目标是……
```

例如：

- 文本分类：把文档分到预先定义的类别；
- NER：识别实体边界并判断实体类型；
- 语义消歧：根据上下文确定多义词的具体义项；
- 机器翻译：将源语言序列转换成目标语言序列。

3.2 输入与输出

一定要具体。

```
输入：中文新闻文本
输出：科技、体育、娱乐等类别之一
```

或者：

```
输入：电子病历文本
输出：每个字符的B-SYM、I-SYM、O标签，并恢复出症状实体
```

3.3 数据及标注体系

说明数据从哪里来、怎么标。

```
收集领域语料
→ 人工清洗
→ 制定标注规范
```

```
4    → 划分训练集、验证集和测试集
```

分类任务标文档类别；序列标注任务标每个字或token；翻译任务需要平行语料；语义消歧需要带词义标签的语料或词汇知识库。语料库既是模型参数训练的数据来源，也是系统评价的标准。

3.4 文本预处理

根据题目选择：

```
代码块
1    去除噪声、统一编码、分句、分词或Tokenization、
2    构建词表、转换Token ID、补齐或截断
```

传统中文模型可能需要先分词；BERT等模型可直接使用WordPiece等子词切分。

3.5 文本表示

根据模型确定：

- 传统分类：词袋、词频、TF-IDF；
- 神经网络：Word2Vec或可训练Embedding；
- BERT：Token Embedding＋Position Embedding＋上下文编码；
- RAG：查询和文档都转换成向量。

3.6 模型结构

不能只写“使用BERT”，要说明输入如何经过模型变成输出：

```
代码块
1    Token ID
2    → BERT编码
3    → 取得[CLS]向量
4    → 全连接层
5    → Softmax类别概率
```

或者：

```
代码块
1    每个token
2    → BERT上下文表示
3    → 每个位置接分类层
```

```
4    → 得到BIO标签概率
```

3.7 参数训练

通常写：

```
代码块
1    在训练集上前向传播
2    → 计算预测值与真实标签之间的交叉熵损失
3    → 反向传播
4    → 使用SGD或Adam更新参数
5    → 根据验证集选择模型
```

HMM则要写初始概率、转移概率和发射概率的统计估计。

3.8 预测和后处理

例如：

- 分类：选择概率最大的类别；
- 序列标注：Viterbi或逐token取最大概率，再合并BIO标签；
- 生成：逐token生成，直到结束符；
- WSD：选择得分最大的义项；
- RAG：检索Top-K文档后生成答案。

3.9 评价和改进

分类、NER和分词常用：

- 准确率；
- 精确率 (P)；
- 召回率 (R)；
- (F_1)。

最后写一两个问题及解决方法：

```
代码块
1    数据不足 → 数据增强或使用预训练模型
2    类别不平衡 → 重采样或类别加权
3    未登录词 → 字符/子词模型
4    长距离依赖 → LSTM、Transformer
```

```
领域差异 → 领域微调
```

四、可以直接套用的总模板

> 本任务可以建模为_____任务，输入为_____，输出为_____。首先收集并清洗_____领域语料，按照_____标签体系进行人工标注，并划分训练集、验证集和测试集。然后对文本进行_____预处理，并采用_____方法获得文本表示。模型选择_____，其结构为“→→”，**选择该模型的原因是**。训练时以_____为损失函数，通过反向传播或最大似然估计学习参数；预测时采用_____得到最终结果。系统使用_____指标进行评价。针对_____问题，可以通过_____进一步改进。

这个模板基本能覆盖大多数“设计系统”题。

五、题型一：文本分类、情感分析和意图识别

3.1 任务本质

都是：

```
一段文本 → 一个类别
```

差别只在类别含义：

- 新闻分类：体育、科技、娱乐；
- 情感分析：正面、负面、中性；
- 意图识别：查询天气、预订机票、取消订单；
- 垃圾邮件识别：垃圾、正常。

文本分类还可以分为：

- 二分类；
- 多分类：从多个互斥类别选一个；
- 多标签分类：一篇文本可属于多个类别。

PPT中BERT的序列分类方法就是使用 `[CLS]` 向量连接全连接分类头，输出文本类别概率。

3.2 可以写出的模型

传统方案

```
文本
→ 分词和清洗
→ 词袋或TF-IDF向量
→ 朴素贝叶斯/SVM
→ 类别
```

优点：简单、速度快、可解释。

缺点：特征稀疏，忽略词序和深层语义。

TextCNN方案

```
文本
→ 词向量矩阵
→ 多尺寸一维卷积
→ 提取不同长度的n-gram特征
→ 最大池化
→ 全连接层
→ Softmax
```

TextCNN擅长识别局部关键词和短语，但长距离依赖能力较弱。

LSTM方案

```
文本
→ Embedding
→ LSTM依次读取
→ 末状态或所有状态平均池化
→ 全连接层
→ Softmax
```

适合考虑词序和上下文，但需要串行计算。

BERT方案

```
```

```
[CLS] 文本 [SEP]
→ BERT双向编码
→ 取得[CLS]上下文向量
→ Dropout
→ 全连接层
→ Softmax
```

适合语义复杂、上下文依赖明显的分类任务。

3.3 “中文文本分类系统”标准答案框架

> 文本分类的任务是根据文本内容或主题，将文本自动分到一个或多个预定义类别。系统首先收集带有类别标签的中文文本，进行去噪、分词或子词切分，并划分训练集、验证集和测试集。可采用BERT分类模型：在输入首尾加入 `[CLS]` 和 `[SEP]` ，经多层Transformer Encoder获得上下文表示，取 `[CLS]` 向量输入全连接层和Softmax，输出各类别概率。训练时采用交叉熵损失和Adam优化器更新模型参数，预测时选择概率最大的类别。最后使用准确率、精确率、召回率和F1评价系统。针对类别不平衡和领域数据不足问题，可采用类别加权、数据增强或领域微调。

六、题型二：分词、词性标注、NER和症状识别

这几类任务都可以统一建模为：

代码块

```
输入token序列
→ 输出同样长度的标签序列
```

PPT明确把分词、词性标注、命名实体识别等归为N-N序列预测，每个位置输出一个标签。

3.1 常见标签体系

分词：BMES

- B：词首；
- M：词中；
- E：词尾；
- S：单字成词。

例如：

代码块

```
自 然 语 言 很 重 要
B E B E S B E
```

命名实体识别：BIO

- B-PER：人名开始；
- I-PER：人名内部；
- B-LOC、I-LOC：地名；
- B-ORG、I-ORG：机构名；
- O：不属于实体。

命名实体识别包含两个目标：

1. 确定实体边界；
2. 确定实体类型。

PPT第7章把NER统一转换为BIO序列标注，并指出医疗领域还包括部位、疾病、症状、药物、检查和手术等实体。

3.2 模型方案

HMM方案

```
观测序列：句子中的字
隐藏状态：BIO或BMES标签
```

训练参数：

- 初始概率 (\pi)：句首出现每类标签的概率；
- 转移概率 (A)：一个标签转移到下一个标签的概率；
- 发射概率 (B)：某标签生成某个字的概率。

在标注语料中通过频数和最大似然估计得到参数，预测时使用Viterbi算法寻找概率最大的标签路径。

BiLSTM方案

```
字符或词Embedding
→ 正向LSTM＋反向LSTM
→ 每个位置的上下文向量
```

```
4    → 每个位置分类
5    → 标签序列
```

比HMM更能自动学习上下文特征。

BERT Token Classification

```
代码块
1    Token序列
2    → BERT双向上下文编码
3    → 每个token接全连接分类器
4    → 每个token的BIO标签
```

BERT的每个位置输出都可以用于NER、词性标注和关键词抽取。

可以进一步接CRF，用于学习标签之间的约束，避免出现：

```
代码块
1    O后直接出现I-PER
2    B-PER后错误接I-LOC
```

七、往年题：简述人名识别的方法，包括模型与如何训练参数

这道题最好写两部分。

3.1 基于规则和词典的人名识别

PPT给出的基本过程是：

```
代码块
1    利用姓氏作为触发信息
2    → 查询姓氏库和名字用字库
3    → 产生潜在人名
4    → 计算候选姓名概率
5    → 使用阈值和规则筛选
```

例如“孙中山”中：

- “孙”匹配姓氏库；

- “中、山”匹配常用名字用字；
- 根据姓和名用字的统计频率计算候选概率；
- 超过阈值则判定为姓名。

中文人名识别困难在于“张、江、于”等姓氏也可能是普通词，“建国、国庆、计划”等名字也可能是普通词。

3.2 基于HMM的序列标注方法

将字符标为：

```
代码块
1   B-PER：人名开始
2   I-PER：人名内部
3   O：非人名
```

训练过程：

1. 收集并人工标注人名语料；
2. 根据句首标签频数估计初始概率；
3. 根据相邻标签共现次数估计转移概率；
4. 根据标签与字符共现次数估计发射概率；
5. 对测试文本使用Viterbi算法，求最可能标签序列；
6. 合并连续的B-PER和I-PER得到人名。

最后使用精确率、召回率和F1评价。

八、往年题：设计医院电子病历症状识别系统

这题本质上不是文本分类，而是：

> **医疗领域命名实体识别，也就是序列标注。**

3.1 标签设计

只识别症状时：

- B-SYM：症状实体开始；
- I-SYM：症状实体内部；
- O：非症状。

例如：

```
伴 阵 发 性 腹 痛 、 恶 心
O B I I I I O B I
  └──阵发性腹痛──┘   └恶心┘
```

完整医疗NER还可增加：

- B-DIS/I-DIS：疾病；
- B-BODY/I-BODY：身体部位；
- B-DRUG/I-DRUG：药物；
- B-TEST/I-TEST：检查；
- B-OPERATION/I-OPERATION：手术。

PPT的医学语料示例正是以类似BIO标签标注疾病、检查和手术等实体。

3.2 数据准备

```
收集脱敏电子病历
→ 医学专家制定症状定义
→ 人工标注症状边界和类型
→ 双人标注并解决分歧
→ 划分训练、验证和测试集
```

3.3 模型方案

推荐写BERT：

```
电子病历
→ 中文BERT分词器
→ BERT双向编码
→ 每个token接全连接分类层
→ 输出B-SYM/I-SYM/O
→ 合并标签得到症状实体
```

训练时采用token级交叉熵损失，使用反向传播和Adam更新参数。

也可以使用：

```
代码块
1    BERT → CRF
```

让模型同时考虑上下文和标签转移是否合法。

3.4 一个容易得加分的点

例如：

```
代码块
1    无发热、无咳嗽
```

NER仍可能识别出“发热”和“咳嗽”是症状实体，但它们在当前病历中是**否定症状**。

因此可以增加第二阶段：

```
代码块
1    症状实体识别
2    → 根据“无、未见、否认”等上下文
3    → 判断症状为存在、否定或不确定
```

不要简单地因为前面有“无”就不识别“发热”；实体类型和症状是否存在是两个问题。

九、题型三：语义消歧

3.1 任务定义

语义消歧的目标是：

> 对一个具有多个义项的词，根据它所在的上下文，确定该词在当前句子中采用的是哪一个义项。

例如“打”可以表示敲击、购买、编织、拨打电话等。

PPT指出，词义不同通常对应不同上下文，基本上下文特征包括周围词、词性和相对位置。

3.2 基于有监督学习的方案

数据

收集目标多义词的语料，并由人工标注每次出现时对应的义项：

```
打鼓        → 敲击
打毛衣      → 编织
打电话      → 拨打
打酱油      → 购买
```

特征

提取目标词附近窗口中的：

- 上下文词；
- 词性；
- 相对位置；
- 前后搭配词；
- 或直接使用BERT上下文向量。

模型

PPT中的方案可以采用贝叶斯分类：

$$
[
\hat{s}
\arg\max_{s_i}
P(s_i)\prod_{v_k\in C}P(v_k\mid s_i)
]
$$

其中：

- s_i：候选义项；
- C：当前上下文；
- $P(s_i)$：义项先验概率；
- $P(v_k\mid s_i)$：某上下文词在义项 s_i 下出现的概率。

这些概率可从带义项标注的语料中使用最大似然估计得到。

也可采用：

```
```

```
目标词上下文
→ BERT
→ 目标词位置的上下文向量
→ 全连接层
→ 各义项概率
```

预测

计算所有候选义项的概率，选择最大者。

3.3 基于词典或知识库的方案

不需要大量带词义标注的语料，而需要WordNet或HowNet等知识库。

处理步骤：

代码块

```
读取目标词的所有候选义项
→ 取得每个义项的定义、同义词、例句和语义关系
→ 提取目标词当前上下文
→ 比较上下文与各义项描述的词汇重叠度或语义相似度
→ 选择相似度最大的义项
```

例如“食堂没地方”中，“食堂”与“空间、建筑物”等相关概念可以帮助判断“地方”表示空间，而不是地理地区。WordNet按词义组织词汇，并提供同义、反义、上下位和部分—整体等语义关系，可用于词义消歧、语义推理和相似度计算。

3.4 两种方法的对比

十、题型四：文本特征提取的特点、缺陷与解决方案

回答这类题，最好按文本表示的发展过程写。

3.1 词袋或TF-IDF表示

特点：

- 每个维度对应一个词；
- 文本表示成词频或词权重向量；
- 简单、直观、可解释；
- 适合朴素贝叶斯、逻辑回归和SVM。

缺陷：

- 词表大，向量高维稀疏；
- 忽略词序；
- 无法表示同义关系；
- “好”和“优秀”被视为完全不同；
- 多义词在不同句子中使用相同维度；
- 未登录词难以处理。

改进：

- 使用n-gram保留部分局部词序；
- 使用TF-IDF降低高频无效词影响；
- 进行停用词过滤和特征选择；
- 使用Word2Vec等稠密词向量；
- 使用BERT获得上下文化表示。

3.2 Word2Vec静态词向量

特点：

- 低维稠密；
- 由上下文预测任务自动学习；
- 语义相似的词向量较接近；
- 能缓解词袋模型中的语义鸿沟。

缺陷：

- 一个词始终只有一个固定向量；
- 无法解决多义词；

- 单个词向量还不能直接完整表示整个句子；
- 简单求和或平均仍会丢失词序。

解决：

- 使用CNN、LSTM对词向量序列建模；
- 使用ELMo、BERT等动态上下文表示。

PPT将Word2Vec、GloVe归为静态词向量，将ELMo、BERT和RoBERTa归为动态上下文表示；动态表示会根据当前句子实时计算同一个词的不同向量。

3.3 BERT动态上下文表示

特点：

- 同一个词在不同句子中得到不同向量；
- 同时利用左右上下文；
- 能表示词义、句法和上下文关系；
- 可通过微调适配分类、NER等任务。

缺陷：

- 参数量和计算量大；
- 需要较多显存；
- 推理速度慢；
- 领域差异大时仍需微调。

解决：

- 使用蒸馏模型；
- 冻结部分参数；
- LoRA等参数高效微调；
- 模型量化。

所以“特点—缺陷—解决方案”可以总结成：

代码块

```
词袋：简单但无语义
→ Word2Vec：有语义但无上下文变化
→ CNN/LSTM：进一步学习局部和顺序特征
→ BERT：动态建模完整上下文
```

十一、往年题：词袋模型和Word2Vec有何异同

相同点

- 都把自然语言转换成数值向量；
- 都需要构建或使用词表；
- 都可以作为文本分类等下游模型的输入；
- 都利用语料中的词语统计信息。

不同点

最容易漏掉的一点是：

> Word2Vec虽然比词袋模型更有语义，但它仍是静态词向量，也不能根据句子自动区分多义词。

十二、题型五：机器翻译、摘要和对话系统

这三类都可以归入：

```
输入序列 → 输出序列
```

PPT中Seq2Seq使用两个RNN分别作为Encoder和Decoder；Encoder负责读取输入，Decoder根据语义表示逐步生成输出。

3.1 通用实现模板

```
输入文本
→ Tokenization和Embedding
```

```
→ Encoder理解输入
→ 获得上下文表示
→ Decoder逐token生成
→ Linear＋Softmax
→ 解码得到输出文本
```

训练时：

代码块

```
输入与目标文本对
→ Teacher Forcing
→ 计算每个输出位置的交叉熵
→ 反向传播更新参数
```

预测时：

代码块

```
输入开始标记
→ 预测第一个token
→ 将其加入Decoder输入
→ 继续生成
→ 直到结束符
```

3.2 机器翻译

数据：中英平行语料。

模型可选：

- RNN/LSTM Encoder-Decoder；
- Seq2Seq＋Attention；
- Transformer Encoder-Decoder。

Attention使Decoder在生成每个目标词时动态关注不同源语言位置。

3.3 文本摘要

- 抽取式摘要：从原文中选择重要句子；
- 生成式摘要：Encoder理解原文，Decoder生成新的简短文本。

可使用Transformer Encoder-Decoder或微调生成式大模型。

3.4 对话系统

基本流程：

```
用户输入
→ 意图识别和实体抽取
→ 对话状态或知识检索
→ 回复生成
```

任务型对话更重视意图和槽位；开放域对话更适合Decoder-only大模型。

3.5 模型选择原则

- 需要理解后分类：BERT；
- 需要给每个词标标签：BERT Token Classification；
- 输入和输出都是文本且内容不同：Encoder-Decoder Transformer；
- 根据前文自由续写：GPT、Qwen等Decoder-only模型。

十三、题型六：信息检索、问答和RAG

PPT中还可能形成综合题的内容包括：

- 信息检索；
- 问答系统；
- 企业知识库问答；
- RAG；
- 知识图谱辅助问答。

RAG系统模板

RAG分为离线索引和在线查询两个阶段。

离线阶段

```
收集外部文档
→ 清洗并分块
→ 使用Embedding模型转成向量
→ 存入向量数据库
```

```
5    → 建立索引
```

在线阶段

代码块

```
1    用户问题
2    → 问题向量化
3    → 从向量数据库检索Top-K文档块
4    → 将文档块和问题拼接成Prompt
5    → 大模型生成答案
```

PPT给出的完整流程正是“文档加载—分块—向量化—相似度检索—上下文注入—生成答案”。

可写出的改进：

- 向量检索＋BM25混合检索；
- 对候选文档重排序；
- 调整文本块大小和重叠；
- 要求模型只依据检索资料回答；
- 输出引用来源，减少幻觉。

什么时候选RAG，什么时候选微调

- **知识经常更新、强调准确事实**：RAG；
- **希望模型学会分类格式、回答风格或领域任务**：微调；
- **文本分类和NER**：通常微调BERT，而不是RAG；
- **企业文档问答**：通常RAG，而不是把全部文档直接拿去微调。

十四、题型七：语义解析、信息抽取和知识图谱

PPT中“语义解析”没有像分词或Transformer那样集中展开，所以答题时要先明确输出结构。

3.1 意图与槽位解析

例如：

代码块

```
1    输入：帮我订明天去北京的机票
2    输出：
```

```
意图 = 预订机票
日期 = 明天
目的地 = 北京
```

实现：

代码块

```
BERT共享编码
├→ [CLS]接分类层，预测意图
└→ 每个token接分类层，预测槽位BIO标签
```

3.2 知识图谱信息抽取

例如：

代码块

```
姚明生于上海
→（姚明，出生地，上海）
```

实现：

代码块

```
文本
→ NER提取实体
→ 关系分类模型判断实体关系
→ 构成三元组
→ 消歧和实体链接
→ 存入知识图谱
```

知识图谱可以组织实体、属性和关系，并用于检索、推理和问答。

十五、六道往年题分别该抓什么

3.1 文本分类主要任务和系统方案

得分点：

代码块

```
任务定义
类别类型
标注语料
文本表示
分类模型
训练方法
预测过程
评价指标
```

推荐主线：

代码块

```
中文文本 → BERT → [CLS] → 全连接 → Softmax
```

3.2 有监督和词典语义消歧

有监督：

代码块

```
带义项标签语料
→ 上下文特征
→ 贝叶斯/BERT分类
→ 选择概率最大义项
```

词典：

代码块

```
WordNet/HowNet候选义项
→ 定义和例句
→ 与当前上下文计算重叠或相似度
→ 选择得分最大义项
```

必须明确两者分别依赖：

代码块

```
有监督：标注语料
词典法：知识库
```

3.3 文本分类中特征提取特点、缺陷、解决方案

按发展路线写：

```
词袋/TF-IDF
→ 高维稀疏、忽略词序和语义
→ Word2Vec稠密语义向量
→ 仍不能解决多义词和句子结构
→ CNN/LSTM建模局部和序列信息
→ BERT获得动态上下文表示
```

3.4 词袋模型与Word2Vec异同

不要只写“一个稀疏、一个稠密”，还要写：

- 文档表示和词表示的区别；
- 维度；
- 是否需要训练；
- 是否包含语义；
- 是否解决多义词；
- 如何送入下游模型。

3.5 人名识别方法及参数训练

推荐写：

```
规则词典法
+
HMM序列标注法
```

HMM必须出现：

```
```

```
π、A、B
最大似然估计
Viterbi解码
```

3.6 症状识别系统

必须出现：

代码块

```
序列标注
BIO标签
训练语料
HMM/BERT模型
逐token输出
实体合并
P/R/F1
```

加分点：

代码块

```
“无发热”中的“发热”仍是症状实体，
再单独进行肯定/否定状态判断。
```

十六、复习时重点看哪些内容

按照综合题的重要程度，可以这样排。

第一重点：任务如何建模

- 一段文本一个标签；
- 每个token一个标签；
- 输入序列生成输出序列；
- 查询后检索并生成；
- 多义词选择一个义项。

第二重点：表示方法的发展

代码块

```
one-hot
→ 词袋/TF-IDF
→ Word2Vec
→ CNN/RNN/LSTM
→ BERT动态上下文表示
```

第三重点：分词与NER

- 歧义切分；
- 未登录词；
- BMES和BIO；
- 人名、地名和机构名；
- 医疗实体；
- P、R、F1。

第四重点：HMM

- 观测状态与隐藏状态；
- (\pi,A,B)；
- 最大似然参数估计；
- Viterbi最优标签序列；
- 如何应用于分词和NER。

第五重点：模型与任务匹配

- TextCNN：局部短语；
- LSTM：顺序和长期信息；
- BERT：理解、分类、NER；
- Transformer Encoder-Decoder：翻译、摘要；
- Decoder-only：续写和对话。

第六重点：新型应用方案

- 预训练＋任务微调；
- LoRA；
- RAG；

- 模型量化。

最后记住一句总原则：

> **综合题不要从“我会哪些模型”出发，而要从“输入是什么、输出是什么、标签怎么设计”出发，再依次写数据、表示、模型、训练、预测和评价。**